

Elixir Mastery Lab Guide

Hands-On Labs with Complete Solutions

Every lab is a real Mix project

Tested on Elixir 1.20.4 | Erlang/OTP 29

Table of Contents

Day 1 Labs — Foundations and Data	8
Lab 1 — First Mix Project	9
Objective	9
Steps	9
Requirements	9
Hints	9
Lab 1 Solution	9
Lab 2 — Types and Immutability	12
Objective	12
Steps	12
Requirements	12
Hints	12
Lab 2 Solution	12
Lab 3 — Collections	16
Objective	16
Steps	16
Requirements	16
Hints	16
Lab 3 Solution	16
Lab 4 — Pattern Matching	20
Objective	20
Steps	20
Requirements	20
Hints	20
Lab 4 Solution	20

Lab 5 — Modules, Functions and Pipes 25

Objective	25
Steps	25
Requirements	25
Hints	25
Lab 5 Solution	25
End of Day 1 review	29

Day 2 Labs — The Functional Core 30

Lab 6 — Clauses and Guards 31

Objective	31
Steps	31
Requirements	31
Hints	31
Lab 6 Solution	31

Lab 7 — Control Flow 36

Objective	36
Steps	36
Requirements	36
Hints	36
Lab 7 Solution	36

Lab 8 — Enum Pipelines 42

Objective	42
Steps	42
Requirements	42
Hints	42
Lab 8 Solution	42

Lab 9 — Stream and Comprehensions 48

Objective	48
---------------------	----

Steps	48
Requirements	48
Hints	48
Lab 9 Solution	48
Lab 10 — Recursion	54
Objective	54
Steps	54
Requirements	54
Hints	54
Lab 10 Solution	54
End of Day 2 review	59
Day 3 Labs — Building Real Projects	60
Lab 11 — A Real Mix Project	61
Objective	61
Steps	61
Requirements	61
Hints	61
Lab 11 Solution	62
Lab 12 — Structs, Protocols and Behaviours	74
Objective	74
Steps	74
Requirements	74
Hints	74
Lab 12 Solution	74
Lab 13 — Error Handling	83
Objective	83
Steps	83
Requirements	83

Hints	83
Lab 13 Solution	83
Lab 14 — Testing with ExUnit	89
Objective	89
Steps	89
Requirements	89
Hints	89
Lab 14 Solution	89
Lab 15 — Documentation, Typespecs and Quality Tooling	94
Objective	94
Steps	94
Requirements	94
Hints	94
Lab 15 Solution	94
End of Day 3 review	97
Day 4 Labs — Concurrency and OTP	99
Lab 16 — A Key-Value Store as a Raw Process	100
Objective	100
Requirements	100
Hints	100
Lab 16 Solution	100
Lab 17 — Parallel Health Checks	105
Objective	105
Requirements	105
Hints	105
Lab 17 Solution	105
Lab 18 — A Session GenServer	109

Objective	109
Requirements	109
Hints	109
Lab 18 Solution	109
Lab 19 — Supervising the Sessions	112
Objective	112
Requirements	112
Hints	112
Lab 19 Solution	112
Lab 20 — Per-User Sessions with Registry, DynamicSupervisor and ETS	114
Objective	114
Requirements	114
Hints	114
Lab 20 Solution	114
End of Day 4 review	120
Day 5 Labs — Macros, Phoenix, Ecto, LiveView and the Capstone	122
Lab 21 — Macros	123
Objective	123
Requirements	123
Hints	123
Lab 21 Solution	123
Lab 22 — Phoenix: Adding Endpoints	129
Objective	129
Setup	129
Requirements	129
Hints	129
Lab 22 Solution	129

Lab 23 — Ecto: Business Logic in the Context **133**

Objective	133
Requirements	133
Hints	133
Lab 23 Solution	133

Lab 24 — LiveView **137**

Objective	137
Setup	137
Requirements	137
Hints	137
Lab 24 Solution	137

Capstone — Ticketing Under a Stampede **141**

The brief	141
Requirements by day	141
The acceptance test	141
Design review questions for the paper sketch	141
Capstone Solution	142
End of Day 5 review	149

Day 1 Labs — Foundations and Data

Five labs covering Modules 01 to 05. Each lab is a real Mix project. Work in pairs and rotate the driver at every lab.

Create a scratch directory to hold all lab projects:

```
mkdir elixir-labs  
cd elixir-labs
```

Lab 1 — First Mix Project

After Module 01 · 25 minutes

Objective

Create, compile, test and explore a Mix project, and become comfortable moving between the editor, `mix test` and `iex -S mix`.

Steps

1. Create a project called `unit_converter`.
2. Replace `lib/unit_converter.ex` with a module that converts Celsius to Fahrenheit and back.
3. Write a doctest for each function.
4. Write two ExUnit tests.
5. Run the suite and confirm the doctests execute.
6. Start `iex -S mix`, call both functions, change one, and reload with `recompile()`.

Requirements

- `UnitConverter.c_to_f/1` converts Celsius to Fahrenheit
- `UnitConverter.f_to_c/1` converts Fahrenheit to Celsius, rounded to one decimal place
- Both have a `@doc` containing a runnable `iex>` example
- `mix test` reports at least four passing assertions

Hints

- `Float.round/2` rounds to a given number of decimal places
- Remember `10 / 2` is `5.0`, so all of this is float arithmetic
- The doctest expected value must match exactly what IEx prints, including `.0`

Lab 1 Solution

```
mix new unit_converter
cd unit_converter
```

```
# lib/unit_converter.ex
defmodule UnitConverter do
  @moduledoc """
  Temperature conversions between Celsius and Fahrenheit.
  """

  @doc """
```

```

Converts Celsius to Fahrenheit.

## Examples

iex> UnitConverter.c_to_f(100)
212.0

iex> UnitConverter.c_to_f(0)
32.0

"""
def c_to_f(celsius) do
  celsius * 9 / 5 + 32
end

@doc """
Converts Fahrenheit to Celsius, rounded to one decimal place.

## Examples

iex> UnitConverter.f_to_c(212)
100.0

iex> UnitConverter.f_to_c(98.6)
37.0

"""
def f_to_c(fahrenheit) do
  ((fahrenheit - 32) * 5 / 9)
  |> Float.round(1)
end
end

```

```

# test/unit_converter_test.exs
defmodule UnitConverterTest do
  use ExUnit.Case
  doctest UnitConverter

  test "freezing point round trips" do
    assert UnitConverter.c_to_f(0) == 32.0
    assert UnitConverter.f_to_c(32) == 0.0
  end

  test "body temperature" do
    assert UnitConverter.f_to_c(98.6) == 37.0
  end

  test "negative temperatures" do
    assert UnitConverter.c_to_f(-40) == -40.0
  end
end

```

```

mix test

```

```

Compiling 1 file (.ex)
Generated unit_converter app
Running ExUnit with seed: 780401, max_cases: 24

```

```
.....  
Finished in 0.02 seconds (0.00s async, 0.02s sync)  
  
Result: 7 passed (4 doctests, 3 tests)
```

Seven assertions: four doctests taken from the `@doc` blocks, three ordinary tests.

Verification in IEx

```
iex -S mix
```

```
iex> UnitConverter.c_to_f(37)  
98.6  
  
iex> UnitConverter.f_to_c(100)  
37.8  
  
iex> recompile()  
:ok
```

Discussion point

`c_to_f(0)` returns `32.0` and not `32`. Ask the room why, and what would have to change to return an integer. The answer is `/` and `div/2`, and it leads naturally into why a doctest that expects `32` would fail.

Lab 2 — Types and Immutability

After Module 02 · 25 minutes

Objective

Build a module that validates and normalises user input, exercising atoms, strings, booleans and the truthiness rules.

Steps

1. Create a project called `validator`.
2. Write `Validator.normalize_status/1` that accepts a string and returns an atom, safely.
3. Write `Validator.display_name/1` that picks the first non-nil, non-empty option.
4. Write tests covering the safe and unsafe cases.

Requirements

- `normalize_status/1` returns `{:ok, atom}` for "active", "inactive" and "pending"
- It returns `{:error, :unknown_status}` for anything else
- It must **not** use `String.to_atom/1`
- `display_name/1` takes a map and returns the nickname, then the full name, then "Anonymous"
- An empty string counts as missing

Hints

- `String.to_existing_atom/1` raises if the atom does not exist, so the atoms must already be referenced somewhere in the module
- The `||` operator returns the first truthy operand
- An empty string is truthy in Elixir, so `""` needs handling of its own

Lab 2 Solution

```
mix new validator
cd validator
```

```
# lib/validator.ex
defmodule Validator do
  @moduledoc """
  Input normalisation that never creates atoms from untrusted strings.
  """

  # Listing the atoms here guarantees they exist in the atom table before
  # String.to_existing_atom/1 is ever called.
```

```

@allowed_statuses [:active, :inactive, :pending]

@doc """
Converts a status string into a known atom.

## Examples

    iex> Validator.normalize_status("active")
    {:ok, :active}

    iex> Validator.normalize_status("deleted")
    {:error, :unknown_status}
"""
def normalize_status(status) when is_binary(status) do
  atom = safe_atom(status)

  if atom in @allowed_statuses do
    {:ok, atom}
  else
    {:error, :unknown_status}
  end
end

def normalize_status(_, do: {:error, :unknown_status})

@doc """
Picks the first usable display name.

## Examples

    iex> Validator.display_name(%{nickname: "Pri", full_name: "Priya R"})
    "Pri"

    iex> Validator.display_name(%{nickname: "", full_name: "Priya R"})
    "Priya R"

    iex> Validator.display_name(%{})
    "Anonymous"
"""
def display_name(user) when is_map(user) do
  present(user[:nickname]) || present(user[:full_name]) || "Anonymous"
end

defp safe_atom(status) do
  String.to_existing_atom(status)
rescue
  ArgumentError -> nil
end

defp present(nil), do: nil
defp present(""), do: nil
defp present(value), do: value
end

```

Three things worth walking through with the room.

`@allowed_statuses` does double duty. It is the whitelist, and it is also what puts those three atoms into the atom table at compile time, so `String.to_existing_atom/1` can find them.

`safe_atom/1` uses a function-level `rescue`. A `def` body can carry `rescue`, `catch` and `after` clauses without an explicit `try`. Module 13 covers this properly.

`present/1` has three clauses that turn "missing or empty" into `nil`, so `||` can then do the work. Without it, `""` would win, because an empty string is truthy.

```
# test/validator_test.exs
defmodule ValidatorTest do
  use ExUnit.Case
  doctest Validator

  describe "normalize_status/1" do
    test "accepts known statuses" do
      assert {:ok, :active} = Validator.normalize_status("active")
      assert {:ok, :inactive} = Validator.normalize_status("inactive")
      assert {:ok, :pending} = Validator.normalize_status("pending")
    end

    test "rejects an unknown status" do
      assert {:error, :unknown_status} = Validator.normalize_status("deleted")
    end

    test "rejects a string that is not an existing atom at all" do
      assert {:error, :unknown_status} =
        Validator.normalize_status("definitely_not_an_atom_anywhere")
    end

    test "rejects a non-string" do
      assert {:error, :unknown_status} = Validator.normalize_status(42)
    end
  end

  describe "display_name/1" do
    test "prefers the nickname" do
      assert Validator.display_name(%{nickname: "Pri", full_name: "Priya R"}) == "Pri"
    end

    test "treats an empty nickname as missing" do
      assert Validator.display_name(%{nickname: "", full_name: "Priya R"}) == "Priya R"
    end

    test "falls back to Anonymous" do
      assert Validator.display_name(%{}) == "Anonymous"
      assert Validator.display_name(%{nickname: nil, full_name: ""}) == "Anonymous"
    end
  end
end
```

```
mix test
```

```
Running ExUnit with seed: 132877, max_cases: 24
```

```
.....
Finished in 0.03 seconds (0.00s async, 0.03s sync)
```

```
Result: 12 passed (5 doctests, 7 tests)
```

`describe` groups related tests and prefixes their names in the output. It is covered in Module 14.

Discussion point

Ask what would happen if `safe_atom/1` used `String.to_atom/1` instead. The answer: a client sending ten thousand distinct status strings would add ten thousand permanent entries to the atom table, and a determined client would crash the node. This is a real, exploitable vulnerability, and it has appeared in shipped Elixir code.

Lab 3 — Collections

After Module 03 · 30 minutes

Objective

Model a shopping cart using the right collection for each part of the problem, and practise the `Map` API.

Steps

1. Create a project called `cart`.
2. Represent a cart as a map of SKU to quantity.
3. Implement add, remove, total quantity, and a summary.
4. Add a function that reads a nested configuration map.

Requirements

- `Cart.new/0` returns an empty cart
- `Cart.add/3` adds a quantity, accumulating when the SKU is already present
- `Cart.remove/2` removes a SKU entirely and does not fail if it is absent
- `Cart.total_items/1` returns the sum of all quantities
- `Cart.summary/1` returns a list of `{sku, quantity}` tuples sorted by SKU
- `Cart.currency/1` reads `[ :store, :settings, :currency ]` from a nested map, defaulting to `"INR"`

Hints

- `Map.update/4` handles present and absent keys in one call
- `Map.delete/2` is already safe on missing keys
- `Enum.sum/1` and `Map.values/1` combine well
- `get_in/2` returns `nil` for a missing path

Lab 3 Solution

```
mix new cart
cd cart
```

```
# lib/cart.ex
defmodule Cart do
  @moduledoc """
  A shopping cart represented as a map of SKU to quantity.
  """

  @default_currency "INR"
```



```

@doc """
Returns an empty cart.

## Examples

    iex> Cart.new()
    %{}

"""
def new, do: %{}

@doc """
Adds `quantity` units of `sku`, accumulating with any existing quantity.

## Examples

    iex> Cart.new() |> Cart.add("SKU-1", 2) |> Cart.add("SKU-1", 3)
    %{ "SKU-1" => 5}

"""
def add(cart, sku, quantity) when is_integer(quantity) and quantity > 0 do
  Map.update(cart, sku, quantity, &(&1 + quantity))
end

@doc """
Removes `sku` from the cart. Missing SKUs are ignored.

## Examples

    iex> Cart.new() |> Cart.add("SKU-1", 2) |> Cart.remove("SKU-1")
    %{}

    iex> Cart.remove(%{}, "NOPE")
    %{}

"""
def remove(cart, sku), do: Map.delete(cart, sku)

@doc """
Returns the total number of items across all SKUs.

## Examples

    iex> Cart.total_items(%{"A" => 2, "B" => 3})
    5

"""
def total_items(cart) do
  cart
  |> Map.values()
  |> Enum.sum()
end

@doc """
Returns `{sku, quantity}` tuples sorted by SKU.

## Examples

    iex> Cart.summary(%{"B" => 1, "A" => 2})
    [{"A", 2}, {"B", 1}]

```

```

"""
def summary(cart) do
  cart
  |> Enum.sort_by(fn {sku, _quantity} -> sku end)
end

@doc """
Reads the configured currency, falling back to the default.

## Examples

    iex> Cart.currency(%{store: %{settings: %{currency: "USD"}}})
    "USD"

    iex> Cart.currency(%{})
    "INR"
"""
def currency(config) do
  get_in(config, [:store, :settings, :currency]) || @default_currency
end
end

```

Two details to highlight.

`add/3` has a guard: `when is_integer(quantity) and quantity > 0`. Calling `Cart.add(cart, "X", -1)` raises a `FunctionClauseError` rather than silently corrupting the cart. Guards are Module 06.

`summary/1` relies on `Enum.sort_by/2` because **maps have no order**. Returning `Map.to_list/1` directly would produce an order that looks stable in testing and changes in production.

```

# test/cart_test.exs
defmodule CartTest do
  use ExUnit.Case
  doctest Cart

  setup do
    cart =
      Cart.new()
      |> Cart.add("SKU-1", 2)
      |> Cart.add("SKU-2", 5)

    {:ok, cart: cart}
  end

  test "accumulates repeated SKUs", %{cart: cart} do
    cart = Cart.add(cart, "SKU-1", 3)
    assert cart["SKU-1"] == 5
  end

  test "removing a missing SKU is a no-op", %{cart: cart} do
    assert Cart.remove(cart, "NOPE") == cart
  end

  test "totals all quantities", %{cart: cart} do
    assert Cart.total_items(cart) == 7
  end
end

```

```
test "summary is sorted by SKU", %{cart: cart} do
  assert Cart.summary(cart) == [{"SKU-1", 2}, {"SKU-2", 5}]
end

test "rejects a non-positive quantity", %{cart: cart} do
  assert_raise FunctionClauseError, fn -> Cart.add(cart, "SKU-3", 0) end
end

test "adding does not mutate the original cart", %{cart: cart} do
  _updated = Cart.add(cart, "SKU-1", 100)
  assert cart["SKU-1"] == 2
end
end
```

```
mix test
```

```
Running ExUnit with seed: 610338, max_cases: 24

.....
Finished in 0.04 seconds (0.00s async, 0.04s sync)

Result: 14 passed (8 doctests, 6 tests)
```

The last test is the one that matters most conceptually. It proves that `Cart.add/3` returned a new map and left the original alone. Point at it.

`setup` runs before every test and its return value is merged into the test context, which is then destructured in the test head with `%{cart: cart}`.

Lab 4 — Pattern Matching

After Module 04 · 30 minutes

Objective

Write an HTTP response handler entirely with pattern matching, no `if` statements at all.

Steps

1. Create a project called `http_handler`.
2. Handle responses of the form `{:ok, %{status: integer, body: string}}` and `{:error, reason}`.
3. Add a correlation-ID check using the pin operator.

Requirements

- `HttpHandler.handle/1` returns:
 - `{:ok, body}` for status 200
 - `{:ok, :no_content}` for status 204
 - `{:error, :not_found}` for status 404
 - `{:error, {:server_error, status}}` for any status from 500 to 599
 - `{:error, {:unexpected_status, status}}` for anything else
 - `{:error, {:transport, reason}}` for `{:error, reason}`
- `HttpHandler.verify_correlation/2` takes an expected ID and a response, and returns the payload only when the IDs match, using `^`
- The module must contain no `if`, `cond` or `case`

Hints

- A `when` guard can express the 500 to 599 range
- Clause order matters: specific before general
- `^expected_id` inside a tuple pattern is the whole trick for the second function

Lab 4 Solution

```
mix new http_handler
cd http_handler
```

```
# lib/http_handler.ex
defmodule HttpHandler do
```

```

@moduledoc """
Turns raw HTTP results into tagged tuples using pattern matching alone.
"""

@doc """
Classifies an HTTP result.

## Examples

    iex> HttpHandler.handle({:ok, %{status: 200, body: "hello"}})
    {:ok, "hello"}

    iex> HttpHandler.handle({:ok, %{status: 404, body: ""}})
    {:error, :not_found}

    iex> HttpHandler.handle({:error, :timeout})
    {:error, {:transport, :timeout}}

"""
def handle({:ok, %{status: 200, body: body}}), do: {:ok, body}
def handle({:ok, %{status: 204}}), do: {:ok, :no_content}
def handle({:ok, %{status: 404}}), do: {:error, :not_found}

def handle({:ok, %{status: status}}) when status >= 500 and status <= 599 do
  {:error, {:server_error, status}}
end

def handle({:ok, %{status: status}}), do: {:error, {:unexpected_status, status}}
def handle({:error, reason}), do: {:error, {:transport, reason}}

@doc """
Returns the payload only if the response carries the expected correlation ID.

## Examples

    iex> HttpHandler.verify_correlation("req-1", {:ok, "req-1", %{total: 10}})
    {:ok, %{total: 10}}

    iex> HttpHandler.verify_correlation("req-1", {:ok, "req-9", %{total: 10}})
    {:error, :correlation_mismatch}

"""
def verify_correlation(expected_id, response) do
  case response do
    {:ok, ^expected_id, payload} -> {:ok, payload}
    _ -> {:error, :correlation_mismatch}
  end
end

@doc """
The same check written without `case`, by repeating the variable in the head.

Two occurrences of the same variable in one pattern must be equal, so this
clause only matches when the response ID equals the expected ID.

## Examples

    iex> HttpHandler.verify_correlation_by_head("req-1", {:ok, "req-1", %{total: 10}})
    {:ok, %{total: 10}}

    iex> HttpHandler.verify_correlation_by_head("req-1", {:ok, "req-9", %{total: 10}})

```

```

{:error, :correlation_mismatch}

"""
def verify_correlation_by_head(expected_id, {:ok, expected_id, payload}), do: {:ok, payload}
def verify_correlation_by_head(_expected_id, _response), do: {:error, :correlation_mismatch}
end

```

The mistake almost everyone makes here

The obvious thing to write is this, and it does not compile:

```

# Does NOT compile
def verify_correlation(expected_id, {:ok, ^expected_id, payload}), do: {:ok, payload}

```

```

error: undefined variable ^expected_id. No variable "expected_id" has been defined
before the current pattern

```

Read the message carefully, because it states the rule exactly. **All parameters in a function head are matched at the same time, not left to right.** When the second parameter is being matched, `expected_id` has not been bound yet, so there is nothing to pin.

There are two correct forms, and both are shown in the solution above.

Bind first, then pin in a case. Inside the function body, `expected_id` is already bound, so `^expected_id` works normally. This is the form to reach for by default, because the pin makes the intent explicit to a reader.

Repeat the variable in the head. `def f(expected_id, {:ok, expected_id, payload})` compiles and does the right thing, because two occurrences of the same variable in one pattern must be equal. It is shorter, and it is easier for a reader to miss.

Whichever you choose, removing the check entirely leaves a function that accepts every response silently. That is the bug the pin operator exists to prevent.

```

# test/http_handler_test.exs
defmodule HttpHandlerTest do
  use ExUnit.Case
  doctest HttpHandler

  describe "handle/1" do
    test "200 returns the body" do
      assert {:ok, "hello"} = HttpHandler.handle({:ok, %{status: 200, body: "hello"}})
    end

    test "204 returns no_content" do
      assert {:ok, :no_content} = HttpHandler.handle({:ok, %{status: 204, body: ""}})
    end

    test "404 returns not_found" do
      assert {:error, :not_found} = HttpHandler.handle({:ok, %{status: 404, body: ""}})
    end

    test "5xx returns a server error with the status" do
      assert {:error, {:server_error, 500}} = HttpHandler.handle({:ok, %{status: 500, body: ""}})
    end
  end
end

```

```

    assert {:error, {:server_error, 503}} = HttpHandler.handle({:ok, %{status: 503, body:
      ""}})
  end

  test "418 is unexpected" do
    assert {:error, {:unexpected_status, 418}} =
      HttpHandler.handle({:ok, %{status: 418, body: ""}})
  end

  test "transport failures are tagged" do
    assert {:error, {:transport, :econnrefused}} =
      HttpHandler.handle({:error, :econnrefused})
  end

  test "extra keys in the response map are ignored" do
    response = {:ok, %{status: 200, body: "hi", headers: [], took_ms: 12}}
    assert {:ok, "hi"} = HttpHandler.handle(response)
  end
end

describe "verify_correlation/2" do
  test "accepts a matching ID" do
    assert {:ok, %{total: 10}} =
      HttpHandler.verify_correlation("req-1", {:ok, "req-1", %{total: 10}})
  end

  test "rejects a mismatched ID" do
    assert {:error, :correlation_mismatch} =
      HttpHandler.verify_correlation("req-1", {:ok, "req-9", %{total: 10}})
  end

  test "rejects a malformed response" do
    assert {:error, :correlation_mismatch} =
      HttpHandler.verify_correlation("req-1", {:error, :timeout})
  end
end

describe "verify_correlation_by_head/2" do
  test "accepts a matching ID" do
    assert {:ok, %{total: 10}} =
      HttpHandler.verify_correlation_by_head("req-1", {:ok, "req-1", %{total: 10}})
  end

  test "rejects a mismatched ID" do
    assert {:error, :correlation_mismatch} =
      HttpHandler.verify_correlation_by_head("req-1", {:ok, "req-9", %{total: 10}})
  end
end
end

```

```

mix test

```

```

Running ExUnit with seed: 689528, max_cases: 24
.....
Finished in 0.08 seconds (0.00s async, 0.08s sync)

Result: 19 passed (7 doctests, 12 tests)

```

Discussion point

The test "extra keys in the response map are ignored" demonstrates partial map matching. Ask the room what would happen if the response were a tuple instead of a map, and why that difference exists. The answer is that tuples have a fixed, meaningful size, so matching them exactly is the correct default; maps are open records, so partial matching is the correct default.

Lab 5 — Modules, Functions and Pipes

After Module 05 · 30 minutes

Objective

Build a small log-line analyser as a pipeline, using private helpers, the capture operator and module attributes.

Steps

1. Create a project called `log_analyzer`.
2. Parse lines of the form `LEVEL|timestamp|message`.
3. Count lines by level, find errors, and produce a report string.

Requirements

- `LogAnalyzer.parse_line/1` returns `{:ok, %{level: atom, timestamp: string, message: string}}` or `{:error, :malformed}`
- `LogAnalyzer.count_by_level/1` takes a list of raw lines and returns a map of level to count, ignoring malformed lines
- `LogAnalyzer.errors/1` returns the messages of `:error` lines only
- `LogAnalyzer.report/1` returns a multi-line summary string
- Levels are limited to `:debug`, `:info`, `:warn` and `:error`
- The parsing pipeline must use `|>` and at least one `&` capture

Hints

- `String.split(line, "|")` gives you the three parts, or something else if the line is malformed
- Match on the resulting three-element list
- `Enum.frequencies_by/2` counts by a derived key in one step
- Build the report with `Enum.map_join/3`

Lab 5 Solution

```
mix new log_analyzer
cd log_analyzer
```

```
# lib/log_analyzer.ex
defmodule LogAnalyzer do
  @moduledoc """
    Parses and summarises pipe-delimited log lines of the form:
```

```

    LEVEL|timestamp|message

"""

@levels %{"DEBUG" => :debug, "INFO" => :info, "WARN" => :warn, "ERROR" => :error}

@doc """
Parses one log line.

## Examples

iex> LogAnalyzer.parse_line("INFO|2026-01-05T10:00:00Z|started")
{:ok, %{level: :info, timestamp: "2026-01-05T10:00:00Z", message: "started"}}

iex> LogAnalyzer.parse_line("nonsense")
{:error, :malformed}

iex> LogAnalyzer.parse_line("TRACE|2026-01-05T10:00:00Z|x")
{:error, :malformed}

"""
def parse_line(line) when is_binary(line) do
  line
  |> String.trim()
  |> String.split("|", parts: 3)
  |> build_entry()
end

@doc """
Counts valid lines by level, ignoring malformed ones.

## Examples

iex> LogAnalyzer.count_by_level(["INFO|t|a", "ERROR|t|b", "INFO|t|c", "junk"])
%{info: 2, error: 1}

"""
def count_by_level(lines) do
  lines
  |> valid_entries()
  |> Enum.frequencies_by(& &1.level)
end

@doc """
Returns the messages of error-level lines, in order.

## Examples

iex> LogAnalyzer.errors(["INFO|t|ok", "ERROR|t|disk full"])
["disk full"]

"""
def errors(lines) do
  lines
  |> valid_entries()
  |> Enum.filter(&(&1.level == :error))
  |> Enum.map(& &1.message)
end

@doc """
Builds a human-readable summary.

```

```

"""
def report(lines) do
  counts = count_by_level(lines)
  error_messages = errors(lines)

  """
  Log report
  =====
  Lines parsed: #{counts |> Map.values() |> Enum.sum()}
  #{format_counts(counts)}
  Errors (#{length(error_messages)}):
  #{format_errors(error_messages)}
  """
end

# ----- private

defp valid_entries(lines) do
  lines
  |> Enum.map(&parse_line/1)
  |> Enum.flat_map(fn
    {:ok, entry} -> [entry]
    {:error, _} -> []
  end)
end

defp build_entry([raw_level, timestamp, message]) do
  case Map.fetch(@levels, raw_level) do
    {:ok, level} -> {:ok, %{level: level, timestamp: timestamp, message: message}}
    :error -> {:error, :malformed}
  end
end

defp build_entry(_, do: {:error, :malformed})

defp format_counts(counts) do
  counts
  |> Enum.sort_by(fn {level, _count} -> level end)
  |> Enum.map_join("\n", fn {level, count} -> "  #{level}: #{count}" end)
end

defp format_errors([]), do: "  (none)"

defp format_errors(messages) do
  Enum.map_join(messages, "\n", &("  - " <> &1))
end
end

```

Four techniques worth naming out loud:

`String.split("|", parts: 3)` limits the split to three parts, so a message containing a pipe character survives intact.

`Enum.flat_map/2` with a two-clause anonymous function is the idiomatic "filter and unwrap in one pass". Returning `[entry]` keeps the item; returning `[]` drops it.

`& &1.level` is the capture form for "the `:level` field of the argument". Note the space: `&1.level` without the leading `&` is not a function.

`Enum.map_join/3` maps and joins in one traversal, which is exactly what report building needs.

```
# test/log_analyzer_test.exs
defmodule LogAnalyzerTest do
  use ExUnit.Case
  doctest LogAnalyzer

  @lines [
    "INFO|2026-01-05T10:00:00Z|service started",
    "DEBUG|2026-01-05T10:00:01Z|cache warm",
    "ERROR|2026-01-05T10:00:02Z|disk full",
    "WARN|2026-01-05T10:00:03Z|retrying",
    "ERROR|2026-01-05T10:00:04Z|connection lost",
    "this line is garbage",
    "TRACE|2026-01-05T10:00:05Z|unsupported level"
  ]

  describe "parse_line/1" do
    test "parses a well-formed line" do
      assert {:ok, entry} = LogAnalyzer.parse_line("WARN|2026-01-05T10:00:00Z|slow query")
      assert entry.level == :warn
      assert entry.message == "slow query"
    end

    test "keeps pipes inside the message" do
      assert {:ok, entry} = LogAnalyzer.parse_line("INFO|t|a|b|c")
      assert entry.message == "a|b|c"
    end

    test "rejects a malformed line" do
      assert {:error, :malformed} = LogAnalyzer.parse_line("nope")
    end

    test "rejects an unknown level" do
      assert {:error, :malformed} = LogAnalyzer.parse_line("TRACE|t|x")
    end
  end

  test "counts by level and skips bad lines" do
    assert LogAnalyzer.count_by_level(@lines) == %{info: 1, debug: 1, error: 2, warn: 1}
  end

  test "collects error messages in order" do
    assert LogAnalyzer.errors(@lines) == ["disk full", "connection lost"]
  end

  test "report contains the counts and the errors" do
    report = LogAnalyzer.report(@lines)

    assert report =~ "Lines parsed: 5"
    assert report =~ "error: 2"
    assert report =~ "- disk full"
  end

  test "report handles an empty input" do
    report = LogAnalyzer.report([])

    assert report =~ "Lines parsed: 0"
    assert report =~ "(none)"
  end
end
```

```
end
```

```
mix test
```

```
Running ExUnit with seed: 634332, max_cases: 24
.....
Finished in 0.04 seconds (0.00s async, 0.04s sync)

Result: 13 passed (5 doctests, 8 tests)
```

`assert report =~ "error: 2"` uses the match operator `=~`, which tests whether a string contains a substring or matches a regex. It is the right assertion for generated text, because it does not break every time the formatting changes slightly.

Stretch goal

Add `LogAnalyzer.between/3` that filters entries to a timestamp range, and have the report accept an optional range. It forces a design conversation about whether the range belongs in the parsing pipeline or in a separate filtering step.

End of Day 1 review

Ask each pair to answer these out loud before they leave.

1. Show a line in your lab code where a value was transformed without anything being mutated.
2. Show a place where a pattern match replaced an `if`.
3. Where did you use a tagged tuple, and what would the alternative have been in Java or Go?
4. Which collection did you choose for the main data in Lab 3 and Lab 5, and why that one?

Day 2 Labs — The Functional Core

Five labs covering Modules 06 to 10. Work in the same `elixir-labs` directory you created on Day 1.

Lab 6 — Clauses and Guards

After Module 06 · 30 minutes

Objective

Build a permission system entirely from function clauses and guards. The module must contain no `if`, `cond` or `case`.

Steps

1. Create a project called `access_control`.
2. Define guards for a valid role, a valid action, and a valid age.
3. Implement permission checks, an age-band classifier, and a full authorisation function.

Requirements

- Roles are `:owner`, `:admin`, `:editor`, `:viewer`. Actions are `:read`, `:write`, `:delete`, `:admin`.
- `allowed?/2` returns a boolean: owner may do anything, admin everything but `:admin`, editor may read and write, viewer may only read.
- `age_band/1` returns `:child` under 13, `:teen` under 18, `:adult` under 65, `:senior` otherwise, and `{:error, :invalid_age}` for anything impossible.
- `authorize/2` takes a user map and an action and returns `:ok`, `{:error, :suspended}`, `{:error, :unknown_role}`, `{:error, :unknown_action}` or `{:error, :forbidden}`.
- Suspension must be checked before anything else.
- Use `defguard` at least three times.

Hints

- `defguard is_role(role) when role in @roles` works because `@roles` is a compile-time attribute
- Guard clauses are tried in order, so put the narrowest band first
- `not is_role(role)` is a valid guard
- A private helper that converts `true/false` into `:ok/{:error, :forbidden}` avoids an `if`

Lab 6 Solution

```
mix new access_control
cd access_control
```

```
# lib/access_control.ex
defmodule AccessControl do
  @moduledoc """
    Role and resource permission checks built from function clauses and guards.
  """
```

```

No `if`, `cond` or `case` appears in this module.
"""

@roles [:owner, :admin, :editor, :viewer]
@actions [:read, :write, :delete, :admin]

defguard is_role(role) when role in @roles
defguard is_action(action) when action in @actions
defguard is_valid_age(age) when is_integer(age) and age >= 0 and age < 150

@doc """
Returns `true` when `role` may perform `action`.

## Examples

    iex> AccessControl.allowed?(:owner, :delete)
    true

    iex> AccessControl.allowed?(:viewer, :write)
    false

    iex> AccessControl.allowed?(:intern, :read)
    false

"""
def allowed?(:owner, action) when is_action(action), do: true
def allowed?(:admin, action) when action in [:read, :write, :delete], do: true
def allowed?(:editor, action) when action in [:read, :write], do: true
def allowed?(:viewer, :read), do: true
def allowed?(_role, _action), do: false

@doc """
Returns the highest action a role may perform.

## Examples

    iex> AccessControl.max_action(:editor)
    :write

    iex> AccessControl.max_action(:intern)
    {:error, :unknown_role}

"""
def max_action(:owner), do: :admin
def max_action(:admin), do: :delete
def max_action(:editor), do: :write
def max_action(:viewer), do: :read
def max_action(_role), do: {:error, :unknown_role}

@doc """
Classifies an age band.

## Examples

    iex> AccessControl.age_band(10)
    :child

    iex> AccessControl.age_band(17)
    :teen

```



```

iex> AccessControl.age_band(40)
:adult

iex> AccessControl.age_band(200)
{:error, :invalid_age}

"""
def age_band(age) when is_valid_age(age) and age < 13, do: :child
def age_band(age) when is_valid_age(age) and age < 18, do: :teen
def age_band(age) when is_valid_age(age) and age < 65, do: :adult
def age_band(age) when is_valid_age(age), do: :senior
def age_band(_age), do: {:error, :invalid_age}

@doc """
Grants access only when the role is known, the action is known, the role
permits the action, and the account is not suspended.

## Examples

iex> AccessControl.authorize(%{role: :editor, suspended: false}, :write)
:ok

iex> AccessControl.authorize(%{role: :editor, suspended: true}, :write)
{:error, :suspended}

iex> AccessControl.authorize(%{role: :viewer, suspended: false}, :write)
{:error, :forbidden}

iex> AccessControl.authorize(%{role: :intern, suspended: false}, :write)
{:error, :unknown_role}

iex> AccessControl.authorize(%{role: :owner, suspended: false}, :launch_missiles)
{:error, :unknown_action}

"""
def authorize(%{suspended: true}, _action), do: {:error, :suspended}

def authorize(%{role: role}, _action) when not is_role(role), do: {:error, :unknown_role}

def authorize(%{role: _role}, action) when not is_action(action), do: {:error,
:unknown_action}

def authorize(%{role: role}, action) do
  permit(allowed?(role, action))
end

defp permit(true), do: :ok
defp permit(false), do: {:error, :forbidden}
end

```

Three design points to discuss.

Clause order in `authorize/2` encodes the business precedence. Suspension first, then role validity, then action validity, then permission. Reordering these lines changes the error a user sees. That is worth making explicit, because in an imperative version the same precedence would be buried in nested conditionals.

age_band/1 puts the narrow bands first. Each clause repeats `is_valid_age(age)` and then adds one upper bound. The final valid clause needs no upper bound because everything below 150 that got past the earlier clauses is 65 or over.

permit/1 exists purely to avoid an if. Two one-line clauses turn a boolean into a tagged result. This is a small pattern you will reuse constantly.

```
# test/access_control_test.exs
defmodule AccessControlTest do
  use ExUnit.Case
  doctest AccessControl

  describe "allowed?/2" do
    test "owner may do everything" do
      for action <- [:read, :write, :delete, :admin] do
        assert AccessControl.allowed?(:owner, action)
      end
    end

    test "admin may not administer" do
      assert AccessControl.allowed?(:admin, :delete)
      refute AccessControl.allowed?(:admin, :admin)
    end

    test "editor may read and write only" do
      assert AccessControl.allowed?(:editor, :write)
      refute AccessControl.allowed?(:editor, :delete)
    end

    test "viewer may only read" do
      assert AccessControl.allowed?(:viewer, :read)
      refute AccessControl.allowed?(:viewer, :write)
    end

    test "unknown role is refused" do
      refute AccessControl.allowed?(:intern, :read)
    end

    test "unknown action is refused even for owner" do
      refute AccessControl.allowed?(:owner, :launch_missiles)
    end
  end

  describe "age_band/1" do
    test "bands are exclusive at the boundaries" do
      assert AccessControl.age_band(12) == :child
      assert AccessControl.age_band(13) == :teen
      assert AccessControl.age_band(17) == :teen
      assert AccessControl.age_band(18) == :adult
      assert AccessControl.age_band(64) == :adult
      assert AccessControl.age_band(65) == :senior
    end

    test "rejects impossible ages" do
      assert AccessControl.age_band(-1) == {:error, :invalid_age}
      assert AccessControl.age_band(150) == {:error, :invalid_age}
      assert AccessControl.age_band("40") == {:error, :invalid_age}
    end
  end
end
```

```
describe "authorize/2" do
  test "suspension beats everything" do
    assert {:error, :suspended} =
      AccessControl.authorize(%{role: :owner, suspended: true}, :read)
  end

  test "allows a permitted action" do
    assert :ok = AccessControl.authorize(%{role: :editor, suspended: false}, :write)
  end

  test "forbids a permitted role doing an unpermitted action" do
    assert {:error, :forbidden} =
      AccessControl.authorize(%{role: :viewer, suspended: false}, :delete)
  end

  test "distinguishes unknown role from unknown action" do
    assert {:error, :unknown_role} =
      AccessControl.authorize(%{role: :intern, suspended: false}, :read)

    assert {:error, :unknown_action} =
      AccessControl.authorize(%{role: :owner, suspended: false}, :teleport)
  end
end
```

```
mix test
```

```
Running ExUnit with seed: 386782, max_cases: 24
.....
Finished in 0.08 seconds (0.00s async, 0.08s sync)

Result: 26 passed (14 doctests, 12 tests)
```

The boundary test is the one that earns its keep. Off-by-one errors in guard ranges are the most common bug in this style of code, and testing 12, 13, 17, 18, 64 and 65 catches every one of them.

Stretch goal

Add `AccessControl.escalate/2` that promotes a user one level, refusing to promote past `:owner` and refusing to promote a suspended user. Doing it with clauses forces a conversation about whether the role hierarchy belongs in clauses or in a list.

Lab 7 — Control Flow

After Module 07 · 35 minutes

Objective

Build an order state machine and a shipping workflow, choosing the right construct for each part.

Steps

1. Create a project called `order_state`.
2. Implement transitions with `case` over a private clause-based rule function.
3. Implement a shipping workflow with `with`.
4. Implement an urgency classifier with `cond`.

Requirements

- Valid transitions: draft to placed; placed to paid or cancelled; paid to shipped or refunded; shipped to delivered. Delivered, cancelled and refunded are terminal.
- `transition/2` returns `{:ok, order}`, `{:error, {:invalid_transition, from, to}}` or `{:error, {:terminal_state, state}}`.
- Transitioning must preserve all other fields on the order map.
- `ship/2` checks address, then items, then the state transition, and adds a tracking number.
- `ship/2` must use `with` and must not use an `else` block.
- `urgency/1` returns `:critical`, `:high`, `:elevated` or `:normal` based on age and value.

Hints

- A private `allowed_transition/2` with one clause per valid pair keeps the rules in one readable block
- Put the terminal-state clause first, with a guard, so it wins over every specific pair
- `%{order | state: target}` preserves the other keys
- The order of `cond` clauses is the priority order of the rules

Lab 7 Solution

```
mix new order_state
cd order_state
```

```
# lib/order_state.ex
defmodule OrderState do
  @moduledoc """
  An order state machine, and a shipping workflow built with `with`.
```

Valid transitions:

```
:draft    -> :placed
:placed   -> :paid | :cancelled
:paid     -> :shipped | :refunded
:shipped  -> :delivered
:delivered -> (terminal)
:cancelled -> (terminal)
:refunded -> (terminal)
```

"""

@terminal [:delivered, :cancelled, :refunded]

@doc """

Attempts a transition.

Examples

```
iex> OrderState.transition(%{state: :placed}, :paid)
{:ok, %{state: :paid}}

iex> OrderState.transition(%{state: :draft}, :shipped)
{:error, {:invalid_transition, :draft, :shipped}}

iex> OrderState.transition(%{state: :delivered}, :refunded)
{:error, {:terminal_state, :delivered}}
```

"""

```
def transition(%{state: state} = order, target) do
  case allowed_transition(state, target) do
    :ok -> {:ok, %{order | state: target}}
    {:error, reason} -> {:error, reason}
  end
end
```

@doc """

Returns `true` when no further transition is possible.

Examples

```
iex> OrderState.terminal?(%{state: :delivered})
true

iex> OrderState.terminal?(%{state: :paid})
false
```

"""

```
def terminal?(%{state: state}), do: state in @terminal
```

@doc """

Runs the full ship workflow: the order must be paid, have a shipping address, and have at least one item.

Examples

```
iex> OrderState.ship(%{state: :paid, address: "12 Main St", items: [:a]}, "TRK-1")
{:ok, %{state: :shipped, address: "12 Main St", items: [:a], tracking: "TRK-1"}}

iex> OrderState.ship(%{state: :placed, address: "12 Main St", items: [:a]}, "TRK-1")
```

```

{:error, {:invalid_transition, :placed, :shipped}}

iex> OrderState.ship(%{state: :paid, address: nil, items: [:a]}, "TRK-1")
{:error, :missing_address}

iex> OrderState.ship(%{state: :paid, address: "12 Main St", items: []}, "TRK-1")
{:error, :empty_order}

"""
def ship(order, tracking) do
  with :ok <- check_address(order),
       :ok <- check_items(order),
       {:ok, shipped} <- transition(order, :shipped) do
    {:ok, Map.put(shipped, :tracking, tracking)}
  end
end

@doc """
Describes an order's urgency from its age in days and value.

## Examples

iex> OrderState.urgency(%{days_old: 10, total: 100})
:critical

iex> OrderState.urgency(%{days_old: 1, total: 50_000})
:high

iex> OrderState.urgency(%{days_old: 1, total: 100})
:normal

"""
def urgency(%{days_old: days, total: total}) do
  cond do
    days >= 7 -> :critical
    total >= 10_000 -> :high
    days >= 3 -> :elevated
    true -> :normal
  end
end

# ----- private

defp allowed_transition(state, _target) when state in @terminal do
  {:error, {:terminal_state, state}}
end

defp allowed_transition(:draft, :placed), do: :ok
defp allowed_transition(:placed, :paid), do: :ok
defp allowed_transition(:placed, :cancelled), do: :ok
defp allowed_transition(:paid, :shipped), do: :ok
defp allowed_transition(:paid, :refunded), do: :ok
defp allowed_transition(:shipped, :delivered), do: :ok
defp allowed_transition(state, target), do: {:error, {:invalid_transition, state, target}}

defp check_address(%{address: nil}), do: {:error, :missing_address}
defp check_address(%{address: ""}), do: {:error, :missing_address}
defp check_address(%{address: _address}), do: :ok

defp check_items(%{items: []}), do: {:error, :empty_order}
defp check_items(%{items: _items}), do: :ok

```

```
end
```

Three points.

The transition table is six lines of `defp`. A reviewer can check it against the state diagram by reading. That is the argument for clauses over a nested `case`.

The terminal-state clause is first and guarded. If it were last, `allowed_transition(:delivered, :refunded)` would fall through to the catch-all and report `:invalid_transition` instead of `:terminal_state`, which is a worse error message.

ship/2 has no `else` block. Each of the three steps already returns a tagged tuple, so failures propagate unchanged and the caller can distinguish them. Ask the room what an `else` block would have added here, and the answer is: a place to get it wrong.

```
# test/order_state_test.exs
defmodule OrderStateTest do
  use ExUnit.Case
  doctest OrderState

  describe "transition/2" do
    test "walks the happy path end to end" do
      assert {:ok, order} = OrderState.transition(%{state: :draft}, :placed)
      assert {:ok, order} = OrderState.transition(order, :paid)
      assert {:ok, order} = OrderState.transition(order, :shipped)
      assert {:ok, order} = OrderState.transition(order, :delivered)
      assert order.state == :delivered
    end

    test "rejects a skipped step" do
      assert {:error, {:invalid_transition, :draft, :paid}} =
        OrderState.transition(%{state: :draft}, :paid)
    end

    test "rejects any transition from a terminal state" do
      for state <- [:delivered, :cancelled, :refunded] do
        assert {:error, {:terminal_state, ^state}} =
          OrderState.transition(%{state: state}, :paid)
      end
    end

    test "preserves other order fields" do
      order = %{state: :placed, id: 7, total: 100}
      assert {:ok, %{state: :paid, id: 7, total: 100}} = OrderState.transition(order, :paid)
    end
  end

  describe "ship/2" do
    @valid %{state: :paid, address: "12 Main St", items: [:widget]}

    test "ships a valid paid order" do
      assert {:ok, shipped} = OrderState.ship(@valid, "TRK-1")
      assert shipped.state == :shipped
      assert shipped.tracking == "TRK-1"
    end

    test "checks the address before the state" do
      order = %{@valid | state: :placed, address: nil}
    end
  end
end
```

```

    assert {:error, :missing_address} = OrderState.ship(order, "TRK-1")
  end

  test "rejects an empty order" do
    assert {:error, :empty_order} = OrderState.ship(%{@valid | items: []}, "TRK-1")
  end

  test "rejects an unpaid order" do
    assert {:error, {:invalid_transition, :placed, :shipped}} =
      OrderState.ship(%{@valid | state: :placed}, "TRK-1")
  end
end

describe "urgency/1" do
  test "age beats value" do
    assert OrderState.urgency(%{days_old: 7, total: 1}) == :critical
    assert OrderState.urgency(%{days_old: 10, total: 99_999}) == :critical
  end

  test "value beats moderate age" do
    assert OrderState.urgency(%{days_old: 3, total: 10_000}) == :high
  end

  test "moderate age alone is elevated" do
    assert OrderState.urgency(%{days_old: 3, total: 10}) == :elevated
  end

  test "everything else is normal" do
    assert OrderState.urgency(%{days_old: 0, total: 10}) == :normal
  end
end

test "terminal? identifies end states" do
  assert OrderState.terminal?(%{state: :cancelled})
  refute OrderState.terminal?(%{state: :draft})
end
end

```

```
mix test
```

```

Running ExUnit with seed: 250190, max_cases: 24

.....
Finished in 0.09 seconds (0.00s async, 0.09s sync)

Result: 25 passed (12 doctests, 13 tests)

```

Note the test "rejects any transition from a terminal state". It uses a comprehension inside a test body, and it pins `^state` in the assertion so the error must carry the state it was given. That is the pin operator from Module 04 doing real work inside a test.

Note also the test "checks the address before the state". The order is both unpaid **and** missing an address, and the error reported is the address, because `with` runs the checks in the order written. Changing the order of lines in `ship/2` changes this test. Point that out.

Stretch goal

Add `OrderState.history/1` that records every transition in a list on the order, and make `transition/2` append to it. It raises the question of whether history belongs on the order or in a separate log, which is a real design conversation.

Lab 8 — Enum Pipelines

After Module 08 · 35 minutes

Objective

Build an exam-results analytics module entirely from `Enum` pipelines.

Steps

1. Create a project called `analytics`.
2. Implement seven aggregation functions over a list of result maps.
3. Make every function handle an empty input without crashing.

Requirements

A result looks like `%{student: "priya", subject: :maths, score: 88, attempts: 1}`.

- `average/1` returns the mean score rounded to one decimal, and `0.0` for an empty list
- `best_per_subject/1` returns a map of subject to the highest score
- `top_students/2` returns the top N students by average as `{student, average}`, highest first
- `partition_by_pass/2` splits into `{passed, failed}` in **one pass**
- `grade_distribution/1` counts results by grade band A, B, C, F
- `first_time_passers/1` returns sorted student names who never needed more than one attempt
- `ranking_table/1` formats `{student, average}` pairs as a numbered multi-line string

Hints

- `Enum.group_by/3` takes a value mapper as its third argument
- `Map.new/2` converts a grouped map into a map, `Enum.map/2` would give you a list
- `Enum.split_with/2` is the one-pass partition
- `Enum.frequencies_by/2` counts by a derived key
- `Enum.with_index/2` accepts a starting offset
- `Enum.map_join/3` maps and joins in one traversal

Lab 8 Solution

```
mix new analytics
cd analytics
```

```
# lib/analytics.ex
```

```

defmodule Analytics do
  @moduledoc """
  Enum pipelines over exam results.

  A result looks like:

      %{student: "priya", subject: :maths, score: 88, attempts: 1}

  """

  @doc """
  Average score across all results, rounded to one decimal place.

  ## Examples

      iex> Analytics.average([%{score: 80}, %{score: 91}])
      85.5

      iex> Analytics.average([])
      0.0

  """
  def average([], do: 0.0)

  def average(results) do
    results
    |> Enum.map(& &1.score)
    |> Enum.sum()
    |> Kernel./(length(results))
    |> Float.round(1)
  end

  @doc """
  Best score per subject.

  ## Examples

      iex> Analytics.best_per_subject([
      ...>   %{subject: :maths, score: 70},
      ...>   %{subject: :maths, score: 91},
      ...>   %{subject: :physics, score: 60}
      ...> ])
      %{maths: 91, physics: 60}

  """
  def best_per_subject(results) do
    results
    |> Enum.group_by(& &1.subject, & &1.score)
    |> Map.new(fn {subject, scores} -> {subject, Enum.max(scores)} end)
  end

  @doc """
  The `count` students with the highest average, as `{student, average}`.

  ## Examples

      iex> Analytics.top_students([
      ...>   %{student: "a", score: 90},
      ...>   %{student: "a", score: 80},
      ...>   %{student: "b", score: 100}
      ...> ], 1)
  
```

```

    [{"b", 100.0}]

"""
def top_students(results, count) do
  results
  |> Enum.group_by(& &1.student, & &1.score)
  |> Enum.map(fn {student, scores} ->
    {student, Float.round(Enum.sum(scores) / length(scores), 1)}
  end)
  |> Enum.sort_by(fn {_student, average} -> average end, :desc)
  |> Enum.take(count)
end

@doc """
Splits results into `{passed, failed}` at the given threshold.

## Examples

    iex> Analytics.partition_by_pass([%{score: 40}, %{score: 60}], 50)
    [{%{score: 60}}, [%{score: 40}]]

"""
def partition_by_pass(results, threshold) do
  Enum.split_with(results, &(&1.score >= threshold))
end

@doc """
Counts results by grade band.

## Examples

    iex> Analytics.grade_distribution([%{score: 95}, %{score: 72}, %{score: 30}])
    %{ "A" => 1, "B" => 1, "F" => 1}

"""
def grade_distribution(results) do
  Enum.frequencies_by(results, &grade(&1.score))
end

@doc """
Students who never needed more than one attempt, sorted.

## Examples

    iex> Analytics.first_time_passers([
    ...>   %{student: "a", attempts: 1},
    ...>   %{student: "b", attempts: 2},
    ...>   %{student: "a", attempts: 1}
    ...> ])
    ["a"]

"""
def first_time_passers(results) do
  results
  |> Enum.group_by(& &1.student, & &1.attempts)
  |> Enum.filter(fn {_student, attempts} -> Enum.all?(attempts, &(&1 == 1)) end)
  |> Enum.map(fn {student, _attempts} -> student end)
  |> Enum.sort()
end

@doc """

```

```

A printable ranking table.
"""
def ranking_table(ranked) do
  ranked
  |> Enum.with_index(1)
  |> Enum.map_join("\n", fn {{student, average}, position} ->
    "#{position}. #{student} #{average}"
  end)
end

defp grade(score) when score >= 90, do: "A"
defp grade(score) when score >= 70, do: "B"
defp grade(score) when score >= 50, do: "C"
defp grade(_score), do: "F"
end

```

Four techniques to name out loud.

Kernel./(length(results)) inside a pipe. You cannot write `|> / length(results)`, because `/` is an operator. Piping into `Kernel./` with the divisor as the second argument works, and it keeps the pipeline unbroken. Some teams find it too clever and assign to a variable instead; both are defensible, so let the room argue it.

Enum.group_by/3 with a value mapper. `group_by(results, & &1.subject, & &1.score)` gives `%{maths: [70, 91]}` directly rather than a map of full result maps that then needs another `map`.

Map.new/2 versus Enum.map/2. `best_per_subject/1` needs a map back, so it uses `Map.new/2`. `top_students/2` needs a sortable list, so it uses `Enum.map/2`. Same grouping, different collection at the end, driven by what the next step needs.

Enum.with_index(1). The optional offset removes the `+ 1` that every newcomer writes.

```

# test/analytics_test.exs
defmodule AnalyticsTest do
  use ExUnit.Case
  doctest Analytics

  @results [
    %{student: "priya", subject: :maths, score: 88, attempts: 1},
    %{student: "priya", subject: :physics, score: 92, attempts: 1},
    %{student: "arjun", subject: :maths, score: 71, attempts: 2},
    %{student: "arjun", subject: :physics, score: 45, attempts: 3},
    %{student: "meera", subject: :maths, score: 95, attempts: 1},
    %{student: "meera", subject: :physics, score: 99, attempts: 1}
  ]

  test "average across everything" do
    assert Analytics.average(@results) == 81.7
  end

  test "average of an empty list is zero" do
    assert Analytics.average([]) == 0.0
  end

  test "best score per subject" do
    assert Analytics.best_per_subject(@results) == %{maths: 95, physics: 99}
  end
end

```

```

test "top students are ordered by average" do
  assert Analytics.top_students(@results, 2) == [{"meera", 97.0}, {"priya", 90.0}]
end

test "top students respects the count" do
  assert [{"meera", 97.0}] = Analytics.top_students(@results, 1)
end

test "partition by pass mark" do
  {passed, failed} = Analytics.partition_by_pass(@results, 50)
  assert length(passed) == 5
  assert length(failed) == 1
end

test "grade distribution" do
  assert Analytics.grade_distribution(@results) == %{"A" => 3, "B" => 2, "F" => 1}
end

test "first time passers excludes anyone with a retry" do
  assert Analytics.first_time_passers(@results) == ["meera", "priya"]
end

test "ranking table formats positions" do
  table =
    @results
    |> Analytics.top_students(2)
    |> Analytics.ranking_table()

  assert table == "1. meera 97.0\n2. priya 90.0"
end

test "every function handles an empty list" do
  assert Analytics.best_per_subject([]) == %{}
  assert Analytics.top_students([], 3) == []
  assert Analytics.grade_distribution([]) == %{}
  assert Analytics.first_time_passers([]) == []
  assert Analytics.ranking_table([]) == ""
end
end

```

```
mix test
```

```

Running ExUnit with seed: 921934, max_cases: 24

.....
Finished in 0.06 seconds (0.00s async, 0.06s sync)

Result: 18 passed (8 doctests, 10 tests)

```

The final test is the one most trainees forget to write. Six of these seven functions handle the empty list for free because `Enum` does; only `average/1` needed an explicit clause, because dividing by zero would crash. Ask which one needed special handling and why, before showing them.

Stretch goal

Add `Analytics.improvement/1` that, for students appearing in more than one subject, returns the difference between their best and worst score. It requires `Enum.min_max/1` and a filter on group size.

Lab 9 — Stream and Comprehensions

After Module 09 · 35 minutes

Objective

Build a CSV toolkit that reads lazily and reshapes with comprehensions.

Steps

1. Create a project called `csv_tools`.
2. Stream a CSV file into maps keyed by the header row.
3. Implement lazy search, counting and summing.
4. Use `for` comprehensions for the in-memory reshaping functions.

Requirements

- `stream_rows/1` returns a **stream** of maps, not a list
- Rows with the wrong number of columns are skipped, not crashed on
- `find/4` stops reading the file as soon as it has enough matches
- `count_by/2` and `sum_column/2` walk the whole file without loading it
- `sum_column/2` ignores values that are not integers
- `successes/1`, `index_by/2` and `tag_matrix/3` must each use a `for` comprehension
- `to_csv/2` renders rows back to CSV text

Hints

- `File.stream!/1` gives you a stream of lines including the newline, so trim
- Take the header with `Enum.take(stream, 1)` then `Stream.drop(stream, 1)` for the body
- `Enum.zip/2` plus `Map.new/1` turns headers and values into a row map
- `for {:ok, v} <- results, do: v` is the pattern-filter idiom
- `for row <- rows, into: %{}, do: {key, row}` builds a lookup
- Two generators produce the cartesian product

Lab 9 Solution

```
mix new csv_tools
cd csv_tools
```

```
# lib/csv_tools.ex
```



```

defmodule CsvTools do
  @moduledoc """
  Lazy CSV reading with `Stream`, and reshaping with comprehensions.

  The CSV is assumed to be comma separated with a header row and no
  quoted fields. Real CSV parsing needs a library such as NimbleCSV;
  this module exists to teach Stream and `for`.
  """

  @doc """
  Streams rows as maps keyed by the header names.

  The file is read lazily, one line at a time.
  """
  def stream_rows(path) do
    stream = File.stream!(path) |> Stream.map(&String.trim/1) |> Stream.reject(&(&1 == ""))

    headers =
      stream
      |> Enum.take(1)
      |> List.first()
      |> split_row()

    stream
    |> Stream.drop(1)
    |> Stream.map(&split_row/1)
    |> Stream.filter(&(length(&1) == length(headers)))
    |> Stream.map(&build_row(headers, &1))
  end

  @doc """
  The first `count` rows whose `column` equals `value`.

  Stops reading the file as soon as `count` rows are found.
  """
  def find(path, column, value, count) do
    path
    |> stream_rows()
    |> Stream.filter(&(Map.get(&1, column) == value))
    |> Enum.take(count)
  end

  @doc """
  Counts rows by the value of `column`, over the whole file.
  """
  def count_by(path, column) do
    path
    |> stream_rows()
    |> Stream.map(&Map.get(&1, column))
    |> Enum.frequencies()
  end

  @doc """
  Sums an integer column over the whole file.
  """
  def sum_column(path, column) do
    path
    |> stream_rows()
    |> Stream.map(&Map.get(&1, column))
    |> Stream.map(&to_integer/1)
    |> Stream.reject(&is_nil/1)
  end
end

```

```

    |> Enum.sum()
  end

  @doc """
  Keeps only the successful parses.

  ## Examples

      iex> CsvTools.successes([{:ok, 1}, {:error, :bad}, {:ok, 2}])
      [1, 2]

      """
  def successes(results) do
    for {:ok, value} <- results, do: value
  end

  @doc """
  Indexes rows by one column, into a map.

  ## Examples

      iex> CsvTools.index_by([%{"id" => "1", "n" => "a"}, %{"id" => "2", "n" => "b"}], "id")
      %{"1" => %{"id" => "1", "n" => "a"}, "2" => %{"id" => "2", "n" => "b"}}

      """
  def index_by(rows, column) do
    for row <- rows, into: %{}, do: {Map.get(row, column), row}
  end

  @doc """
  Every pairing of a row's `column` value with each tag.

  ## Examples

      iex> CsvTools.tag_matrix([%{"r" => "north"}, %{"r" => "south"}], "r", ["a", "b"])
      ["north/a", "north/b", "south/a", "south/b"]

      """
  def tag_matrix(rows, column, tags) do
    for row <- rows, tag <- tags, do: "#{Map.get(row, column)}/#{tag}"
  end

  @doc """
  Writes rows back out as CSV text.

      """
  def to_csv(rows, headers) do
    header_line = Enum.join(headers, ",")

    body =
      Enum.map_join(rows, "\n", fn row ->
        Enum.map_join(headers, ",", &Map.get(row, &1, ""))
      end)

    case body do
      "" -> header_line
      _ -> header_line <> "\n" <> body
    end
  end

  # ----- private

```

```

defp split_row(line), do: String.split(line, ",")

defp build_row(headers, values) do
  headers
  |> Enum.zip(values)
  |> Map.new()
end

defp to_integer(nil), do: nil

defp to_integer(value) do
  case Integer.parse(value) do
    {number, ""} -> number
    _ -> nil
  end
end
end
end

```

Three things worth stopping on.

The header is read eagerly, the body lazily. `Enum.take(stream, 1)` forces exactly one line, then `Stream.drop(stream, 1)` builds a fresh lazy pipeline over the same file. Reading the file handle twice is fine here because `File.stream!/1` reopens on each traversal. On a very large file this costs one extra line read, which is negligible.

Malformed rows are filtered by length, not rescued. `Stream.filter(&(length(&1) == length(headers)))` drops the bad row before it can produce a mismatched map. No exception handling needed.

`to_integer/1` uses `{number, ""}`. Matching on an empty remainder rejects `"12abc"`. This is the same technique as Lab 4, and it is the correct way to validate a numeric string in Elixir.

```

# test/csv_tools_test.exs
defmodule CsvToolsTest do
  use ExUnit.Case
  doctest CsvTools

  @csv """
  id,region,amount
  1,north,100
  2,south,250
  3,north,90
  4,east,not_a_number
  5,south,60
  bad,row
  """

  setup do
    path = Path.join(System.tmp_dir!(), "csv_tools_#{System.unique_integer([:positive])}.csv")
    File.write!(path, @csv)
    on_exit(fn -> File.rm(path) end)
    {:ok, path: path}
  end

  test "streams rows as maps", %{path: path} do
    rows = path |> CsvTools.stream_rows() |> Enum.to_list()

    assert length(rows) == 5
  end
end

```

```

    assert List.first(rows) == %{ "id" => "1", "region" => "north", "amount" => "100" }
  end

  test "skips rows with the wrong number of columns", %{path: path} do
    ids = path |> CsvTools.stream_rows() |> Enum.map(&Map.get(&1, "id"))
    refute "bad" in ids
  end

  test "find stops after the requested count", %{path: path} do
    rows = CsvTools.find(path, "region", "north", 1)
    assert length(rows) == 1
    assert List.first(rows)["id"] == "1"
  end

  test "find returns fewer when there are fewer", %{path: path} do
    assert length(CsvTools.find(path, "region", "east", 10)) == 1
  end

  test "count_by groups the whole file", %{path: path} do
    assert CsvTools.count_by(path, "region") == %{ "north" => 2, "south" => 2, "east" => 1 }
  end

  test "sum_column ignores unparseable values", %{path: path} do
    assert CsvTools.sum_column(path, "amount") == 500
  end

  test "successes unwraps ok tuples" do
    assert CsvTools.successes([{:ok, 1}, {:error, :bad}, {:ok, 2}]) == [1, 2]
  end

  test "index_by builds a lookup", %{path: path} do
    index = path |> CsvTools.stream_rows() |> Enum.to_list() |> CsvTools.index_by("id")

    assert index["3"]["region"] == "north"
    assert map_size(index) == 5
  end

  test "tag_matrix produces the cartesian product" do
    assert CsvTools.tag_matrix([%{"r" => "n"}, %{"r" => "s"}], "r", ["a", "b"]) ==
      ["n/a", "n/b", "s/a", "s/b"]
  end

  test "to_csv round trips", %{path: path} do
    rows = path |> CsvTools.stream_rows() |> Enum.take(2)
    csv = CsvTools.to_csv(rows, ["id", "region", "amount"])

    assert csv == "id,region,amount\n1,north,100\n2,south,250"
  end

  test "to_csv of no rows is just the header" do
    assert CsvTools.to_csv([], ["a", "b"]) == "a,b"
  end
end

```

```

mix test

```

```

Running ExUnit with seed: 476796, max_cases: 24

```

```

.....

```

```
Finished in 0.08 seconds (0.00s async, 0.08s sync)
```

```
Result: 15 passed (4 doctests, 11 tests)
```

Discussion point

`stream_rows/1` returns a stream, and the test has to call `Enum.to_list/1` to see anything. Ask what would happen if a caller forgot that and passed the stream straight to `length/1`. The answer is a `Protocol.UndefinedError`, because a stream is not a list. That is the most common Stream mistake, and meeting it here is cheaper than meeting it in production.

Stretch goal

Add `CsvTools.transform/3` that streams an input CSV, applies a function to each row, and writes the result to an output file, still without loading either file into memory. `Stream.into/2` and `File.stream!/2` in write mode are what you need.

Lab 10 — Recursion

After Module 10 · 35 minutes

Objective

Walk a tree recursively, first in memory and then on the real filesystem.

Steps

1. Create a project called `file_tree`.
2. Define a node as `{:file, name, size}` or `{:dir, name, children}`.
3. Implement `size`, `counting`, `depth`, `path listing` and a `largest-files` report.
4. Implement `from_disk/1` that builds the same structure from a real directory.

Requirements

- `total_size/1` sums every file in the tree
- `count/1` returns `%{files: n, dirs: n}`, counting the root directory itself
- `depth/1` returns 1 for a single file and for an empty directory
- `paths/1` returns slash-joined file paths in traversal order
- `largest/2` returns `{path, size}` tuples above a threshold, largest first
- `from_disk/1` builds a node tree from a real directory path
- Every function must handle an empty directory

Hints

- Two clauses per function: one for `:file`, one for `:dir`
- The directory clause maps the function over its children and combines the results
- Carry the path prefix as an accumulator in a private helper
- `Enum.flat_map/2` is the right combiner for path lists
- `File.stat/1` tells you `:directory` or `:regular`, and `File.ls!/1` lists a directory

Lab 10 Solution

```
mix new file_tree
cd file_tree
```

```
# lib/file_tree.ex
defmodule FileTree do
  @moduledoc """
```

Recursion over a nested structure, demonstrated on a directory tree and on an in-memory tree of the same shape.

An in-memory node is either:

```
{:file, name, size}
{:dir, name, [node]}
```

"""

@doc """

Total size of every file in the tree.

Examples

```
iex> FileTree.total_size({:dir, "root", [{:file, "a", 10}, {:file, "b", 5}]})
15
```

```
iex> FileTree.total_size({:dir, "empty", []})
0
```

"""

```
def total_size({:file, _name, size}), do: size
```

```
def total_size({:dir, _name, children}) do
```

```
  children
```

```
  |> Enum.map(&total_size/1)
```

```
  |> Enum.sum()
```

```
end
```

@doc """

Counts files and directories.

Examples

```
iex> FileTree.count({:dir, "root", [{:file, "a", 1}, {:dir, "sub", [{:file, "b", 2}]}]})
%{files: 2, dirs: 2}
```

"""

```
def count({:file, _name, _size}), do: %{files: 1, dirs: 0}
```

```
def count({:dir, _name, children}) do
```

```
  children
```

```
  |> Enum.map(&count/1)
```

```
  |> Enum.reduce(%{files: 0, dirs: 1}, fn child, acc ->
```

```
    %{files: acc.files + child.files, dirs: acc.dirs + child.dirs}
```

```
  end)
```

```
end
```

@doc """

The deepest nesting level. A single file is depth 1.

Examples

```
iex> FileTree.depth({:file, "a", 1})
1
```

```
iex> FileTree.depth({:dir, "r", [{:dir, "s", [{:file, "a", 1}]}]})
3
```

"""

```

def depth({:file, _name, _size}), do: 1
def depth({:dir, _name, []}), do: 1

def depth({:dir, _name, children}) do
  1 + (children |> Enum.map(&depth/1) |> Enum.max())
end

@doc """
Every file path in the tree, as slash-joined strings.

## Examples

    iex> FileTree.paths({:dir, "r", [{:file, "a", 1}, {:dir, "s", [{:file, "b", 2}]}]})
    ["r/a", "r/s/b"]

    """
def paths(node), do: do_paths(node, "")

@doc """
Files larger than `threshold`, as `{path, size}`, largest first.

## Examples

    iex> FileTree.largest({:dir, "r", [{:file, "a", 10}, {:file, "b", 50}]}, 5)
    [{"r/b", 50}, {"r/a", 10}]

    """
def largest(node, threshold) do
  node
  |> do_entries("")
  |> Enum.filter(fn {_path, size} -> size > threshold end)
  |> Enum.sort_by(fn {_path, size} -> size end, :desc)
end

@doc """
Walks a real directory on disk and returns the same node structure.

    """
def from_disk(path) do
  case File.stat(path) do
    {:ok, %File.Stat{type: :directory}} ->
      children =
        path
        |> File.ls!()
        |> Enum.sort()
        |> Enum.map(&from_disk(Path.join(path, &1)))

      {:dir, Path.basename(path), children}

    {:ok, %File.Stat{type: :regular, size: size}} ->
      {:file, Path.basename(path), size}

    _other ->
      {:file, Path.basename(path), 0}
  end
end

# ----- private

defp do_paths({:file, name, _size}, prefix), do: [join(prefix, name)]

defp do_paths({:dir, name, children}, prefix) do

```



```

    new_prefix = join(prefix, name)
    Enum.flat_map(children, &do_paths(&1, new_prefix))
  end

  defp do_entries({:file, name, size}, prefix), do: [{join(prefix, name), size}]

  defp do_entries({:dir, name, children}, prefix) do
    new_prefix = join(prefix, name)
    Enum.flat_map(children, &do_entries(&1, new_prefix))
  end

  defp join("", name), do: name
  defp join(prefix, name), do: prefix <> "/" <> name
end

```

Four observations for the debrief.

Every public function is two or three clauses. One per node shape. The recursion is not clever; it is just the data's own structure written out. That is what tree recursion should look like.

The path accumulator lives in a private helper. The public `paths/1` takes one argument, because callers should not have to pass an empty prefix. This is the same public-sets-up, private-carries pattern as `reverse/1` in Module 10.

depth/1 needs the empty-directory clause. Without `depth({:dir, _, []})`, `Enum.max([])` raises. Ask the room to delete that clause and run the tests; the failure is instructive.

from_disk/1 is the only function that touches the outside world. Everything else operates on plain data, which is why everything else is trivially testable. That separation is a design principle worth naming now and repeating on Day 3.

```

# test/file_tree_test.exs
defmodule FileTreeTest do
  use ExUnit.Case
  doctest FileTree

  @tree {:dir, "root",
    [
      {:file, "readme.md", 120},
      {:dir, "lib",
        [
          {:file, "app.ex", 4_000},
          {:dir, "web", [{:file, "router.ex", 900}]}
        ]},
      {:dir, "empty", []}
    ]}

  test "total size sums every leaf" do
    assert FileTree.total_size(@tree) == 5_020
  end

  test "total size of an empty directory is zero" do
    assert FileTree.total_size({:dir, "empty", []}) == 0
  end

  test "counts files and directories separately" do
    assert FileTree.count(@tree) == %{files: 3, dirs: 4}
  end
end

```

```

test "depth measures the deepest branch" do
  assert FileTree.depth(@tree) == 4
end

test "depth of a single file is one" do
  assert FileTree.depth({:file, "a", 1}) == 1
end

test "paths are joined in order" do
  assert FileTree.paths(@tree) == [
    "root/readme.md",
    "root/lib/app.ex",
    "root/lib/web/router.ex"
  ]
end

test "largest filters and sorts descending" do
  assert FileTree.largest(@tree, 500) == [{"root/lib/app.ex", 4_000},
    {"root/lib/web/router.ex", 900}]
end

test "largest can return nothing" do
  assert FileTree.largest(@tree, 10_000) == []
end

describe "from_disk/1" do
  setup do
    root = Path.join(System.tmp_dir!(), "file_tree_#{System.unique_integer([:positive])}")
    File.mkdir_p!(Path.join(root, "sub"))
    File.write!(Path.join(root, "a.txt"), "hello")
    File.write!(Path.join([root, "sub", "b.txt"]), "hi")
    on_exit(fn -> File.rm_rf(root) end)
    {:ok, root: root}
  end

  test "mirrors a real directory", %{root: root} do
    tree = FileTree.from_disk(root)

    assert {:dir, _name, children} = tree
    assert length(children) == 2
    assert FileTree.total_size(tree) == 7
    assert FileTree.count(tree) == %{files: 2, dirs: 2}
  end
end
end

```

```

mix test

```

```

Running ExUnit with seed: 609721, max_cases: 24

.....
Finished in 0.1 seconds (0.00s async, 0.1s sync)

Result: 16 passed (7 doctests, 9 tests)

```

The `from_disk/1` test writes `"hello"` and `"hi"`, five bytes and two, so the expected total is seven. Point out that the in-memory functions were reused unchanged on the real tree, which is the payoff for keeping

IO at the edge.

Stretch goal

Make `from_disk/1` handle symlinks without following them into a cycle. It is a genuine problem, and the fix, tracking visited inodes in an accumulator, is a good exercise in threading state through a recursion.

End of Day 2 review

1. Show a place where you replaced an `if` ladder with function clauses. What did the reviewer gain?
2. Where did you use `with`, and what would the `case` version have looked like?
3. Which `Enum` function surprised you by already existing?
4. In Lab 9, what exactly is read from disk when `find/4` is asked for one match?
5. In Lab 10, which clause stops the recursion in each function? Name it for `total_size/1`, `depth/1` and `paths/1`.

Day 3 Labs — Building Real Projects

Five labs covering Modules 11 to 15. Labs 13, 14 and 15 build on one another: you write a module, then test it properly, then bring it up to production standard.

Lab 11 — A Real Mix Project

After Module 11 · 45 minutes

Objective

Build a command-line task manager that uses a Hex dependency, application configuration, a custom Mix task and an escript. This is the first lab that looks like a real project rather than an exercise.

Steps

1. Create a project called `task_cli`.
2. Add `jason` as a dependency and fetch it.
3. Add configuration so the test environment writes to a different file.
4. Write the storage layer, the task operations and a CLI parser.
5. Add a `mix tasks.report` custom task.
6. Add an escript target and run the tool from the shell.
7. Use `mix compile` and `mix run` to inspect the build and the configuration in each environment.

Requirements

- `TaskCli.add/3`, `complete/2`, `remove/2` are **pure**: they take a list and return a new list
- `TaskCli.Store` handles all file IO and JSON, and a missing file loads as `{:ok, []}`
- The store path comes from `Application.get_env/3`, and `config/test.exs` overrides it
- `mix.exs` defines a `setup` alias and a `check` alias
- `mix tasks.report` prints a pending-task summary and appears in `mix help`
- `mix escript.build` produces a runnable `task_cli` binary
- `mix compile --warnings-as-errors` passes

Hints

- `Jason.encode!(tasks, pretty: true)` and `Jason.decode/1`
- Task maps use **string keys**, because they round-trip through JSON
- `OptionParser.parse(argv, strict: [priority: :string])` parses the flags
- A Mix task module named `Mix.Tasks.Tasks.Report` becomes `mix tasks.report`
- `escript: [main_module: TaskCli.CLI]` in `project/0`, and `main/1` as the entry point
- `mix run -e '...'` compiles if needed, starts the application, runs the expression and exits

Lab 11 Solution

```
mix new task_cli
cd task_cli
```

mix.exs

```
defmodule TaskCli.MixProject do
  use Mix.Project

  def project do
    [
      app: :task_cli,
      version: "0.1.0",
      elixir: "~> 1.20",
      start_permanent: Mix.env() == :prod,
      escript: [main_module: TaskCli.CLI],
      aliases: aliases(),
      deps: deps()
    ]
  end

  def application do
    [
      extra_applications: [:logger]
    ]
  end

  defp aliases do
    [
      setup: ["deps.get", "compile"],
      check: ["format --check-formatted", "test"]
    ]
  end

  defp deps do
    [
      {:jason, "~> 1.4"}
    ]
  end
end
```

```
mix deps.get
```

```
Resolving Hex dependencies...
New:
  jason 1.4.5
* Getting jason (Hex package)
```

Configuration

```
# config/config.exs
import Config

config :task_cli,
  store_path: "tasks.json",
```

```

    default_priority: :normal

import_config "#{config_env()}.exs"

```

```

# config/test.exs
import Config

config :task_cli, store_path: "tmp/test_tasks.json"

```

`config/dev.exs` and `config/prod.exs` contain only `import Config`. They must exist, because `import_config` names them.

The storage layer

```

# lib/task_cli/store.ex
defmodule TaskCli.Store do
  @moduledoc """
  Reads and writes the task list as JSON on disk.

  The path comes from application configuration, so the test environment
  writes somewhere harmless.
  """

  @doc "Returns the configured store path."
  def path, do: Application.get_env(:task_cli, :store_path, "tasks.json")

  @doc """
  Loads the task list. A missing file is an empty list, not an error.
  """
  def load(store_path \\ path()) do
    case File.read(store_path) do
      {:ok, contents} -> decode(contents)
      {:error, :enoent} -> {:ok, []}
      {:error, reason} -> {:error, {:unreadable, reason}}
    end
  end

  @doc "Writes the task list, creating the directory if needed."
  def save(tasks, store_path \\ path()) do
    store_path |> Path.dirname() |> File.mkdir_p!()
    File.write(store_path, Jason.encode!(tasks, pretty: true))
  end

  @doc "Deletes the store file if it exists."
  def clear(store_path \\ path()) do
    File.rm(store_path)
    :ok
  end

  defp decode(contents) do
    case Jason.decode(contents) do
      {:ok, tasks} when is_list(tasks) -> {:ok, tasks}
      {:ok, _other} -> {:error, :malformed_store}
      {:error, _} -> {:error, :malformed_store}
    end
  end
end

```

Three decisions worth defending in the debrief.

A missing file returns `{:ok, []}`, not an error. On first run there is no file, and that is normal, not exceptional. Module 13's rule applied.

Both `load/1` and `save/2` take an optional path argument defaulting to the configured one. That makes them directly testable with a temporary file, without touching global configuration, which is what lets the test module be `async: true`.

JSON that parses but is not a list is `:malformed_store`. `Jason.decode/1` succeeding is not the same as the data being valid.

The pure core

```
# lib/task_cli.ex
defmodule TaskCli do
  @moduledoc """
  Task list operations. Pure functions over a list of task maps, plus
  thin wrappers that persist through `TaskCli.Store`.

  A task is a map with string keys, because it round-trips through JSON:

      %{id => 1, "title" => "Write slides", "priority" => "high", "done" => false}

  """

  alias TaskCli.Store

  @priorities ~w(low normal high)

  @doc """
  Adds a task to a list, assigning the next id.

  ## Examples

      iex> TaskCli.add([], "Write slides", "high")
      {:ok, [%{"id" => 1, "title" => "Write slides", "priority" => "high", "done" => false}]}

      iex> TaskCli.add([], "", "high")
      {:error, :empty_title}

      iex> TaskCli.add([], "Write slides", "urgent")
      {:error, :invalid_priority}

  """
  def add(_tasks, "", _priority), do: {:error, :empty_title}

  def add(tasks, title, priority) when priority in @priorities do
    task = %{id => next_id(tasks), "title" => title, "priority" => priority, "done" => false}
    {:ok, tasks ++ [task]}
  end

  def add(_tasks, _title, _priority), do: {:error, :invalid_priority}

  @doc """
  Marks a task done.

  ## Examples
  """
end
```



```

iex> tasks = [%{"id" => 1, "title" => "a", "priority" => "low", "done" => false}]
iex> {:ok, [task]} = TaskCli.complete(tasks, 1)
iex> task["done"]
true

iex> TaskCli.complete([], 99)
{:error, :not_found}

"""
def complete(tasks, id) do
  case Enum.find(tasks, &(&1["id"] == id)) do
    nil -> {:error, :not_found}
    _found -> {:ok, Enum.map(tasks, &mark_done(&1, id))}
  end
end

@doc """
Removes a task.

## Examples

iex> tasks = [%{"id" => 1, "title" => "a", "priority" => "low", "done" => false}]
iex> TaskCli.remove(tasks, 1)
{:ok, []}

iex> TaskCli.remove([], 1)
{:error, :not_found}

"""
def remove(tasks, id) do
  case Enum.split_with(tasks, &(&1["id"] == id)) do
    {[], _rest} -> {:error, :not_found}
    {_removed, rest} -> {:ok, rest}
  end
end

@doc """
Counts pending tasks by priority.

## Examples

iex> TaskCli.pending_by_priority([
...>   %{"id" => 1, "title" => "a", "priority" => "high", "done" => false},
...>   %{"id" => 2, "title" => "b", "priority" => "high", "done" => true}
...> ])
%{"high" => 1}

"""
def pending_by_priority(tasks) do
  tasks
  |> Enum.reject(& &1["done"])
  |> Enum.frequencies_by(& &1["priority"])
end

@doc """
Renders the list for a terminal.

## Examples

iex> TaskCli.render([%{"id" => 1, "title" => "a", "priority" => "high", "done" => false}])
"  1 [ ] (high) a"

```

```

iex> TaskCli.render([])
"No tasks."

"""
def render([], do: "No tasks.")

def render(tasks) do
  Enum.map_join(tasks, "\n", fn task ->
    box = if task["done"], do: "x", else: " "
    id = String.pad_leading(to_string(task["id"]), 3)
    "#{id} [#{box}] (#{task["priority"]}) #{task["title"]}"
  end)
end

# ----- persisted wrappers

@doc "Loads, adds and saves in one step."
def add!(title, priority) do
  with {:ok, tasks} <- Store.load(),
       {:ok, updated} <- add(tasks, title, priority),
       :ok <- Store.save(updated) do
    {:ok, updated}
  end
end

@doc "Loads, completes and saves in one step."
def complete!(id) do
  with {:ok, tasks} <- Store.load(),
       {:ok, updated} <- complete(tasks, id),
       :ok <- Store.save(updated) do
    {:ok, updated}
  end
end

@doc "Loads the current list."
def list, do: Store.load()

# ----- private

defp next_id([], do: 1)
defp next_id(tasks), do: (tasks |> Enum.map(& &1["id"]) |> Enum.max()) + 1

defp mark_done(%{"id" => id} = task, id), do: Map.put(task, "done", true)
defp mark_done(task, _id), do: task
end

```

The structure here is the lesson of the lab. **Pure functions at the centre, IO at the edge.** Every rule about ids, priorities and completion is in a function that takes a list and returns a list. Those functions need no fixtures, no temporary files and no configuration to test. The three persisting wrappers at the bottom are trivially thin and use `with` to sequence load, transform and save.

`mark_done/2` uses the repeated-variable trick from Day 1: `mark_done(%{"id" => id} = task, id)` matches only when the task's id equals the target id.

The CLI

```
# lib/task_cli/cli.ex
```

```

defmodule TaskCli.CLI do
  @moduledoc """
  Escript entry point.

      mix escript.build
      ./task_cli add "Write slides" --priority high
      ./task_cli list
      ./task_cli done 1

  """

  def main(argv) do
    argv
    |> parse()
    |> run()
    |> IO.puts()
  end

  @doc """
  Parses argv into a command tuple.

  ## Examples

      iex> TaskCli.CLI.parse(["add", "Write slides", "--priority", "high"])
      {:add, "Write slides", "high"}

      iex> TaskCli.CLI.parse(["add", "Write slides"])
      {:add, "Write slides", "normal"}

      iex> TaskCli.CLI.parse(["list"])
      :list

      iex> TaskCli.CLI.parse(["done", "3"])
      {:complete, 3}

      iex> TaskCli.CLI.parse(["fly"])
      :help

  """
  def parse(argv) do
    {opts, positional, _invalid} = OptionParser.parse(argv, strict: [priority: :string])

    case positional do
      ["add", title] -> {:add, title, Keyword.get(opts, :priority, "normal")}
      ["list"] -> :list
      ["done", id] -> parse_id(id)
      _other -> :help
    end
  end

  @doc "Runs a parsed command and returns the text to print."
  def run({:add, title, priority}) do
    case TaskCli.add!(title, priority) do
      {:ok, tasks} -> TaskCli.render(tasks)
      {:error, reason} -> "Error: #{reason}"
    end
  end

  def run({:complete, id}) do
    case TaskCli.complete!(id) do
      {:ok, tasks} -> TaskCli.render(tasks)
    end
  end
end

```

```

        {:error, reason} -> "Error: #{reason}"
      end
    end

  def run(:list) do
    case TaskCli.list() do
      {:ok, tasks} -> TaskCli.render(tasks)
      {:error, reason} -> "Error: #{reason}"
    end
  end

  def run(:help) do
    """
    Usage:
      task_cli add TITLE [--priority low|normal|high]
      task_cli list
      task_cli done ID
    """
  end

  defp parse_id(id) do
    case Integer.parse(id) do
      {number, ""} -> {:complete, number}
      _other -> :help
    end
  end
end

```

`parse/1` and `run/1` are separate and both public, which is what makes the CLI testable without spawning a process or capturing standard output. `main/1` is three lines of glue.

The custom Mix task

```

# lib/mix/tasks/tasks.report.ex
defmodule Mix.Tasks.Tasks.Report do
  @moduledoc """
  Prints a summary of the pending task list.

      mix tasks.report

  """
  @shortdoc "Summarises pending tasks by priority"

  use Mix.Task

  @impl Mix.Task
  def run(_args) do
    Mix.Task.run("app.start")

    case TaskCli.list() do
      {:ok, tasks} ->
        counts = TaskCli.pending_by_priority(tasks)

        Mix.shell().info("""
        Tasks:      #{length(tasks)}
        Pending:    #{counts |> Map.values() |> Enum.sum()}
        By priority: #{inspect(counts)}
        """)
    end
  end
end

```

```

{:error, reason} ->
  Mix.shell().error("Could not read tasks: #{inspect(reason)}")
end
end
end

```

The tests

```

# test/task_cli_test.exs
defmodule TaskCliTest do
  use ExUnit.Case, async: true
  doctest TaskCli
  doctest TaskCli.CLI

  describe "add/3" do
    test "assigns sequential ids" do
      assert {:ok, tasks} = TaskCli.add([], "first", "low")
      assert {:ok, tasks} = TaskCli.add(tasks, "second", "high")
      assert Enum.map(tasks, &1["id"]) == [1, 2]
    end

    test "ids continue after a removal" do
      {:ok, tasks} = TaskCli.add([], "first", "low")
      {:ok, tasks} = TaskCli.add(tasks, "second", "low")
      {:ok, tasks} = TaskCli.remove(tasks, 1)
      {:ok, tasks} = TaskCli.add(tasks, "third", "low")

      assert Enum.map(tasks, &1["id"]) == [2, 3]
    end

    test "rejects an empty title" do
      assert {:error, :empty_title} = TaskCli.add([], "", "low")
    end

    test "rejects an unknown priority" do
      assert {:error, :invalid_priority} = TaskCli.add([], "x", "urgent")
    end
  end

  describe "complete/2 and remove/2" do
    setup do
      {:ok, tasks} = TaskCli.add([], "first", "low")
      {:ok, tasks} = TaskCli.add(tasks, "second", "high")
      {:ok, tasks: tasks}
    end

    test "completes only the named task", %{tasks: tasks} do
      assert {:ok, updated} = TaskCli.complete(tasks, 1)
      assert Enum.map(updated, &1["done"]) == [true, false]
    end

    test "reports a missing id", %{tasks: tasks} do
      assert {:error, :not_found} = TaskCli.complete(tasks, 99)
      assert {:error, :not_found} = TaskCli.remove(tasks, 99)
    end

    test "removes by id", %{tasks: tasks} do
      assert {:ok, [remaining]} = TaskCli.remove(tasks, 1)
      assert remaining["id"] == 2
    end
  end
end

```

```

    end
  end

  describe "reporting" do
    test "counts pending by priority" do
      tasks = [
        %{id => 1, "title" => "a", "priority" => "high", "done" => false},
        %{id => 2, "title" => "b", "priority" => "high", "done" => true},
        %{id => 3, "title" => "c", "priority" => "low", "done" => false}
      ]

      assert TaskCli.pending_by_priority(tasks) == %{"high" => 1, "low" => 1}
    end

    test "renders with a done marker" do
      tasks = [
        %{id => 1, "title" => "a", "priority" => "high", "done" => false},
        %{id => 2, "title" => "b", "priority" => "low", "done" => true}
      ]

      assert TaskCli.render(tasks) == "  1 [ ] (high) a\n  2 [x] (low) b"
    end
  end

  describe "CLI parsing" do
    test "defaults the priority" do
      assert {:add, "x", "normal"} = TaskCli.CLI.parse(["add", "x"])
    end

    test "rejects a non-numeric id" do
      assert :help = TaskCli.CLI.parse(["done", "abc"])
    end

    test "unknown commands show help" do
      assert TaskCli.CLI.run(:help) =~ "Usage:"
    end
  end
end

```

```

# test/task_cli/store_test.exs
defmodule TaskCli.StoreTest do
  use ExUnit.Case, async: true

  alias TaskCli.Store

  setup do
    path =
      Path.join(System.tmp_dir!(), "task_cli_#{System.unique_integer([:positive])}.json")

    on_exit(fn -> File.rm(path) end)
    {:ok, path: path}
  end

  test "a missing file loads as an empty list", %{path: path} do
    assert {:ok, []} = Store.load(path)
  end

  test "round trips through JSON", %{path: path} do
    tasks = [%{"id" => 1, "title" => "a", "priority" => "low", "done" => false}]
  end
end

```

```

    assert :ok = Store.save(tasks, path)
    assert {:ok, ^tasks} = Store.load(path)
  end

  test "reports a malformed store", %{path: path} do
    File.write!(path, "this is not json")
    assert {:error, :malformed_store} = Store.load(path)
  end

  test "reports JSON that is not a list", %{path: path} do
    File.write!(path, ~s({"not": "a list"}))
    assert {:error, :malformed_store} = Store.load(path)
  end

  test "creates missing directories", %{path: path} do
    nested = Path.join([Path.dirname(path), "nested_#{System.unique_integer([:positive])}",
      "t.json"])
    on_exit(fn -> File.rm_rf(Path.dirname(nested)) end)

    assert :ok = Store.save([], nested)
    assert {:ok, []} = Store.load(nested)
  end

  test "the configured path is used by default" do
    assert Store.path() == "tmp/test_tasks.json"
  end
end

```

```
mix test
```

```

Running ExUnit with seed: 81708, max_cases: 24
.....
Finished in 0.08 seconds (0.08s async, 0.00s sync)

Result: 33 passed (15 doctests, 18 tests)

```

Both modules are `async: true`, because the pure functions have no state and the store tests pass an explicit path rather than mutating configuration. The round-trip test uses the pin operator, `assert {:ok, ^tasks} = Store.load(path)`, which asserts the exact same value came back.

Build and run the escript

```
mix escript.build
```

```

Compiling 4 files (.ex)
Generated escript task_cli with MIX_ENV=dev

```

```

./task_cli add "Write slides" --priority high
./task_cli add "Book room"
./task_cli done 1
./task_cli list

```

```

1 [ ] (high) Write slides
1 [ ] (high) Write slides

```

```
2 [ ] (normal) Book room
1 [x] (high) Write slides
2 [ ] (normal) Book room
1 [x] (high) Write slides
2 [ ] (normal) Book room
```

Each command prints the whole list after acting, so the output accumulates down the page.

```
mix tasks.report
```

```
Tasks:      2
Pending:    1
By priority: %{"normal" => 1}
```

Compile, run, package

Module 11 described three stages: build the code, run it locally, package it. Walk through all three on this project.

Build. Make warnings fatal, as CI would:

```
mix compile --warnings-as-errors
```

```
Compiling 4 files (.ex)
```

Run it again and it prints nothing: compilation is incremental, and nothing changed. The output is plain bytecode in `_build/dev/lib/task_cli/ebin/`:

```
Elixir.Mix.Tasks.Tasks.Report.beam
Elixir.TaskCli.beam
Elixir.TaskCli.CLI.beam
Elixir.TaskCli.Store.beam
task_cli.app
```

Run. `mix run -e` boots the application and evaluates one expression. Use it to see configuration change with the environment:

```
mix run -e 'IO.inspect({Mix.env(), TaskCli.Store.path()})'
MIX_ENV=test mix run -e 'IO.inspect({Mix.env(), TaskCli.Store.path()})'
```

On Windows PowerShell, set the environment first:

```
mix run -e 'IO.inspect({Mix.env(), TaskCli.Store.path()})'
$env:MIX_ENV = "test"; mix run -e 'IO.inspect({Mix.env(), TaskCli.Store.path()})'
Remove-Item Env:MIX_ENV
```

```
{:dev, "tasks.json"}
Compiling 4 files (.ex)
{:test, "tmp/test_tasks.json"}
```


PowerShell and `mix run -e`. In PowerShell, `mix` goes through `mix.ps1` and then `cmd`, which can mangle the expression. `mix run -e 'IO.inspect(TaskCli.list())'` fails with `) was unexpected at this time`, and double quotes inside the expression are stripped. Run it through `cmd` instead: `cmd /c 'mix run -e "IO.inspect(TaskCli.list())"'`, which prints `{:ok, []}`.

The test run may print `Compiling` because `:test` has its own `_build/test` directory, built separately from `_build/dev`. `config/test.exs` changed the store path without touching any code.

Now look at what `mix run` actually started:

```
mix run -e 'IO.inspect(Application.started_applications() |> Enum.map(&elem(&1, 0)))'
```

```
[:task_cli, :jason, :logger, :hex, :inets, :ssl, :public_key, :asn1, :crypto,
:mix, :elixir, :compiler, :stdlib, :kernel]
```

`:mix` and `:hex` are running because Mix is running your code. Calling `Mix.env()` works here for the same reason. Neither would exist in a release, which is why application code must not depend on them.

Package. This lab packages with an escript. The alternative is a release, built in Lab 20:

	Escript	Release
Built with	<code>mix escript.build</code>	<code>MIX_ENV=prod mix release</code>
Output	One executable file	A directory
Contains	Your app, deps and Elixir, as bytecode	Your app, deps, Elixir and the Erlang runtime
Needs Erlang on the target?	Yes	No, by default
Suited to	Command-line tools	Long-running services

Note that the escript above reported `MIX_ENV=dev`. Neither command switches to `:prod` for you.

Discussion points

Ask the room to name every place configuration changed behaviour. There are two: the store path in test versus dev, and the default priority. Then ask what would have broken if `Store.path/0` had been a module attribute instead of a function. The answer is that the test environment override would have been ignored, because attributes are resolved at compile time. That is the Module 11 pitfall, made concrete.

Finally, ask why `Mix.env()` worked in `mix run` but would crash in a release. `Mix.Tasks.Tasks.Report` is compiled into `ebin/` like any other module, yet it could never run in a release. Mix tasks are build-time tools.

Lab 12 — Structs, Protocols and Behaviours

After Module 12 · 45 minutes

Objective

Build a notification system with a message struct, an auditing protocol and a delivery-channel behaviour.

Steps

1. Create a project called `notifier`.
2. Define a `Notifier.Message` struct with enforced keys and a `t()` type.
3. Implement `String.Chars` and `Inspect` for it.
4. Define a `Notifier.Auditable` protocol with three implementations.
5. Define a `Notifier.Channel` behaviour with three implementations.
6. Dispatch to a channel chosen by configuration, and to the first channel that supports a message.

Requirements

- `Message` enforces `:to` and `:body`, defaults `priority` to `:normal`
- The custom `Inspect` implementation must **not** print the body
- `Auditable` is implemented for the message struct, for `Map` and for `List`
- `Channel` declares `name/0`, `deliver/1` and an **optional** `supports?/1`
- Email requires an at sign, SMS requires a `+` and digits and rejects bodies over 160 bytes
- The null channel omits `supports?/1` entirely and still compiles
- `Notifier.send_via_first/2` picks the first channel that supports the message

Hints

- `@optional_callbacks [supports?: 1]`
- `defimpl Inspect` with `Inspect.Algebra.concat/1`
- `function_exported?/3` tells you whether an optional callback was implemented
- `module.__info__(:attributes)[:behaviour]` lists a module's declared behaviours

Lab 12 Solution

```
mix new notifier
cd notifier
```

The struct and its protocol implementations

```

# lib/notifier/message.ex
defmodule Notifier.Message do
  @moduledoc """
  A message to be delivered through some channel.
  """

  @enforce_keys [:to, :body]
  defstruct [:to, :body, :subject, priority: :normal, metadata: %{}]

  @type t :: %__MODULE__{
    to: String.t(),
    body: String.t(),
    subject: String.t() | nil,
    priority: :low | :normal | :urgent,
    metadata: map()
  }

  @doc """
  Builds a message.

  ## Examples

      iex> Notifier.Message.new("a@example.com", "hello")
      %Notifier.Message{to: "a@example.com", body: "hello", subject: nil,
        priority: :normal, metadata: %{}}

  """
  @spec new(String.t(), String.t(), keyword()) :: t()
  def new(to, body, opts \\ []) do
    struct!(__MODULE__, [to: to, body: body] ++ opts)
  end

  @doc """
  Returns `true` when the message must be delivered immediately.

  ## Examples

      iex> Notifier.Message.urgent?(Notifier.Message.new("a", "b", priority: :urgent))
      true

      iex> Notifier.Message.urgent?(Notifier.Message.new("a", "b"))
      false

  """
  @spec urgent?(t()) :: boolean()
  def urgent?(%__MODULE__{priority: :urgent}), do: true
  def urgent?(%__MODULE__{}), do: false
end

defimpl String.Chars, for: Notifier.Message do
  def to_string(%Notifier.Message{to: to, subject: nil}), do: "message to #{to}"
  def to_string(%Notifier.Message{to: to, subject: subject}), do: "#{subject} to #{to}"
end

defimpl Inspect, for: Notifier.Message do
  import Inspect.Algebra

  # The body may contain personal data, so it never reaches a log line.
  def inspect(%Notifier.Message{to: to, priority: priority, _opts: _}) do
    concat(["#Notifier.Message<", to, " ", Atom.to_string(priority), ">"])
  end
end

```

```
end
end
```

The **Inspect** implementation is the part to dwell on. Anything logged with `inspect/1`, printed in an IEx session, or included in an error report now shows only the recipient and priority. The body cannot leak by accident. This is a genuine production technique, not an exercise.

The behaviour and its implementations

```
# lib/notifier/channel.ex
defmodule Notifier.Channel do
  @moduledoc """
  Contract every delivery channel must satisfy.
  """

  alias Notifier.Message

  @doc "A short name used in logs and reports."
  @callback name() :: String.t()

  @doc "Delivers the message, returning a receipt or an error."
  @callback deliver(Message.t()) :: {:ok, map()} | {:error, term()}

  @doc "Whether this channel can carry the given message at all."
  @callback supports?(Message.t()) :: boolean()

  @optional_callbacks [supports?: 1]
end

defmodule Notifier.Channel.Email do
  @moduledoc "Delivers by email. Requires an address containing an at sign."

  @behaviour Notifier.Channel

  alias Notifier.Message

  @impl Notifier.Channel
  def name, do: "email"

  @impl Notifier.Channel
  def supports?(%Message{to: to}), do: String.contains?(to, "@")

  @impl Notifier.Channel
  def deliver(%Message{} = message) do
    if supports?(message) do
      {:ok, %{channel: name(), to: message.to, id: "email-#{:erlang.phash2(message.body)}"}}
    else
      {:error, :not_an_email_address}
    end
  end
end

defmodule Notifier.Channel.Sms do
  @moduledoc "Delivers by SMS. Requires a leading plus and digits only."

  @behaviour Notifier.Channel

  alias Notifier.Message
```

```

@max_length 160

@impl Notifier.Channel
def name, do: "sms"

@impl Notifier.Channel
def supports?(%Message{to: to}), do: Regex.match?(~r/^\d{6,15}$/, to)

@impl Notifier.Channel
def deliver(%Message{body: body}) when byte_size(body) > @max_length do
  {:error, {:too_long, byte_size(body), @max_length}}
end

def deliver(%Message{} = message) do
  if supports?(message) do
    {:ok, %{channel: name(), to: message.to, id: "sms-#{:erlang.phash2(message.body)}}"}
  else
    {:error, :not_a_phone_number}
  end
end

defmodule Notifier.Channel.Null do
  @moduledoc "Accepts everything and delivers nothing. Used in tests."

  @behaviour Notifier.Channel

  @impl Notifier.Channel
  def name, do: "null"

  @impl Notifier.Channel
  def deliver(_message), do: {:ok, %{channel: "null", to: nil, id: "null"}}
end

```

`Notifier.Channel.Null` declares the behaviour and implements only two of the three callbacks. It compiles without warning because `supports?: 1` is optional. Delete the `@optional_callbacks` line and recompile in front of the room; the warning that appears is the point of the exercise.

The protocol and the public API

```

# lib/notifier.ex
defprotocol Notifier.Auditable do
  @doc "Returns a one-line audit entry for the value."
  def audit_entry(value)
end

defimpl Notifier.Auditable, for: Notifier.Message do
  def audit_entry(message) do
    "MESSAGE #{message.priority} -> #{message.to}"
  end
end

defimpl Notifier.Auditable, for: Map do
  def audit_entry(%{channel: channel, id: id}), do: "RECEIPT #{channel} #{id}"
  def audit_entry(other), do: "RECEIPT unknown #{inspect(Map.keys(other))}"
end

defimpl Notifier.Auditable, for: List do
  def audit_entry(items) do

```

```

    Enum.map_join(items, "; ", &Notifier.Auditable.audit_entry/1)
  end
end

defmodule Notifier do
  @moduledoc """
  Sends messages through a channel chosen by configuration.
  """

  alias Notifier.{Auditable, Message}

  @doc "The configured channel module."
  @spec channel() :: module()
  def channel, do: Application.get_env(:notifier, :channel, Notifier.Channel.Email)

  @doc """
  Delivers a message through the configured channel.
  """
  @spec send(Message.t()) :: {:ok, map()} | {:error, term()}
  def send(%Message{} = message), do: channel().deliver(message)

  @doc """
  Delivers a message through the first channel that supports it.

  Channels that do not implement `supports?/1` are treated as accepting
  everything.
  """
  @spec send_via_first([module()], Message.t()) :: {:ok, map()} | {:error, :no_channel}
  def send_via_first(channels, %Message{} = message) do
    channels
    |> Enum.filter(&supports?(&1, message))
    |> deliver_first(message)
  end

  @doc """
  Builds an audit line for anything implementing `Notifier.Auditable`.
  """
  @spec audit(term()) :: String.t()
  def audit(value), do: Auditable.audit_entry(value)

  # Code.ensure_loaded?/1 matters: function_exported?/3 answers false for a
  # module the VM has not loaded yet, which silently defeats this check.
  defp supports?(module, message) do
    if Code.ensure_loaded?(module) and function_exported?(module, :supports?, 1) do
      module.supports?(message)
    else
      true
    end
  end

  defp deliver_first([], _message), do: {:error, :no_channel}
  defp deliver_first([module | _rest], message), do: module.deliver(message)
end

```

The bug this lab is really about

The first version of `supports?/2` was written without `Code.ensure_loaded?/1`:

```
# Subtly broken
```

```
defp supports?(module, message) do
  if function_exported?(module, :supports?, 1) do
    module.supports?(message)
  else
    true
  end
end
```

Every test passed except one:

```
1) test send_via_first/2 reports when nothing supports the message (NotifierTest)
   left: {:error, :no_channel}
   right: {:error, :not_an_email_address}
```

`function_exported?/3` returns `false` for a module the VM has not loaded yet. The BEAM loads modules lazily. `Notifier.Channel.Email` implements `supports?/1`, but because nothing had called into that module yet in that test, `function_exported?/3` said no, the filter treated the channel as accepting everything, and delivery was attempted anyway.

`Code.ensure_loaded?/1` forces the module to load before the check. This is a real production bug pattern in Elixir, and a lab is a cheap place to meet it. Anywhere you interrogate a module dynamically, ensure it is loaded first.

The tests

```
# test/notifier_test.exs
defmodule NotifierTest do
  use ExUnit.Case
  doctest Notifier.Message

  alias Notifier.Channel
  alias Notifier.Message

  describe "Message struct" do
    test "enforces the required keys" do
      assert_raise ArgumentError, fn -> struct!(Message, %{to: "a@example.com"}) end
    end

    test "applies defaults" do
      message = Message.new("a@example.com", "hello")
      assert message.priority == :normal
      assert message.subject == nil
      assert message.metadata == %{}
    end

    test "String.Chars uses the subject when present" do
      assert to_string(Message.new("a@example.com", "b")) == "message to a@example.com"

      assert to_string(Message.new("a@example.com", "b", subject: "Invoice")) ==
        "Invoice to a@example.com"
    end

    test "Inspect hides the body" do
      message = Message.new("a@example.com", "SECRET BODY", priority: :urgent)
      rendered = inspect(message)
```

```

    refute rendered =~ "SECRET"
    assert rendered == "#Notifier.Message<a@example.com urgent>"
  end
end

describe "Auditable protocol" do
  test "audits a message" do
    assert Notifier.audit(Message.new("a@example.com", "b", priority: :urgent)) ==
      "MESSAGE urgent -> a@example.com"
  end

  test "audits a receipt map" do
    assert Notifier.audit(%{channel: "email", to: "a", id: "x1"}) == "RECEIPT email x1"
  end

  test "audits a list of anything auditable" do
    entries = [
      Message.new("a@example.com", "b"),
      %{channel: "email", to: "a@example.com", id: "x1"}
    ]

    assert Notifier.audit(entries) ==
      "MESSAGE normal -> a@example.com; RECEIPT email x1"
  end

  test "raises for a type with no implementation" do
    assert_raise Protocol.UndefinedError, fn -> Notifier.audit("plain string") end
  end
end

describe "Channel behaviour" do
  test "every channel declares the behaviour" do
    for module <- [Channel.Email, Channel.Sms, Channel.Null] do
      assert Notifier.Channel in module.__info__(:attributes)[:behaviour]
    end
  end

  test "email accepts an address and rejects a phone number" do
    assert {:ok, receipt} = Channel.Email.deliver(Message.new("a@example.com", "hi"))
    assert receipt.channel == "email"
    assert {:error, :not_an_email_address} = Channel.Email.deliver(Message.new("+911234567",
      "hi"))
  end

  test "sms accepts a phone number and rejects an address" do
    assert {:ok, receipt} = Channel.Sms.deliver(Message.new("+911234567", "hi"))
    assert receipt.channel == "sms"
    assert {:error, :not_a_phone_number} = Channel.Sms.deliver(Message.new("a@example.com",
      "hi"))
  end

  test "sms rejects an over-long body" do
    long = String.duplicate("x", 161)
    assert {:error, {:too_long, 161, 160}} = Channel.Sms.deliver(Message.new("+911234567",
      long))
  end

  test "the null channel accepts everything and omits supports?/1" do
    assert {:ok, %{channel: "null"}} = Channel.Null.deliver(Message.new("anything", "hi"))
    refute function_exported?(Channel.Null, :supports?, 1)
  end
end

```



```

end

describe "send/1 with configuration" do
  setup do
    original = Application.get_env(:notifier, :channel)
    on_exit(fn -> Application.put_env(:notifier, :channel, original) end)
    :ok
  end

  test "uses the default channel" do
    Application.delete_env(:notifier, :channel)
    assert Notifier.channel() == Channel.Email
    assert {:ok, %{channel: "email"}} = Notifier.send(Message.new("a@example.com", "hi"))
  end

  test "configuration swaps the channel" do
    Application.put_env(:notifier, :channel, Channel.Null)
    assert {:ok, %{channel: "null"}} = Notifier.send(Message.new("anything", "hi"))
  end
end

describe "send_via_first/2" do
  test "picks the channel that supports the recipient" do
    channels = [Channel.Email, Channel.Sms]

    assert {:ok, %{channel: "email"}} =
      Notifier.send_via_first(channels, Message.new("a@example.com", "hi"))

    assert {:ok, %{channel: "sms"}} =
      Notifier.send_via_first(channels, Message.new("+911234567", "hi"))
  end

  test "a channel without supports?/1 accepts everything" do
    assert {:ok, %{channel: "null"}} =
      Notifier.send_via_first([Channel.Null], Message.new("!!!", "hi"))
  end

  test "reports when nothing supports the message" do
    assert {:error, :no_channel} =
      Notifier.send_via_first([Channel.Email, Channel.Sms], Message.new("!!!", "hi"))
  end
end
end

```

```
mix test
```

```

Running ExUnit with seed: 422678, max_cases: 24

.....
Finished in 0.1 seconds (0.00s async, 0.1s sync)

Result: 21 passed (3 doctests, 18 tests)

```

This module is **not** `async: true`, because two tests mutate application environment. That is the honest trade-off: configuration-based dispatch is simple and needs no library, and the price is that those particular tests are serial.

Stretch goal

Add a `Notifier.Channel.Retrying` that wraps another channel and retries a configurable number of times. It must itself implement the `Channel` behaviour, which makes it composable with everything else. That is the decorator pattern, achieved with no inheritance.

Lab 13 — Error Handling

After Module 13 · 40 minutes

Objective

Build a configuration loader that offers both a tuple-returning and a raising API, with custom exceptions carrying structured fields.

Steps

1. Create a project called `config_loader`.
2. Define three custom exceptions with structured fields.
3. Parse `KEY=value` text, reporting the line number of a bad line.
4. Provide `fetch/2`, `fetch!/2`, `fetch_integer/2`, `fetch_integer!/2`.
5. Add a `boundary/1` that converts known exceptions and lets others through.

Requirements

- `parse/1` ignores blank lines and `#` comments, and reports `{:error, {:parse_error, line, text}}`
- `parse/1` stops at the **first** bad line
- Values may contain `=`, so only split on the first one
- `fetch!/2` raises `MissingKeyError` listing the available keys
- `fetch_integer!/2` raises a **different** exception for a missing key and for an unparseable value
- `load_file/1` treats a missing file as an error tuple, not an exception
- `boundary/1` rescues exactly the three known exceptions and no others

Hints

- `Enum.reduce_while/3` gives you the early exit on a bad line
- `String.split(line, "=", parts: 2)` keeps equals signs in the value
- `Integer.parse/1` returning `{n, ""}` is the full-string check
- `defexception [:key, :available]` plus a `message/1` callback
- `with {:ok, raw} <- fetch(config, key)` propagates the missing-key error for free

Lab 13 Solution

```
mix new config_loader
cd config_loader
```

The exceptions

```
# lib/config_loader/errors.ex
defmodule ConfigLoader.MissingKeyError do
  @moduledoc "Raised when a required configuration key is absent."

  defexception [:key, :available]

  @impl true
  def message(%__MODULE__{key: key, available: available}) do
    "missing required key #{inspect(key)}; available keys: #{inspect(Enum.sort(available))}"
  end
end

defmodule ConfigLoader.InvalidValueError do
  @moduledoc "Raised when a key is present but its value is unusable."

  defexception [:key, :value, :expected]

  @impl true
  def message(%__MODULE__{key: key, value: value, expected: expected}) do
    "key #{inspect(key)} expected #{inspect(expected)}, got: #{inspect(value)}"
  end
end

defmodule ConfigLoader.ParseError do
  @moduledoc "Raised when the source text cannot be parsed at all."

  defexception [:line_number, :line]

  @impl true
  def message(%__MODULE__{line_number: n, line: line}) do
    "could not parse line #{n}: #{inspect(line)}"
  end
end
```

Every exception carries **structured fields**, and `message/1` builds the English from them. A caller that rescues `MissingKeyError` can read `e.key` and `e.available` rather than parsing a sentence. `MissingKeyError` listing the available keys is a small thing that saves real debugging time.

The loader

```
# lib/config_loader.ex
defmodule ConfigLoader do
  @moduledoc """
  Parses `KEY=value` configuration text and reads typed values from it.

  Every operation comes in two forms: a tuple-returning one for expected
  failures, and a bang one that raises when a broken assumption means the
  program cannot sensibly continue.
  """

  alias ConfigLoader.{InvalidValueError, MissingKeyError, ParseError}

  @type config :: %{optional(String.t()) => String.t()}

  @doc """
  Parses configuration text into a map.
```

Blank lines and lines starting with `#` are ignored.

Examples

```
iex> ConfigLoader.parse("PORT=4000\\n# a comment\\nHOST=localhost")
{:ok, %{ "PORT" => "4000", "HOST" => "localhost"}}
```

```
iex> ConfigLoader.parse("")
{:ok, %{}}
```

```
iex> ConfigLoader.parse("PORT=4000\\nnonsense")
{:error, {:parse_error, 2, "nonsense"}}
```

"""

```
@spec parse(String.t()) :: {:ok, config()} | {:error, {:parse_error, pos_integer(),
String.t()}}
```

```
def parse(text) do
  text
  |> String.split("\n")
  |> Enum.with_index(1)
  |> Enum.reduce_while({:ok, %{}}, &parse_line/2)
end
```

@doc """

Parses configuration text, raising on a malformed line.

Examples

```
iex> ConfigLoader.parse!("PORT=4000")
%{ "PORT" => "4000"}
```

"""

```
@spec parse!(String.t()) :: config()
```

```
def parse!(text) do
  case parse(text) do
    {:ok, config} -> config
    {:error, {:parse_error, n, line}} -> raise ParseError, line_number: n, line: line
  end
end
```

@doc """

Reads a required string value.

Examples

```
iex> ConfigLoader.fetch(%{ "HOST" => "localhost"}, "HOST")
{:ok, "localhost"}
```

```
iex> ConfigLoader.fetch(%{}, "HOST")
{:error, {:missing_key, "HOST"}}
```

"""

```
@spec fetch(config(), String.t()) :: {:ok, String.t()} | {:error, {:missing_key, String.t()}}
```

```
def fetch(config, key) do
  case Map.fetch(config, key) do
    {:ok, value} -> {:ok, value}
    :error -> {:error, {:missing_key, key}}
  end
end
```

```

@doc """
Reads a required string value, raising when it is absent.

## Examples

    iex> ConfigLoader.fetch!({"HOST" => "localhost"}, "HOST")
    "localhost"

"""
@spec fetch!(config(), String.t()) :: String.t()
def fetch!(config, key) do
  case fetch(config, key) do
    {:ok, value} -> value
    {:error, {:missing_key, key}} -> raise MissingKeyError, key: key, available:
      Map.keys(config)
  end
end

@doc """
Reads a required integer value.

## Examples

    iex> ConfigLoader.fetch_integer({"PORT" => "4000"}, "PORT")
    {:ok, 4000}

    iex> ConfigLoader.fetch_integer({"PORT" => "abc"}, "PORT")
    {:error, {:invalid_value, "PORT", "abc"}}

    iex> ConfigLoader.fetch_integer({}, "PORT")
    {:error, {:missing_key, "PORT"}}

"""
@spec fetch_integer(config(), String.t()) ::
  {:ok, integer()}
  | {:error, {:missing_key, String.t()} | {:invalid_value, String.t(), String.t()}}
def fetch_integer(config, key) do
  with {:ok, raw} <- fetch(config, key) do
    case Integer.parse(raw) do
      {number, ""} -> {:ok, number}
      _other -> {:error, {:invalid_value, key, raw}}
    end
  end
end

@doc """
Reads a required integer value, raising on either failure.

## Examples

    iex> ConfigLoader.fetch_integer!({"PORT" => "4000"}, "PORT")
    4000

"""
@spec fetch_integer!(config(), String.t()) :: integer()
def fetch_integer!(config, key) do
  case fetch_integer(config, key) do
    {:ok, number} ->
      number
    {:error, {:missing_key, key}} ->

```

```

        raise MissingKeyError, key: key, available: Map.keys(config)

        {:_error, {:_invalid_value, key, value}} ->
            raise InvalidValueError, key: key, value: value, expected: "an integer"
        end
    end

    @doc """
    Reads an optional value with a default.

    ## Examples

    iex> ConfigLoader.get(%{"HOST" => "localhost"}, "HOST", "0.0.0.0")
    "localhost"

    iex> ConfigLoader.get(%{}, "HOST", "0.0.0.0")
    "0.0.0.0"

    """
    @spec get(config(), String.t(), String.t()) :: String.t()
    def get(config, key, default), do: Map.get(config, key, default)

    @doc """
    Validates that every listed key is present.

    ## Examples

    iex> ConfigLoader.require_keys(%{"A" => "1", "B" => "2"}, ["A", "B"])
    :ok

    iex> ConfigLoader.require_keys(%{"A" => "1"}, ["A", "B", "C"])
    {:_error, {:_missing_keys, ["B", "C"]}}

    """
    @spec require_keys(config(), [String.t()]) :: :ok | {:_error, {:_missing_keys, [String.t()]}}
    def require_keys(config, keys) do
        case Enum.reject(keys, &Map.has_key?(config, &1)) do
            [] -> :ok
            missing -> {:_error, {:_missing_keys, Enum.sort(missing)}}
        end
    end

    @doc """
    Loads configuration from a file.

    A missing file is an expected outcome and returns an error tuple.

    """
    @spec load_file(Path.t()) :: {:_ok, config()} | {:_error, term()}
    def load_file(path) do
        with {:_ok, contents} <- File.read(path) do
            parse(contents)
        end
    end

    @doc """
    Runs `fun` and converts any ConfigLoader exception into an error tuple.

    This is the system boundary: it rescues the exceptions it understands
    and lets every other exception through untouched.

    """
    @spec boundary((-> result)) :: {:_ok, result} | {:_error, tuple()} when result: term()

```

```

def boundary(fun) when is_function(fun, 0) do
  {:ok, fun.()}
rescue
  e in MissingKeyError -> {:error, {:missing_key, e.key}}
  e in InvalidValueError -> {:error, {:invalid_value, e.key, e.value}}
  e in ParseError -> {:error, {:parse_error, e.line_number}}
end

# ----- private

defp parse_line({line, number}, {:ok, config}) do
  trimmed = String.trim(line)

  cond do
    trimmed == "" -> {:cont, {:ok, config}}
    String.starts_with?(trimmed, "#") -> {:cont, {:ok, config}}
    true -> parse_pair(trimmed, number, config)
  end
end

defp parse_pair(line, number, config) do
  case String.split(line, "=", parts: 2) do
    [key, value] when key != "" ->
      {:cont, {:ok, Map.put(config, String.trim(key), String.trim(value))}}

    _other ->
      {:halt, {:error, {:parse_error, number, line}}}
  end
end
end

```

Four points for the debrief.

fetch_integer/2 uses with for one step. That looks odd until you see why: if **fetch/2** returns **{:error, {:missing_key, key}}**, **with** returns it unchanged and the integer parsing never runs. The missing-key error propagates with no code at all.

fetch_integer!/2 raises two different exceptions. A missing key and an unparseable value are different problems with different fixes, so they get different types. A single **ArgumentError** would have thrown that distinction away.

load_file/1 returns {:error, :enoent} for a missing file. A missing config file is an expected condition, so it is a value, not an exception.

boundary/1 lists three exception modules explicitly. A bare **rescue e ->** would swallow a genuine bug and return a plausible-looking error tuple. The test suite in Lab 14 proves it does not.

Write a handful of tests to confirm parsing works, then move on. Lab 14 builds the complete suite.

Lab 14 — Testing with ExUnit

After Module 14 · 45 minutes

Objective

Write a complete, production-quality test suite for the `config_loader` module from Lab 13, using the full ExUnit vocabulary.

Steps

1. Stay in the `config_loader` project.
2. Group tests with `describe` blocks, one per public function.
3. Use `setup` with `on_exit` for the file tests.
4. Use `assert_raise` to inspect the structured fields of custom exceptions.
5. Make the module `async: true`, and be able to justify it.

Requirements

- Every public function has a `describe` block
- Each block covers the happy path, at least one edge case and the empty or missing input
- The file tests use a uniquely named temporary file and clean up after themselves
- At least one test inspects a custom exception's fields, not just its type
- At least one test proves `boundary/1` does **not** rescue an unknown exception
- The module is `async: true`

Hints

- `assert_raise/2` returns the exception
- `System.unique_integer([:positive])` makes a collision-proof filename
- `on_exit/1` runs even when the test fails
- `~s(...)` writes a string containing double quotes without escaping

Lab 14 Solution

```
# test/config_loader_test.exs
defmodule ConfigLoaderTest do
  use ExUnit.Case, async: true
  doctest ConfigLoader

  alias ConfigLoader.{InvalidValueError, MissingKeyError, ParseError}

  @text ""
```

```

# Application settings
PORT=4000

HOST=localhost
DEBUG=true
"""

describe "parse/1" do
  test "ignores comments and blank lines" do
    assert {:ok, config} = ConfigLoader.parse(@text)
    assert config == %{"PORT" => "4000", "HOST" => "localhost", "DEBUG" => "true"}
  end

  test "trims whitespace around keys and values" do
    assert {:ok, %{"A" => "1"}} = ConfigLoader.parse(" A = 1 ")
  end

  test "keeps equals signs in the value" do
    assert {:ok, %{"URL" => "a=b=c"}} = ConfigLoader.parse("URL=a=b=c")
  end

  test "allows an empty value" do
    assert {:ok, %{"EMPTY" => ""}} = ConfigLoader.parse("EMPTY=")
  end

  test "reports the failing line number" do
    assert {:error, {:parse_error, 3, "nonsense"}} =
      ConfigLoader.parse("A=1\nB=2\nnonsense")
  end

  test "stops at the first bad line" do
    assert {:error, {:parse_error, 1, "bad"}} = ConfigLoader.parse("bad\nalso bad")
  end
end

describe "parse!/1" do
  test "returns the map on success" do
    assert ConfigLoader.parse!("A=1") == %{"A" => "1"}
  end

  test "raises with the line number" do
    error = assert_raise ParseError, fn -> ConfigLoader.parse!("A=1\nnope") end

    assert error.line_number == 2
    assert Exception.message(error) == ~s(could not parse line 2: "nope")
  end
end

describe "fetch/2 and fetch!/2" do
  setup do
    {:ok, config: ConfigLoader.parse!(@text)}
  end

  test "returns a present value", %{config: config} do
    assert {:ok, "localhost"} = ConfigLoader.fetch(config, "HOST")
    assert ConfigLoader.fetch!(config, "HOST") == "localhost"
  end

  test "reports a missing key as a tuple", %{config: config} do
    assert {:error, {:missing_key, "NOPE"}} = ConfigLoader.fetch(config, "NOPE")
  end
end

```

```

test "raises with the available keys listed", %{config: config} do
  error = assert_raise MissingKeyError, fn -> ConfigLoader.fetch!(config, "NOPE") end

  assert error.key == "NOPE"
  assert Enum.sort(error.available) == ["DEBUG", "HOST", "PORT"]
  assert Exception.message(error) =~ ~s(available keys: ["DEBUG", "HOST", "PORT"])
end
end

describe "fetch_integer/2 and fetch_integer!/2" do
  test "parses a valid integer" do
    assert {:ok, 4000} = ConfigLoader.fetch_integer(%{"PORT" => "4000"}, "PORT")
    assert ConfigLoader.fetch_integer!(%{"PORT" => "4000"}, "PORT") == 4000
  end

  test "rejects trailing characters" do
    assert {:error, {:invalid_value, "PORT", "4000abc"}} =
      ConfigLoader.fetch_integer(%{"PORT" => "4000abc"}, "PORT")
  end

  test "propagates a missing key without touching the parser" do
    assert {:error, {:missing_key, "PORT"}} = ConfigLoader.fetch_integer(%{}, "PORT")
  end

  test "raises a different exception for each failure" do
    assert_raise MissingKeyError, fn -> ConfigLoader.fetch_integer!(%{}, "PORT") end

    error =
      assert_raise InvalidValueError, fn ->
        ConfigLoader.fetch_integer!(%{"PORT" => "abc"}, "PORT")
      end

    assert error.expected == "an integer"
    assert Exception.message(error) == ~s(key "PORT" expected an integer, got: "abc")
  end
end

describe "require_keys/2" do
  test "accepts a complete config" do
    assert :ok = ConfigLoader.require_keys(%{"A" => "1", "B" => "2"}, ["A", "B"])
  end

  test "lists every missing key, sorted" do
    assert {:error, {:missing_keys, ["B", "C"]}} =
      ConfigLoader.require_keys(%{"A" => "1"}, ["C", "A", "B"])
  end
end

describe "load_file/1" do
  setup do
    path = Path.join(System.tmp_dir!(), "cfg_#{System.unique_integer([:positive])}.env")
    on_exit(fn -> File.rm(path) end)
    {:ok, path: path}
  end

  test "loads and parses", %{path: path} do
    File.write!(path, "A=1")
    assert {:ok, %{"A" => "1"}} = ConfigLoader.load_file(path)
  end
end

```

```

test "a missing file is an error tuple, not an exception", %{path: path} do
  assert {:error, :enoent} = ConfigLoader.load_file(path)
end

test "a malformed file reports the line", %{path: path} do
  File.write!(path, "A=1\nbroken")
  assert {:error, {:parse_error, 2, "broken"}} = ConfigLoader.load_file(path)
end

describe "boundary/1" do
  test "wraps a success" do
    assert {:ok, 4000} =
      ConfigLoader.boundary(fn -> ConfigLoader.fetch_integer!("#{P}" => "4000"}, "P")
      end)
  end

  test "converts each known exception" do
    assert {:error, {:missing_key, "P"}} =
      ConfigLoader.boundary(fn -> ConfigLoader.fetch!({}, "P") end)

    assert {:error, {:invalid_value, "P", "abc"}} =
      ConfigLoader.boundary(fn -> ConfigLoader.fetch_integer!("#{P}" => "abc"}, "P")
      end)

    assert {:error, {:parse_error, 1}} =
      ConfigLoader.boundary(fn -> ConfigLoader.parse!("broken") end)
  end

  test "lets an unknown exception through" do
    assert_raise RuntimeError, "unrelated bug", fn ->
      ConfigLoader.boundary(fn -> raise "unrelated bug" end)
    end
  end
end
end
end

```

```
mix test
```

```

Running ExUnit with seed: 840244, max_cases: 24
.....
Finished in 0.1 seconds (0.1s async, 0.00s sync)
Result: 38 passed (15 doctests, 23 tests)

```

Thirty-eight assertions, fifteen of them from the documentation.

What to point at in the debrief

"lets an unknown exception through" is the most valuable test in the suite. Ask a trainee to change `boundary/1` to a bare `rescue e -> {:error, e}` and rerun. That one test fails, and it is the only thing standing between the team and a boundary that silently swallows real bugs.

"raises a different exception for each failure" tests the distinction, not just the failure. A weaker test would have asserted that something raised. This one asserts which thing, and reads a field off it.

The module is `async: true`. Every file test uses a uniquely named path in the system temporary directory, and nothing touches application environment. Contrast this with the Lab 12 notifier tests, which could not be async. Ask which design is better and why.

Every `describe` block has an empty or missing input test. `parse("")`, `fetch(%{}, ...)`, `require_keys` with nothing missing, a missing file. That is the category trainees skip, and it is where the bugs are.

Stretch goal

Add a property-style test: generate a random map of keys and values, render it back to `KEY=value` text, parse it, and assert you get the original map. Round-trip tests find parser bugs that hand-written examples miss.

Lab 15 — Documentation, Typespecs and Quality Tooling

After Module 15 · 40 minutes

Objective

Bring `config_loader` up to production standard: complete documentation, typespecs that Dialyzer accepts, and a single `mix check` that a CI pipeline can run.

Steps

1. Stay in the `config_loader` project.
2. Add `ex_doc`, `credo` and `dialyxir` as development dependencies.
3. Add `@type`, `@typedoc` and `@spec` to every public function.
4. Add a `check` alias that runs formatter, compiler, Credo, tests and Dialyzer.
5. Run each tool and fix what it reports.

Requirements

- Every public function has a `@doc` with at least one example
- Every public function has a `@spec`
- `mix docs` generates without warnings
- `mix format --check-formatted` passes
- `mix compile --warnings-as-errors` passes
- `mix credo --strict` reports nothing you have not deliberately disabled
- `mix dialyzer` reports nothing

Hints

- `@type config :: %{optional(String.t()) => String.t()}` for a map with unknown string keys
- Use `String.t()` for text, never `binary()`
- The first `mix dialyzer` run builds a PLT and takes several minutes
- `{:credo, "~> 1.7", only: [:dev, :test], runtime: false}` keeps it out of production

Lab 15 Solution

Dependencies

```
defp deps do
```

```
[
  {:credo, "~> 1.7", only: [:dev, :test], runtime: false},
  {:dialyxir, "~> 1.4", only: [:dev], runtime: false},
  {:ex_doc, "~> 0.34", only: :dev, runtime: false}
]
end
```

All three are `runtime: false`, because they are tools, not libraries the application calls. All three are restricted by environment, so none reaches a production release.

The check alias

```
def project do
  [
    app: :config_loader,
    version: "0.1.0",
    elixir: "~> 1.20",
    name: "ConfigLoader",
    docs: [main: "ConfigLoader", extras: ["README.md"]],
    aliases: aliases(),
    deps: deps()
  ]
end

defp aliases do
  [
    setup: ["deps.get", "compile"],
    check: [
      "format --check-formatted",
      "compile --warnings-as-errors",
      "credo --strict",
      "test",
      "dialyzer"
    ]
  ]
end
```

```
mix setup
mix check
```

The specs, already written in Lab 13

The solution in Lab 13 already carries them. Read them with the room and justify each:

```
@type config :: %{optional(String.t()) => String.t()}

@spec parse(String.t()) :: {:ok, config()} | {:error, {:parse_error, pos_integer(), String.t()}}
@spec parse!(String.t()) :: config()
@spec fetch(config(), String.t()) :: {:ok, String.t()} | {:error, {:missing_key, String.t()}}
@spec fetch!(config(), String.t()) :: String.t()
@spec fetch_integer(config(), String.t()) ::
  {:ok, integer()}
  | {:error, {:missing_key, String.t()} | {:invalid_value, String.t(), String.t()}}
@spec require_keys(config(), [String.t()]) :: :ok | {:error, {:missing_keys, [String.t()]}}
@spec boundary((-> result)) :: {:ok, result} | {:error, tuple()} when result: term()
```

Three things to point out.

`optional(String.t())` in the `map` type says the keys are strings and any of them may be absent. Without `optional`, the type would demand that key.

`pos_integer()` for the line number documents that line numbers start at one. `integer()` would have allowed zero and negatives.

`boundary/1` uses a type variable. when `result: term()` says the success value has whatever type the function returned. That is how you spec a wrapper without losing the inner type.

Prove the tooling catches a real mistake

Add this function temporarily:

```
@spec broken() :: integer()
def broken, do: "not an integer"
```

`mix compile` accepts it. Then:

```
mix dialyzer
```

```
lib/config_loader.ex:12:invalid_contract
The @spec for the function does not match the success typing of the function.

Function:
ConfigLoader.broken/0

Success typing:
() :: <<_:112>>

But the spec is:
() :: integer()
```

"Success typing" is what Dialyzer inferred from the code. `<<_:112>>` is a 112-bit binary, which is the fourteen-character string. The spec claims integer. Delete the function and rerun; Dialyzer goes quiet.

This is the demonstration that makes typespecs feel worth writing. Without running the tool, a spec is a comment that can be wrong.

Run Credo

```
mix credo --strict
```

Real output from a small project:

```
Checking 3 source files ...

Consistency
|
| [C] -> File is using unix line endings while most of the files use windows line
|       endings.
|       test/qualitychk_test.exs:1 #(QualitychkTest)
```



```
Analysis took 0.1 seconds (0.07s to load, 0.09s running 69 checks on 3 files)
5 mods/funs, found 2 consistency issues.
```

Sixty-nine checks. On a mixed Windows and macOS team, that line-endings check is worth configuring rather than fixing file by file:

```
mix credo gen.config
```

Then edit `.credo.exs` to disable `Credo.Check.Consistency.LineEndings`, or fix `.gitattributes` so the repository normalises them. Either is defensible; deciding as a team is the point.

Generate the docs

```
mix docs
```

Open `doc/index.html`. Every `@doc` example appears, every `@spec` appears next to its function, and every `@type` gets its own entry. Ask the room to find `fetch_integer/2` and read its return type in the browser. That is what the annotations were for.

Discussion points

Ask which of the five tools in `mix check` would have caught each of these:

Mistake	Caught by
A typo'd variable name	<code>compile --warnings-as-errors</code>
A <code>@spec</code> that contradicts the code	<code>dialyzer</code>
An unreachable function clause	<code>compile</code> warning, and <code>dialyzer</code>
Inconsistent indentation	<code>format --check-formatted</code>
A function with six levels of nesting	<code>credo --strict</code>
A wrong return value	<code>test</code>

None of them replaces the others. That is why the alias runs all five.

End of Day 3 review

1. In Lab 11, name every function that touches the filesystem. Why are there so few?
2. In Lab 12, `function_exported?/3` returned false for a module that does export the function. Explain why, and what the fix was.
3. Which of your test modules are `async: true` and which are not? What does that tell you about each module's design?
4. In Lab 13, `fetch_integer!/2` raises two different exception types. Defend that choice to someone who would have used one.

5. Your `mix check` runs five tools. Name a mistake that only one of them would catch, for at least three of the five.

Day 4 Labs — Concurrency and OTP

Five labs covering Modules 16 to 20. Labs 18, 19 and 20 grow a single application, `sessions`, from one `GenServer` into a supervised, per-user, registry-backed service.

Every timing-sensitive test in these solutions was run on at least eight fixed seeds to confirm it is stable.

Lab 16 — A Key-Value Store as a Raw Process

After Module 16 · 40 minutes

Objective

Build a key-value store from `spawn`, `send` and `receive` alone, so that tomorrow's abstractions have something concrete to replace.

Requirements

- `KvProcess.start/0` returns a pid holding an empty map
- `put/3` and `delete/2` are asynchronous
- `get/3` and `size/2` are synchronous, with a timeout
- Every synchronous request carries a `make_ref/0` and the reply is matched on it
- Unknown messages are drained, not left in the mailbox
- `get_monitored/3` reports a dead store **immediately**, not after the timeout

Hints

- State is the argument to a tail-recursive `loop/1`
- `Process.monitor/1` delivers `{:DOWN, ref, :process, pid, reason}`
- Monitoring an already-dead pid delivers `:DOWN` with reason `:noproc` straight away
- `Process.demonitor(ref, [:flush])` also removes a `:DOWN` that already arrived

Lab 16 Solution

```
mix new kv_process
cd kv_process
```

```
# lib/kv_process.ex
defmodule KvProcess do
  @moduledoc """
  A key-value store held in a raw process, built from spawn, send and
  receive alone.

  Writes are asynchronous. Reads are synchronous and carry a unique
  reference so a late reply can never be mistaken for the current one.
  """

  @default_timeout 1_000

  @doc "Starts an empty store and returns its pid."
  def start, do: spawn(fn -> loop(%{}) end)

  @doc "Stores a value. Asynchronous: returns immediately."
```

```

def put(store, key, value) do
  send(store, {:put, key, value})
  :ok
end

@doc "Deletes a key. Asynchronous."
def delete(store, key) do
  send(store, {:delete, key})
  :ok
end

@doc """
Reads a key. Synchronous.

Returns `{:ok, value}`, `:error` when the key is absent, or
`:error, :timeout` when the store does not answer in time.
"""
def get(store, key, timeout \\ @default_timeout) do
  request(store, {:get, key}, timeout)
end

@doc "Returns the number of keys. Synchronous."
def size(store, timeout \\ @default_timeout) do
  case request(store, :size, timeout) do
    {:ok, n} -> n
    other -> other
  end
end

@doc "Stops the store."
def stop(store) do
  send(store, :stop)
  :ok
end

@doc """
Reads a key while monitoring the store, so a dead store is reported
immediately as `{:error, {:down, reason}}` rather than after a timeout.
"""
def get_monitored(store, key, timeout \\ @default_timeout) do
  monitor = Process.monitor(store)
  ref = make_ref()
  send(store, {:get, key, self(), ref})

  receive do
    {:reply, ^ref, reply} ->
      Process.demonitor(monitor, [:flush])
      reply

    {:DOWN, ^monitor, :process, ^store, reason} ->
      {:error, {:down, reason}}
  after
    timeout ->
      Process.demonitor(monitor, [:flush])
      {:error, :timeout}
  end
end

# ----- private

defp request(store, message, timeout) do

```

```

    ref = make_ref()
    send(store, envelope(message, self(), ref))

    receive do
      {:reply, ^ref, reply} -> reply
    after
      timeout -> {:error, :timeout}
    end
  end
end

# Adds the reply address and correlation reference to a request.
defp envelope({:get, key}, from, ref), do: {:get, key, from, ref}
defp envelope(:size, from, ref), do: {:size, from, ref}

defp loop(data) do
  receive do
    {:put, key, value} ->
      loop(Map.put(data, key, value))

    {:delete, key} ->
      loop(Map.delete(data, key))

    {:get, key, from, ref} ->
      reply =
        case Map.fetch(data, key) do
          {:ok, value} -> {:ok, value}
          :error -> :error
        end

      send(from, {:reply, ref, reply})
      loop(data)

    {:size, from, ref} ->
      send(from, {:reply, ref, {:ok, map_size(data)}})
      loop(data)

    :stop ->
      :ok

    _unexpected ->
      # Drain it; an unmatched message would slow every later receive.
      loop(data)
  end
end
end
end

```

A note from building this solution: the first draft appended to the request tuple with `Tuple.append/2`, and Elixir 1.20 warned that it is deprecated. Rather than silence the warning, `envelope/3` builds each message shape explicitly, which is also easier to read. Treat compiler warnings in course code as defects.

```

# test/kv_process_test.exs
defmodule KvProcessTest do
  use ExUnit.Case, async: true

  setup do
    store = KvProcess.start()
    on_exit(fn -> KvProcess.stop(store) end)
    {:ok, store: store}
  end
end

```

```

test "stores and reads a value", %{store: store} do
  KvProcess.put(store, :a, 1)
  assert {:ok, 1} = KvProcess.get(store, :a)
end

test "a missing key is :error, not a crash", %{store: store} do
  assert :error = KvProcess.get(store, :nope)
end

test "a later put overwrites", %{store: store} do
  KvProcess.put(store, :a, 1)
  KvProcess.put(store, :a, 2)
  assert {:ok, 2} = KvProcess.get(store, :a)
end

test "delete removes a key", %{store: store} do
  KvProcess.put(store, :a, 1)
  KvProcess.delete(store, :a)
  assert :error = KvProcess.get(store, :a)
end

test "size counts keys after async writes", %{store: store} do
  Enum.each(1..10, &KvProcess.put(store, &1, &1))
  # The synchronous size request is processed after all ten puts,
  # because messages from one process arrive in order.
  assert KvProcess.size(store) == 10
end

test "stores are isolated from each other", %{store: store} do
  other = KvProcess.start()
  on_exit(fn -> KvProcess.stop(other) end)

  KvProcess.put(store, :a, "mine")
  assert :error = KvProcess.get(other, :a)
end

test "unexpected messages do not kill the store", %{store: store} do
  send(store, :garbage)
  send(store, {:also, :garbage})
  KvProcess.put(store, :a, 1)
  assert {:ok, 1} = KvProcess.get(store, :a)
end

test "a dead store times out on a plain get" do
  store = KvProcess.start()
  KvProcess.stop(store)
  Process.sleep(20)

  assert {:error, :timeout} = KvProcess.get(store, :a, 50)
end

test "a dead store is reported immediately with a monitor" do
  store = KvProcess.start()
  KvProcess.stop(store)
  Process.sleep(20)

  {microseconds, result} = :timer.tc(fn -> KvProcess.get_monitored(store, :a, 5_000) end)

  assert {:error, {:down, :noproc}} = result
  assert div(microseconds, 1_000) < 100

```

```
end

test "concurrent writers from many processes lose nothing", %{store: store} do
  1..200
  |> Enum.map(fn i -> Task.async(fn -> KvProcess.put(store, i, i) end) end)
  |> Task.await_many()

  assert KvProcess.size(store) == 200
end
end
```

```
mix test
```

```
Running ExUnit with seed: 352092, max_cases: 24

.....
Finished in 0.1 seconds (0.1s async, 0.00s sync)

Result: 10 passed
```

Debrief

Compare the two "dead store" tests. The plain `get/3` waits out its full timeout before reporting failure. `get_monitored/3` is given five seconds and answers in under a hundred milliseconds, with reason `:noprocs`, because monitoring a pid that is already dead delivers `:DOWN` immediately. That difference is why `GenServer.call/3` monitors the server internally.

The size test depends on message ordering. Ten casts, then a call, all from one process. The call cannot be answered until the ten puts are processed. Ask what would change if the puts came from ten different processes. The answer is that there would be no ordering guarantee between them, which is why the concurrent test waits for every task with `Task.await_many/1` before checking.

Lab 17 — Parallel Health Checks

After Module 17 · 40 minutes

Objective

Run many health checks concurrently, bound the concurrency, enforce a per-check deadline, and survive checks that are slow, that return nonsense, or that raise.

Requirements

- `ParallelChecker.run/2` takes `[{name, fun}]` and returns `%{healthy: [name], unhealthy: [{name, reason}], elapsed_ms: n}`
- `max_concurrency:` and `timeout:` are configurable
- A slow check reports `:timeout`; a raising check reports `{:raised, message}`
- Neither kind of failure may kill the caller
- Results keep the order the checks were given in
- `run_one/2` uses the yield-then-shutdown idiom

Hints

- `Task.async_stream/3` with `on_timeout: :kill_task`
- `Task.async_stream/3` **links**: a raising function kills the caller unless you wrap it
- Results come back in input order by default, so `Enum.zip/2` with the checks recovers each name
- A `def` body can carry `rescue` directly

Lab 17 Solution

```
mix new parallel_checker
cd parallel_checker
```

```
# lib/parallel_checker.ex
defmodule ParallelChecker do
  @moduledoc """
  Runs health checks concurrently with bounded concurrency and a per-check
  deadline.

  A check is `{name, fun}` where `fun` returns `:ok` or `{:error, reason}`.
  A check may also be slow, or raise. Neither may take the caller down.
  """

  @default_opts [max_concurrency: 8, timeout: 500]

  @doc """
  Runs every check and returns a report.
```

```

    %{healthy: [name], unhealthy: [{name, reason}], elapsed_ms: n}

Names in both lists keep the order the checks were given in.
"""
def run(checks, opts \\ []) do
  opts = Keyword.merge(@default_opts, opts)
  started = System.monotonic_time(:millisecond)

  results =
    checks
    |> Task.async_stream(&run_check/1,
      max_concurrency: opts[:max_concurrency],
      timeout: opts[:timeout],
      on_timeout: :kill_task
    )
    |> Enum.zip(checks)
    |> Enum.map(fn
      {:_ok, result}, _check -> result
      {:_exit, :timeout}, {name, _fun} -> {name, {:_error, :timeout}}
    end)

  {healthy, unhealthy} = Enum.split_with(results, &match?({_name, :ok}, &1))

  %{
    healthy: Enum.map(healthy, fn {name, :ok} -> name end),
    unhealthy: Enum.map(unhealthy, fn {name, {:_error, reason}} -> {name, reason} end),
    elapsed_ms: System.monotonic_time(:millisecond) - started
  }
end

@doc "True when every check is healthy."
def all_healthy?(report), do: report.unhealthy == []

@doc """
Runs a single check with a deadline using the yield-then-shutdown idiom.
"""
def run_one({name, _fun} = check, timeout \\ 500) do
  task = Task.async(fn -> run_check(check) end)

  case Task.yield(task, timeout) || Task.shutdown(task, :brutal_kill) do
    {:_ok, result} -> result
    nil -> {name, {:_error, :timeout}}
  end
end

# The check is wrapped so it can never raise. Task.async_stream/3 links
# each task to the caller, so an unwrapped raise would kill the caller.
defp run_check({name, fun}) do
  case fun.() do
    :ok -> {name, :ok}
    {:_error, reason} -> {name, {:_error, reason}}
    other -> {name, {:_error, {:_unexpected_return, other}}}
  end
rescue
  exception -> {name, {:_error, {:_raised, Exception.message(exception)}}}
end
end

```

```

# test/parallel_checker_test.exs
defmodule ParallelCheckerTest do

```

```

use ExUnit.Case, async: true

defp ok(ms \\ 0), do: fn -> Process.sleep(ms); :ok end

test "all healthy" do
  report = ParallelChecker.run([{:db, ok()}, {:cache, ok()}])

  assert report.healthy == [:db, :cache]
  assert report.unhealthy == []
  assert ParallelChecker.all_healthy?(report)
end

test "reports an error result" do
  report = ParallelChecker.run([{:db, ok()}, {:queue, fn -> {:error, :unreachable} end}])

  assert report.healthy == [:db]
  assert report.unhealthy == [{:queue, :unreachable}]
  refute ParallelChecker.all_healthy?(report)
end

test "a slow check times out without killing the caller" do
  report = ParallelChecker.run([{:fast, ok(10)}, {:slow, ok(2_000)}], timeout: 100)

  assert report.healthy == [:fast]
  assert report.unhealthy == [{:slow, :timeout}]
end

test "a raising check is reported, not fatal" do
  report = ParallelChecker.run([{:boom, fn -> raise "disk on fire" end}, {:db, ok()}])

  assert report.healthy == [:db]
  assert report.unhealthy == [{:boom, {:raised, "disk on fire"}}]
end

test "an unexpected return value is unhealthy" do
  report = ParallelChecker.run([{:weird, fn -> :maybe end}])
  assert report.unhealthy == [{:weird, {:unexpected_return, :maybe}}]
end

test "checks run concurrently" do
  checks = for i <- 1..8, do: {i, ok(100)}
  report = ParallelChecker.run(checks, max_concurrency: 8)

  assert length(report.healthy) == 8
  # Eight 100ms checks in parallel must take far less than 800ms.
  assert report.elapsed_ms < 400
end

test "max_concurrency bounds parallelism" do
  checks = for i <- 1..4, do: {i, ok(100)}
  report = ParallelChecker.run(checks, max_concurrency: 1, timeout: 1_000)

  assert length(report.healthy) == 4
  assert report.elapsed_ms >= 400
end

test "an empty list is healthy" do
  report = ParallelChecker.run([])
  assert report.healthy == []
  assert ParallelChecker.all_healthy?(report)
end

```

```
describe "run_one/2" do
  test "returns a fast result" do
    assert {:db, :ok} = ParallelChecker.run_one({:db, ok()})
  end

  test "times out and cleans up" do
    assert {:slow, {:error, :timeout}} = ParallelChecker.run_one({:slow, ok(1_000)}, 50)
  end

  test "survives a raise" do
    assert {:boom, {:error, {:raised, "x"}}} =
      ParallelChecker.run_one({:boom, fn -> raise "x" end})
  end
end
```

```
mix test
```

```
Running ExUnit with seed: 56228, max_cases: 24

.....
Finished in 0.7 seconds (0.7s async, 0.00s sync)

Result: 11 passed
```

Debrief

The two timing tests bracket the behaviour from both sides. Eight checks at concurrency eight finish well under 400 ms. Four checks at concurrency one take at least 400 ms. Together they prove that **max_concurrency**: really is the dial, not just that things happen to be fast.

Delete the `rescue` from `run_check/1` and rerun. The "raising check" test does not fail politely; the test process itself dies with the **RuntimeError**. That is the task link from Module 17, and it is the most important thing to see with your own eyes today.

Lab 18 — A Session GenServer

After Module 18 · 45 minutes

Objective

Build a shopping session as a GenServer, with a client API, calls and casts chosen deliberately, and an idle timeout implemented with `handle_info/2`.

Requirements

- `Sessions.Session.add/3` validates the SKU against a price table and accumulates quantities
- `items/1` and `total/1` are calls; `touch/1` is a cast
- Any call or cast resets an idle timer; when it fires, the session stops with `:normal`
- A stale timer message must not stop a session that was active since
- A catch-all `handle_info/2` ignores unexpected messages
- Prices are read from `Sessions.Prices`, which Lab 20 turns into ETS; for now it can be a simple module

Hints

- `Process.send_after/3` returns a timer reference you can cancel with `Process.cancel_timer/1`
- Cancelling is racy: the message may already be in the mailbox. Tag each timer with `make_ref/0` and ignore messages whose ref does not match the current one
- `{:stop, :normal, state}` from `handle_info/2` ends the process cleanly

Lab 18 Solution

The complete application is shown under Lab 20, because Labs 19 and 20 extend this same project. The session module does not change between labs:

```
# lib/sessions/session.ex
defmodule Sessions.Session do
  @moduledoc """
  One shopping session per user.

  * `add/3` and `total/1` are calls, because the caller needs the answer
  * `touch/1` is a cast, because nobody waits on it
  * the session stops itself after an idle period, via `handle_info/2`
  """

  use GenServer, restart: :transient

  require Logger

  alias Sessions.Prices
```

```

@idle_timeout 5 * 60_000

# ----- client

def start_link(opts) do
  user_id = Keyword.fetch!(opts, :user_id)
  idle = Keyword.get(opts, :idle_timeout, @idle_timeout)
  GenServer.start_link(__MODULE__, {user_id, idle}, name: via(user_id))
end

def via(user_id), do: {:via, Registry, {Sessions.Registry, user_id}}

def add(session, sku, quantity), do: GenServer.call(session, {:add, sku, quantity})
def items(session), do: GenServer.call(session, :items)
def total(session), do: GenServer.call(session, :total)
def touch(session), do: GenServer.cast(session, :touch)

# ----- callbacks

@impl GenServer
def init({user_id, idle}) do
  state = %{user_id: user_id, items: %{}, idle: idle, timer: nil}
  {:ok, reset_idle(state)}
end

@impl GenServer
def handle_call({:add, sku, quantity}, _from, state)
  when is_integer(quantity) and quantity > 0 do
  case Prices.price(sku) do
    {:ok, _cents} ->
      items = Map.update(state.items, sku, quantity, &(&1 + quantity))
      state = reset_idle(%{state | items: items})
      {:reply, {:ok, items}, state}

    {:error, reason} ->
      {:reply, {:error, reason}, reset_idle(state)}
  end
end

def handle_call({:add, _sku, _quantity}, _from, state) do
  {:reply, {:error, :invalid_quantity}, state}
end

def handle_call(:items, _from, state), do: {:reply, state.items, reset_idle(state)}

def handle_call(:total, _from, state) do
  total =
    Enum.reduce(state.items, 0, fn {sku, quantity}, acc ->
      {:ok, cents} = Prices.price(sku)
      acc + cents * quantity
    end)

  {:reply, total, reset_idle(state)}
end

@impl GenServer
def handle_cast(:touch, state), do: {:noreply, reset_idle(state)}

@impl GenServer
def handle_info({:idle_timeout, ref}, %{timer: {ref, _}} = state) do
  # :normal exit plus restart: :transient means the supervisor lets it go.

```

```

    {:stop, :normal, state}
  end

  def handle_info({:idle_timeout, _stale_ref}, state) do
    # A timer that was replaced by a later reset. Ignore it.
    {:noreply, state}
  end

  def handle_info(message, state) do
    Logger.debug("session #{state.user_id} ignored #{inspect(message)}")
    {:noreply, state}
  end

  # ----- private

  defp reset_idle(%{timer: {_ref, timer}} = state) do
    Process.cancel_timer(timer)
    reset_idle(%{state | timer: nil})
  end

  defp reset_idle(%{timer: nil, idle: idle} = state) do
    ref = make_ref()
    timer = Process.send_after(self(), {:idle_timeout, ref}, idle)
    %{state | timer: {ref, timer}}
  end
end

```

Debrief

The first `handle_info/2` clause matches the `ref` twice. `%{timer: {ref, _}}` in the state pattern and `{:idle_timeout, ref}` in the message must hold the same `ref`, the repeated-variable technique from Day 1. A timer message from before the last reset carries an old `ref`, falls through to the second clause, and is ignored. Without the `ref`, a session touched one millisecond before its old timer fired would still be stopped.

`add/3` with `quantity: 0` replies with an error rather than crashing. A caller sent bad input, which is expected; it is not a broken assumption inside the server.

Lab 19 — Supervising the Sessions

After Module 19 · 45 minutes

Objective

Put sessions under supervision and prove, with tests, what supervision does and does not do.

Requirements

- An application tree starts the registry, the price table and a supervisor for sessions
- The top-level strategy is justified in a comment
- Sessions are `restart: :transient`
- A killed session is restarted, re-registered and has **empty** state
- A session that stops itself on idle is **not** restarted
- Killing one user's session leaves another user's session and state untouched

Hints

- `Process.exit(pid, :kill)` is an abnormal exit that cannot be trapped
- Wait for restarts by polling with a bound, not with a fixed sleep
- `restart: :transient` restarts after an abnormal exit only

Lab 19 Solution

```
# lib/sessions/application.ex
defmodule Sessions.Application do
  @moduledoc false

  use Application

  @impl true
  def start(_type, _args) do
    children = [
      {Registry, keys: :unique, name: Sessions.Registry},
      Sessions.Prices,
      {DynamicSupervisor, name: Sessions.SessionSupervisor, strategy: :one_for_one}
    ]

    # rest_for_one: sessions depend on the registry and the price table,
    # so if either restarts, every session must restart after it.
    Supervisor.start_link(children, strategy: :rest_for_one, name: Sessions.Supervisor)
  end
end
```

The supervision tests are in the "Lab 19: supervision" block of the test file under Lab 20.

Debrief

"a crashed session is restarted with empty state" is Module 19's central claim, tested again on different code. Three apples before the kill; an empty cart after.

"an idle stop is not restarted" is `:transient` doing its job. Change the session to use `GenServer`, `restart: :permanent`, rerun, and watch that test fail: an idle session keeps coming back forever. Choosing a restart type is a design decision with visible consequences.

Lab 20 — Per-User Sessions with Registry, DynamicSupervisor and ETS

After Module 20 · 45 minutes

Objective

Make sessions start on demand, one per user, found by user id, reading prices from an ETS table any session can read concurrently. Then build a release.

Requirements

- `Sessions.open/2` starts a session or returns the existing one, with no race
- Fifty concurrent opens for a new user produce exactly one process
- Prices live in a `:protected` ETS table: any process reads, only the owner writes
- A price change is visible to every session on its next total
- `MIX_ENV=prod mix release` produces a runnable release
- The release contains no source files and no Mix, and application code makes no `Mix.*` calls

Hints

- `{:via, Registry, {Sessions.Registry, user_id}}` as the GenServer name
- `{:error, {:already_started, pid}}` from `DynamicSupervisor.start_child/2` is success
- Writing to a `:protected` table from a non-owner raises `ArgumentError`
- `mix release` uses the current `MIX_ENV`; it does not switch to `:prod` for you
- Anything that varies by environment belongs in config, read with `Application.get_env/3`, not `Mix.env()`

Lab 20 Solution

```
# lib/sessions/prices.ex
defmodule Sessions.Prices do
  @moduledoc """
  Owns a protected ETS table of prices in minor units.

  `:protected` means any process may read, but only this owner may write,
  so writes go through a `call` while reads go straight to ETS.
  """

  use GenServer

  @table :session_prices

  @seed %{ "apple" => 40, "bread" => 120, "milk" => 65 }
```

```

def start_link(_opts), do: GenServer.start_link(__MODULE__, @seed, name: __MODULE__)

@doc "Reads a price directly from ETS, in the caller's process."
def price(sku) do
  case :ets.lookup(@table, sku) do
    [{^sku, cents}] -> {:ok, cents}
    [] -> {:error, :unknown_sku}
  end
end

@doc "Sets a price. Serialised through the owner process."
def set_price(sku, cents) when is_integer(cents) and cents >= 0 do
  GenServer.call(__MODULE__, {:set, sku, cents})
end

@impl GenServer
def init(seed) do
  table = :ets.new(@table, [:named_table, :set, :protected, read_concurrency: true])
  Enum.each(seed, fn {sku, cents} -> :ets.insert(table, {sku, cents}) end)
  {:ok, table}
end

@impl GenServer
def handle_call({:set, sku, cents}, _from, table) do
  :ets.insert(table, {sku, cents})
  {:reply, :ok, table}
end
end

```

```

# lib/sessions.ex
defmodule Sessions do
  @moduledoc """
  Public API for per-user shopping sessions.
  """

  alias Sessions.Session

  @doc "Starts a session for the user, or returns the existing one."
  def open(user_id, opts \\ []) do
    spec = {Session, Keyword.put(opts, :user_id, user_id)}

    case DynamicSupervisor.start_child(Sessions.SessionSupervisor, spec) do
      {:ok, pid} -> {:ok, pid}
      {:error, {:already_started, pid}} -> {:ok, pid}
      {:error, reason} -> {:error, reason}
    end
  end

  @doc "Finds a running session."
  def lookup(user_id) do
    case Registry.lookup(Sessions.Registry, user_id) do
      [{pid, _}] -> {:ok, pid}
      [] -> :error
    end
  end

  @doc "Adds an item to the user's session, opening it if needed."
  def add(user_id, sku, quantity) do
    with {:ok, _pid} <- open(user_id) do

```

```

    Session.add(Session.via(user_id), sku, quantity)
  end
end

@doc "The session total in minor units."
def total(user_id), do: Session.total(Session.via(user_id))

@doc "Closes a session."
def close(user_id) do
  case lookup(user_id) do
    {:ok, pid} -> DynamicSupervisor.terminate_child(Sessions.SessionSupervisor, pid)
    :error -> {:error, :not_found}
  end
end

@doc "Number of open sessions."
def count, do: DynamicSupervisor.count_children(Sessions.SessionSupervisor).active
end

```

The complete test suite for Labs 18 to 20

```

# test/sessions_test.exs
defmodule SessionsTest do
  use ExUnit.Case

  alias Sessions.{Prices, Session}

  @moduletag :capture_log

  setup do
    user = "user-#{System.unique_integer([:positive])}"
    on_exit(fn -> Sessions.close(user) end)
    {:ok, user: user}
  end

  describe "Lab 18: the Session GenServer" do
    test "adds items and accumulates quantities", %{user: user} do
      assert {:ok, %{"apple" => 2}} = Sessions.add(user, "apple", 2)
      assert {:ok, %{"apple" => 5}} = Sessions.add(user, "apple", 3)
    end

    test "rejects an unknown sku without changing state", %{user: user} do
      Sessions.add(user, "apple", 1)
      assert {:error, :unknown_sku} = Sessions.add(user, "caviar", 1)
      assert Session.items(Session.via(user)) == %{"apple" => 1}
    end

    test "rejects a non-positive quantity", %{user: user} do
      assert {:error, :invalid_quantity} = Sessions.add(user, "apple", 0)
    end

    test "totals from the price table", %{user: user} do
      Sessions.add(user, "apple", 2)
      Sessions.add(user, "bread", 1)
      assert Sessions.total(user) == 2 * 40 + 120
    end

    test "an unexpected message does not crash the session", %{user: user} do
      {:ok, pid} = Sessions.open(user)
    end
  end
end

```

```

    send(pid, :noise)
    assert Session.items(pid) == %{}
  end

  test "stops itself after the idle timeout", %{user: user} do
    {:ok, pid} = Sessions.open(user, idle_timeout: 50)
    ref = Process.monitor(pid)

    assert_receive {:DOWN, ^ref, :process, ^pid, :normal}, 500
  end

  test "activity postpones the idle timeout", %{user: user} do
    {:ok, pid} = Sessions.open(user, idle_timeout: 120)

    for _ <- 1..4 do
      Process.sleep(60)
      Session.touch(pid)
    end

    assert Process.alive?(pid)
  end
end

describe "Lab 19: supervision" do
  test "a crashed session is restarted with empty state", %{user: user} do
    Sessions.add(user, "apple", 3)
    {:ok, old} = Sessions.lookup(user)

    ref = Process.monitor(old)
    Process.exit(old, :kill)
    assert_receive {:DOWN, ^ref, :process, ^old, :killed}, 500

    new = wait_for(fn -> match_new(Sessions.lookup(user), old) end)
    assert new != old
    assert Session.items(new) == %{}
  end

  test "an idle stop is not restarted, because the session is transient", %{user: user} do
    {:ok, pid} = Sessions.open(user, idle_timeout: 30)
    ref = Process.monitor(pid)
    assert_receive {:DOWN, ^ref, :process, ^pid, :normal}, 500

    assert :error = wait_for(fn -> if Sessions.lookup(user) == :error, do: :error end)
  end

  test "one user's crash leaves another user's session alone", %{user: user} do
    other = user <> "-other"
    on_exit(fn -> Sessions.close(other) end)

    Sessions.add(user, "apple", 1)
    Sessions.add(other, "milk", 2)
    {:ok, victim} = Sessions.lookup(user)
    {:ok, bystander} = Sessions.lookup(other)

    Process.exit(victim, :kill)
    Process.sleep(50)

    assert {:ok, ^bystander} = Sessions.lookup(other)
    assert Session.items(bystander) == %{"milk" => 2}
  end
end
end

```

```

describe "Lab 20: Registry, DynamicSupervisor and ETS" do
  test "opening twice returns the same process", %{user: user} do
    {:ok, a} = Sessions.open(user)
    {:ok, b} = Sessions.open(user)
    assert a == b
  end

  test "concurrent opens for a new user create exactly one session", %{user: user} do
    pids =
      1..50
      |> Task.async_stream(fn _ -> Sessions.open(user) end, max_concurrency: 50)
      |> Enum.map(fn {:ok, {:_ok, pid}} -> pid end)
      |> Enum.uniq()

    assert length(pids) == 1
  end

  test "hundreds of sessions run side by side", %{user: user} do
    users = for i <- 1..300, do: "#{user}-#{i}"
    on_exit(fn -> Enum.each(users, &Sessions.close/1) end)

    before = Sessions.count()
    Enum.each(users, &Sessions.open/1)
    assert Sessions.count() == before + 300
  end

  test "a price change is visible to every session immediately", %{user: user} do
    Sessions.add(user, "milk", 1)
    original = Sessions.total(user)

    :ok = Prices.set_price("milk", 99)
    on_exit(fn -> Prices.set_price("milk", 65) end)

    assert Sessions.total(user) == 99
    assert original == 65
  end

  test "the price table is protected: only the owner may write" do
    assert_raise ArgumentError, fn -> :ets.insert(:session_prices, {"apple", 0}) end
  end
end

# ----- helpers

defp match_new({:_ok, pid}, old) when pid != old, do: pid
defp match_new(_, _), do: nil

defp wait_for(fun, attempts \\ 50) do
  case fun.() do
    nil when attempts > 0 -> Process.sleep(10) && wait_for(fun, attempts - 1)
    result -> result
  end
end
end

```

```

mix test

```

```

Running ExUnit with seed: 375939, max_cases: 24

```

```
.....
Finished in 0.6 seconds (0.00s async, 0.6s sync)

Result: 15 passed
```

The suite passed on eight consecutive fixed seeds.

Build the release

Set the environment explicitly. Without `MIX_ENV=prod`, `mix release` packages the development build.

```
MIX_ENV=prod mix release
```

On Windows PowerShell:

```
$env:MIX_ENV = "prod"; mix release
```

```
* assembling sessions-0.1.0 on MIX_ENV=prod
* skipping runtime configuration (config/runtime.exs not found)

Release created at _build/prod/rel/sessions
```

Inspect what was built

```
_build/prod/rel/sessions/
|-- bin/           sessions (Unix) and sessions.bat (Windows)
|-- erts-17.0.6/   the bundled Erlang runtime
|-- lib/           compiler, elixir, iex, kernel, logger, sasl, sessions, stdlib
`-- releases/     version info and boot scripts
```

The whole directory is 113 MB. Three things to confirm:

- `lib/sessions-0.1.0/ebin/` holds `Elixir.Sessions.beam`, `Elixir.Sessions.Application.beam`, `Elixir.Sessions.Prices.beam`, `Elixir.Sessions.Session.beam` and `sessions.app`. No `.ex` or `.exs` file exists anywhere in the release.
- There is no `mix` directory in `lib/`. Mix is a build tool and is not shipped.
- `erts-17.0.6/` is native code for the machine that built it. Build on the target's OS, CPU architecture and C library.

On Windows, run it with the batch launcher, and use PowerShell's stop-parsing token so the quotes reach Elixir intact:

```
& .\_build\prod\rel\sessions\bin\sessions.bat --% eval "IO.puts(Application.spec(:sessions, :vs))"
```

On macOS or Linux:

```
_build/prod/rel/sessions/bin/sessions eval 'IO.puts(Application.spec(:sessions, :vs))'
```

Both print `0.1.0`.

Prove Mix is not there

Under `mix run`, `Mix` is always loaded. In the release it is not:

```
& .\_build\prod\rel\sessions\bin\sessions.bat --% eval "IO.inspect(Code.ensure_loaded?(Mix))"
```

```
false
```

```
& .\_build\prod\rel\sessions\bin\sessions.bat --% eval "Mix.env()"
```

```
** (UndefinedFunctionError) function Mix.env/0 is undefined (module Mix is not available). Make
    sure the module name is correct and has been specified in full (or that an alias has been
        defined)
    Mix.env()
```

This is the crash any application code that calls `Mix.env()` would hit in production. On macOS or Linux, run the same expressions with `bin/sessions eval '...'`.

In PowerShell, `--%` passes the **whole rest of the line** through untouched, including any `;`. Put each release command on its own line.

Debrief

"concurrent opens for a new user create exactly one session" is the registry guarantee from Module 20, under real contention: fifty simultaneous opens, one pid.

"the price table is protected" shows ETS access control. Reads from any session work; a direct write from the test process raises `ArgumentError`. Writes must go through `Prices.set_price/2`, which runs in the owner. Change the table to `:public` and that test fails, which is the point: the access mode is part of the design.

The release is the Module 11 and Module 20 comparison made concrete: compiled `.beam` files and the Erlang runtime, with no source and no Mix. The `Mix.env()` crash is why environment decisions live in config, not in code.

End of Day 4 review

1. In Lab 16, why does `get_monitored/3` answer in under a hundred milliseconds when the store is dead, while `get/3` waits for its whole timeout?
2. In Lab 17, what exactly happens if you remove the `rescue` from `run_check/1`?
3. In Lab 18, why does every idle timer carry its own `make_ref/0`?
4. In Lab 19, which test fails if sessions become `restart: :permanent`, and why is that the right test to fail?
5. In Lab 20, fifty concurrent opens produced one process. Which component guarantees that, and how?

6. In Lab 20, `Code.ensure_loaded?(Mix)` returned `false` inside the release. What would happen to a module that called `Mix.env()` at runtime, and where should that decision live instead?

Day 5 Labs — Macros, Phoenix, Ecto, LiveView and the Capstone

Four labs covering Modules 21 to 24, and the capstone from Module 25.

Before starting, install the Phoenix generator once per machine:

```
mix archive.install hex phx_new
```

All web labs use SQLite, so no database server is needed.

Lab 21 — Macros

After Module 21 · 45 minutes

Objective

Write a timing macro, prove the double-evaluation bug with a test, and build a small validation DSL.

Requirements

- `MacroLab.Measure.measure/3` takes a label, a reporter function and a `do` block, calls the reporter with the elapsed microseconds, and returns the block's value
- `MacroLab.Square.bad_square/1` and `safe_square/1` differ only in whether the argument is evaluated once; a test proves it with a counter
- `use MacroLab.Validator` plus `validate :field, rule` declarations generate a real `validate/1` function and a `__validations__/0` reflection function
- Rules supported: `:required`, `{:format, regex}`, `{:min, number}`

Hints

- `defmacro measure(label, reporter, do: block)` captures the block
- `quote bind_quoted: [value: expression]` evaluates once
- `Module.register_attribute(__MODULE__, :validations, accumulate: true)`
- `@before_compile` runs after every declaration has been recorded
- Only use `Macro.escape/1` on a runtime **value** you are embedding, never on AST you received

Lab 21 Solution

```
mix new macro_lab
cd macro_lab
```

```
# lib/macro_lab/measure.ex
defmodule MacroLab.Measure do
  @moduledoc """
  A timing macro. It receives a block of code as AST, wraps it with timing
  code, and returns the block's own value.
  """

  @doc """
  Runs `block`, reports how long it took to `reporter`, and returns the
  block's value.

  ## Examples

      iex> require MacroLab.Measure
      iex> MacroLab.Measure.measure "sum", fn _label, _us -> :ok end do
```

```

...> Enum.sum(1..10)
...> end
55

"""
defmacro measure(label, reporter, do: block) do
  quote do
    started = System.monotonic_time(:microsecond)
    result = unquote(block)
    elapsed = System.monotonic_time(:microsecond) - started
    unquote(reporter).(unquote(label), elapsed)
    result
  end
end
end
end

```

```

# lib/macro_lab/square.ex
defmodule MacroLab.Square do
  @moduledoc """
  Two versions of the same macro, to show why evaluating an argument more
  than once is a bug.
  """

  @doc """
  Unsafe: inserts the argument's code twice, so it runs twice.
  """
  defmacro bad_square(expression) do
    quote do
      unquote(expression) * unquote(expression)
    end
  end

  @doc """
  Safe: `bind_quoted` evaluates the argument once and binds the result.
  """
  defmacro safe_square(expression) do
    quote bind_quoted: [value: expression] do
      value * value
    end
  end
end

```

```

# lib/macro_lab/validator.ex
defmodule MacroLab.Validator do
  @moduledoc """
  A tiny declarative validation DSL, built the way Ecto and Phoenix build
  theirs: `use` injects code, a macro records declarations in a module
  attribute at compile time, and `@before_compile` turns the collected
  declarations into a real function.
  """

  defmacro __using__(_opts) do
    quote do
      import MacroLab.Validator, only: [validate: 2]
      Module.register_attribute(__MODULE__, :validations, accumulate: true)
      @before_compile MacroLab.Validator
    end
  end
end

```

```

@doc "Declares one rule for one field. Runs at compile time."
defmacro validate(field, rule) do
  # `rule` arrives as AST. Unquoting it (without Macro.escape) means it is
  # evaluated in the caller at compile time, so `~r/@/` becomes a real
  # %Regex{} before it is stored. Escaping it would store the sigil's AST.
  quote do
    @validations {unquote(field), unquote(rule)}
  end
end

defmacro __before_compile__(env) do
  rules = env.module |> Module.get_attribute(:validations) |> Enum.reverse()

  quote do
    @doc "The rules declared in this module, in declaration order."
    def __validations__, do: unquote(Macro.escape(rules))

    @doc "Validates `params` against every declared rule."
    def validate(params) when is_map(params) do
      MacroLab.Validator.run(__validations__(), params)
    end
  end
end

@doc false
def run(rules, params) do
  errors =
    for {field, rule} <- rules,
      message = check(rule, Map.get(params, field)),
      message != nil,
      do: {field, message}

  case errors do
    [] -> :ok
    errors -> {:error, errors}
  end
end

defp check(:required, nil), do: "is required"
defp check(:required, ""), do: "is required"
defp check(:required, _value), do: nil

defp check({:format, _regex}, nil), do: nil

defp check({:format, regex}, value) when is_binary(value) do
  if Regex.match?(regex, value), do: nil, else: "has invalid format"
end

defp check({:min, _min}, nil), do: nil
defp check({:min, min}, value) when is_number(value) and value >= min, do: nil
defp check({:min, min}, _value), do: "must be at least #{min}"
end

```

```

# test/macro_lab_test.exs
defmodule MacroLabTest do
  use ExUnit.Case, async: true

  require MacroLab.Measure
  require MacroLab.Square

```

```

doctest MacroLab.Measure

defmodule SignupForm do
  use MacroLab.Validator

  validate :email, :required
  validate :email, {:format, ~r/@/}
  validate :age, {:min, 18}
end

describe "quote and unquote" do
  test "quote turns code into data" do
    assert quote(do: 1 + 2) == {:+, [context: MacroLabTest, imports: [{1, Kernel}, {2, Kernel}]], [1, 2]}
  end

  test "Macro.to_string turns AST back into source" do
    assert Macro.to_string(quote(do: Enum.map(list, &(&1 * 2)))) == "Enum.map(list, &(&1 * 2))"
  end

  test "unquote injects a value into generated code" do
    field = :email
    ast = quote(do: Map.get(user, unquote(field)))
    assert Macro.to_string(ast) == "Map.get(user, :email)"
  end

  test "literals quote to themselves" do
    assert quote(do: :ok) == :ok
    assert quote(do: "text") == "text"
    assert quote(do: {1, 2}) == {1, 2}
    assert quote(do: [1, 2]) == [1, 2]
  end
end

describe "measure/3" do
  test "returns the block's value and reports the timing" do
    parent = self()

    result =
      MacroLab.Measure.measure "work", fn label, us -> send(parent, {label, us}) end do
        Process.sleep(10)
        :done
      end

    assert result == :done
    assert_received {"work", microseconds}
    assert microseconds >= 10_000
  end

  test "expands into plain code" do
    ast =
      quote do
        MacroLab.Measure.measure("x", reporter, do: 1 + 2)
      end

    expanded = ast |> Macro.expand_once(__ENV__) |> Macro.to_string()

    assert expanded =~ "System.monotonic_time(:microsecond)"
    assert expanded =~ "1 + 2"
  end
end

```

```

end

describe "evaluating an argument once" do
  test "bad_square runs its argument twice" do
    {:ok, counter} = Agent.start_link(fn -> 0 end)
    next = fn -> Agent.get_and_update(counter, &{&1 + 1, &1 + 1}) end

    # 1 * 2, because the expression ran twice and returned 1 then 2
    assert MacroLab.Square.bad_square(next.()) == 2
    assert Agent.get(counter, & &1) == 2
  end

  test "safe_square runs its argument once" do
    {:ok, counter} = Agent.start_link(fn -> 0 end)
    next = fn -> Agent.get_and_update(counter, &{&1 + 1, &1 + 1}) end

    assert MacroLab.Square.safe_square(next.()) == 1
    assert Agent.get(counter, & &1) == 1
  end
end

describe "the validation DSL" do
  test "collects declarations in order at compile time" do
    assert [
      {:email, :required},
      {:email, {:format, %Regex{source: "@"}}},
      {:age, {:min, 18}}
    ] = SignupForm.__validations__()
  end

  test "valid params pass" do
    assert :ok = SignupForm.validate(%{email: "a@example.com", age: 30})
  end

  test "every failing rule is reported" do
    assert {:error, errors} = SignupForm.validate(%{email: "nope", age: 12})
    assert errors == [email: "has invalid format", age: "must be at least 18"]
  end

  test "a missing required field is reported once" do
    assert {:error, [email: "is required"]} = SignupForm.validate(%{age: 30})
  end

  test "use injected a real function" do
    assert function_exported?(SignupForm, :validate, 1)
  end
end

```

```

mix test

```

```

Running ExUnit with seed: 286656, max_cases: 24

```

```

.....
Finished in 0.2 seconds (0.2s async, 0.00s sync)

```

```

Result: 14 passed (1 doctest, 13 tests)

```

Debrief

Reproduce the bug this solution hit. Change `unquote(rule)` back to `unquote(Macro.escape(rule))` and run the tests. Three fail with `no function clause matching in Regex.match?/2`, and the first argument printed is `{:sigil_r, ...}`: the stored rule was code, not a regex. That error message is the best single lesson in when to use `Macro.escape/1`.

Compare the "declarations" test with the rules. The regex is matched by `%Regex{source: "@"}` rather than compared with `==`, because compiled regexes are not reliably equal by value. Pattern matching on the part you care about is the Module 04 answer.

Lab 22 — Phoenix: Adding Endpoints

After Module 22 · 45 minutes

Objective

Generate a Phoenix JSON API, then add two hand-written endpoints that follow the same structure as the generated code: thin controller, logic in the context, errors mapped in the fallback controller.

Setup

```
mix phx.new store_api --database sqlite3 --no-html --no-assets --no-gettext --no-mailer
--no-dashboard
cd store_api
mix deps.get
mix phx.gen.json Catalog Product products name:string sku:string:unique price_cents:integer
stock:integer
```

Add the generated resource to the router, then:

```
mix ecto.create
mix ecto.migrate
mix test
```

The two context functions used below, `Catalog.low_stock/1` and `Catalog.reserve_stock/2`, are built in Lab 23. If you are doing the labs in order, write them first; the Lab 23 solution lists them.

Requirements

- `GET /api/inventory/low?threshold=N` returns products at or below `N`, defaulting to 5
- `POST /api/inventory/:sku/reserve` with `{"quantity": n}` reserves stock
- Responses: 200 on success, 400 for a bad threshold or quantity, 404 for an unknown SKU, 409 for insufficient stock
- The controller contains no Ecto code and no status-code logic for errors

Hints

- Return `{:error, :bad_request}` from the controller and add a fallback clause for it
- `Catalog.reserve_stock/2` already returns `{:error, :not_found}` and `{:error, {:insufficient_stock, n}}`
- Query parameters are always strings; parse with `Integer.parse/1` and match `{n, ""}`

Lab 22 Solution

```
# lib/store_api_web/router.ex (the /api scope)
```

```

scope "/api", StoreApiWeb do
  pipe_through :api

  resources "/products", ProductController, except: [:new, :edit]
  get "/inventory/low", InventoryController, :low_stock
  post "/inventory/:sku/reserve", InventoryController, :reserve
end

```

```

# lib/store_api_web/controllers/inventory_controller.ex
defmodule StoreApiWeb.InventoryController do
  use StoreApiWeb, :controller

  alias StoreApi.Catalog

  action_fallback StoreApiWeb.FallbackController

  @default_threshold 5

  def low_stock(conn, params) do
    with {:ok, threshold} <- parse_threshold(Map.get(params, "threshold")) do
      render(conn, :low_stock, products: Catalog.low_stock(threshold), threshold: threshold)
    end
  end

  def reserve(conn, %{"sku" => sku, "quantity" => quantity}) do
    with {:ok, quantity} <- parse_positive(quantity),
         {:ok, product} <- Catalog.reserve_stock(sku, quantity) do
      render(conn, :reserved, product: product, quantity: quantity)
    end
  end

  def reserve(_conn, _params), do: {:error, :bad_request}

  defp parse_threshold(nil), do: {:ok, @default_threshold}
  defp parse_threshold(value), do: parse_non_negative(value)

  defp parse_non_negative(value) when is_integer(value) and value >= 0, do: {:ok, value}

  defp parse_non_negative(value) when is_binary(value) do
    case Integer.parse(value) do
      {n, ""} when n >= 0 -> {:ok, n}
      _ -> {:error, :bad_request}
    end
  end

  defp parse_non_negative(_), do: {:error, :bad_request}

  defp parse_positive(value) do
    case parse_non_negative(value) do
      {:ok, n} when n > 0 -> {:ok, n}
      _ -> {:error, :bad_request}
    end
  end
end

```

```

# lib/store_api_web/controllers/inventory_json.ex
defmodule StoreApiWeb.InventoryJSON do
  def low_stock(%{products: products, threshold: threshold}) do
    %{data: products, threshold: threshold}
  end
end

```

```

end

def reserved(%{product: product, quantity: quantity}) do
  %{data: %{sku: product.sku, reserved: quantity, remaining: product.stock}}
end
end

```

Two clauses added to the generated fallback controller, above the existing `:not_found` clause:

```

# lib/store_api_web/controllers/fallback_controller.ex
def call(conn, {:error, {:insufficient_stock, available}}) do
  conn
  |> put_status(:conflict)
  |> json(%{errors: %{stock: ["only #{available} available"]}})
end

def call(conn, {:error, :bad_request}) do
  conn
  |> put_status(:bad_request)
  |> json(%{errors: %{request: ["is invalid"]}})
end

```

```

# test/store_api_web/controllers/inventory_controller_test.exs
defmodule StoreApiWeb.InventoryControllerTest do
  use StoreApiWeb.ConnCase

  alias StoreApi.Catalog

  setup %{conn: conn} do
    {:ok, _} = Catalog.create_product(%{name: "Widget", sku: "W-1", price_cents: 500, stock: 3})
    {:ok, _} = Catalog.create_product(%{name: "Gadget", sku: "G-1", price_cents: 900, stock: 40})
    {:ok, conn: put_req_header(conn, "accept", "application/json")}
  end

  describe "GET /api/inventory/low" do
    test "uses the default threshold", %{conn: conn} do
      conn = get(conn, ~p"/api/inventory/low")

      assert %{ "threshold" => 5, "data" => [%{"sku" => "W-1", "stock" => 3}] } =
        json_response(conn, 200)
    end

    test "accepts a threshold", %{conn: conn} do
      conn = get(conn, ~p"/api/inventory/low?threshold=50")
      assert length(json_response(conn, 200)["data"]) == 2
    end

    test "rejects a non-numeric threshold", %{conn: conn} do
      conn = get(conn, ~p"/api/inventory/low?threshold=lots")
      assert json_response(conn, 400)["errors"]
    end
  end

  describe "POST /api/inventory/:sku/reserve" do
    test "reserves and reports what remains", %{conn: conn} do
      conn = post(conn, ~p"/api/inventory/W-1/reserve", quantity: 2)

      assert %{ "data" => %{ "sku" => "W-1", "reserved" => 2, "remaining" => 1 } } =

```

```

        json_response(conn, 200)
    end

    test "returns 409 when there is not enough stock", %{conn: conn} do
        conn = post(conn, ~p"/api/inventory/W-1/reserve", quantity: 10)
        assert json_response(conn, 409) == %{"errors" => %{"stock" => ["only 3 available"]}}
    end

    test "returns 404 for an unknown sku", %{conn: conn} do
        conn = post(conn, ~p"/api/inventory/NOPE/reserve", quantity: 1)
        assert json_response(conn, 404)
    end

    test "returns 400 for a bad quantity", %{conn: conn} do
        assert json_response(post(conn, ~p"/api/inventory/W-1/reserve", quantity: 0), 400)
        assert json_response(post(conn, ~p"/api/inventory/W-1/reserve", quantity: "x"), 400)
        assert json_response(post(conn, ~p"/api/inventory/W-1/reserve", %{}), 400)
    end
end
end
end

```

```
mix test
```

```

Running ExUnit with seed: 359340, max_cases: 24
.....
Finished in 0.1 seconds (0.1s async, 0.08s sync)
Result: 33 passed

```

Thirty-three: the generator's sixteen, Lab 23's ten, and these seven.

Debrief

The controller has no error handling of its own. Every failure is a tagged tuple that `with` passes through to the fallback controller, which is the only place that knows about HTTP status codes. Ask a trainee to add a new failure mode, such as a product that is discontinued, and point out that it takes one context clause and one fallback clause, and no controller change.

The 404 here is different from Module 22's. There, `get_product!/1` raised and Phoenix mapped the exception. Here, `reserve_stock/2` returned `{:error, :not_found}` and the generated fallback clause mapped the tuple. Ask which is appropriate when: a raise for "this should exist", a tuple for "this is a normal outcome the caller will handle".

Lab 23 — Ecto: Business Logic in the Context

After Module 23 · 45 minutes

Objective

Go beyond generated CRUD: add a query and a race-free stock reservation to the `Catalog` context, and test changeset behaviour directly.

Requirements

- `Catalog.low_stock/1` returns maps of `sku`, `name`, `stock` at or below a threshold, lowest first
- `Catalog.reserve_stock/2` decrements stock atomically and returns `{:ok, product}`, `{:error, :not_found}` or `{:error, {:insufficient_stock, available}}`
- The stock check must be **inside** the UPDATE statement, not a read followed by a write
- Tests prove string casting, an invalid cast, and the unique SKU constraint

Hints

- `from(p in Product, where: p.stock <= ^threshold, ...)`
- `Repo.update_all(query, inc: [stock: -quantity])` returns `{rows_changed, nil}`
- `Repo.rollback/1` inside `Repo.transaction/1` becomes `{:error, reason}`
- `errors_on/1` is provided by the generated `DataCase`

Lab 23 Solution

Added to the generated `lib/store_api/catalog.ex`:

```
@doc """
Products at or below `threshold` units, lowest stock first.

Written with Ecto's query DSL; nothing runs until `Repo.all/1`.
"""
def low_stock(threshold) when is_integer(threshold) do
  from(p in Product,
    where: p.stock <= ^threshold,
    order_by: [asc: p.stock, asc: p.name],
    select: %{sku: p.sku, name: p.name, stock: p.stock}
  )
  |> Repo.all()
end

@doc """
Reserves `quantity` units of the product with `sku`, atomically.

Returns `{:ok, %Product{}}` with the new stock, or
`{:error, :not_found}` or `{:error, {:insufficient_stock, available}}`.
```

The read and the write happen in one transaction, and the update is conditional on enough stock still being there, so two concurrent reservations cannot both succeed against the same last unit.

```
"""
def reserve_stock(sku, quantity) when is_integer(quantity) and quantity > 0 do
  Repo.transaction(fn ->
    query = from(p in Product, where: p.sku == ^sku and p.stock >= ^quantity)

    case Repo.update_all(query, inc: [stock: -quantity]) do
      {1, _} ->
        Repo.get_by!(Product, sku: sku)

      {0, _} ->
        case Repo.get_by(Product, sku: sku) do
          nil -> Repo.rollback(:not_found)
          product -> Repo.rollback({:insufficient_stock, product.stock})
        end
      end
    end
  end)
end
```

```
# test/store_api/catalog_business_test.exs
defmodule StoreApi.CatalogBusinessTest do
  use StoreApi.DataCase

  alias StoreApi.Catalog
  alias StoreApi.Catalog.Product

  defp product(attrs) do
    {:ok, product} =
      attrs
      |> Enum.into(%{name: "Widget", sku: "SKU-#{System.unique_integer([:positive])}",
        price_cents: 500, stock: 10})
      |> Catalog.create_product()

    product
  end

  describe "changeset validation" do
    test "requires every field" do
      changeset = Product.changeset(%Product{}, %{})

      refute changeset.valid?
      assert %{name: ["can't be blank"], sku: ["can't be blank"]} = errors_on(changeset)
    end

    test "casts string input from a form or JSON" do
      changeset =
        Product.changeset(%Product{}, %{name => "Pen", sku => "P1", price_cents => "250",
          stock => "3"})

      assert changeset.valid?
      assert Ecto.Changeset.get_change(changeset, :price_cents) == 250
    end

    test "rejects an uncastable integer" do
      changeset = Product.changeset(%Product{}, %{name: "Pen", sku: "P1", price_cents: "lots",
        stock: 1})
      assert %{price_cents: ["is invalid"]} = errors_on(changeset)
    end
  end
end
```

```

test "a duplicate sku is caught by the unique constraint" do
  product(sku: "DUP")
  assert {:error, changeset} = Catalog.create_product(%{name: "B", sku: "DUP", price_cents:
    1, stock: 1})
  assert %{sku: ["has already been taken"]} = errors_on(changeset)
end
end

describe "low_stock/1" do
  test "returns products at or below the threshold, lowest first" do
    product(name: "A", sku: "A", stock: 5)
    product(name: "B", sku: "B", stock: 1)
    product(name: "C", sku: "C", stock: 50)

    assert [%{sku: "B", stock: 1}, %{sku: "A", stock: 5}] = Catalog.low_stock(5)
  end

  test "returns an empty list when nothing is low" do
    product(stock: 100)
    assert Catalog.low_stock(3) == []
  end
end

describe "reserve_stock/2" do
  test "decrements stock and returns the updated product" do
    product(sku: "R1", stock: 10)

    assert {:ok, %Product{stock: 7}} = Catalog.reserve_stock("R1", 3)
    assert Repo.get_by!(Product, sku: "R1").stock == 7
  end

  test "refuses to go below zero and reports what is available" do
    product(sku: "R2", stock: 2)

    assert {:error, {:insufficient_stock, 2}} = Catalog.reserve_stock("R2", 3)
    assert Repo.get_by!(Product, sku: "R2").stock == 2
  end

  test "reports an unknown sku" do
    assert {:error, :not_found} = Catalog.reserve_stock("NOPE", 1)
  end

  test "can take the last unit exactly" do
    product(sku: "R3", stock: 1)
    assert {:ok, %Product{stock: 0}} = Catalog.reserve_stock("R3", 1)
    assert {:error, {:insufficient_stock, 0}} = Catalog.reserve_stock("R3", 1)
  end
end
end

```

Before the Lab 22 endpoints were added, the suite reported:

```

Running ExUnit with seed: 888083, max_cases: 24
.....
Finished in 0.1 seconds (0.1s async, 0.08s sync)
Result: 26 passed

```

Debrief

Rewrite `reserve_stock/2` the naive way — `get_by!`, check `stock >= quantity` in Elixir, then `Repo.update` — and run the tests. They still pass. That is the dangerous part: a sequential test suite cannot see a race. Discuss what test would expose it, and why the database-level condition makes the question go away.

Note the `database is locked` log line that SQLite may print during the run. It is SQLite's single-writer limit meeting concurrent test connections, it does not fail the suite, and it does not occur on PostgreSQL.

Lab 24 — LiveView

After Module 24 · 40 minutes

Objective

Build a live stock counter that stays in sync across browser tabs, and test it, including the two-browser case, without a browser.

Setup

```
mix phx.new shop_live --database sqlite3 --no-assets --no-gettext --no-mailer --no-dashboard
cd shop_live
mix deps.get
```

On Windows, compilation may warn that it failed to symlink `node_modules` for colocated assets. It is harmless for this lab.

Requirements

- `live "/stock"`, `StockLive` in the `:browser` scope
- Buttons reserve 1, reserve 5 and restock 5; over-reserving shows a message and changes nothing
- A change in one tab appears in every other tab through PubSub
- Subscribe only when the socket is connected
- Tests cover the static render, events, the refusal, and two simultaneous viewers

Hints

- `connected?(socket)` is false during the static render
- `Phoenix.PubSub.broadcast_from/4` skips the sender
- `phx-value-quantity` arrives as a string
- `live(conn, path)` returns `{:ok, view, html}`; `live(build_conn(), path)` gives you a second viewer

Lab 24 Solution

```
# lib/shop_live_web/router.ex (the browser scope)
scope "/", ShopLiveWeb do
  pipe_through :browser

  get "/", PageController, :home
  live "/stock", StockLive
end
```

```

# lib/shop_live_web/live/stock_live.ex
defmodule ShopLiveWeb.StockLive do
  @moduledoc """
  A live stock counter.

  Every connected browser shows the same level. A reservation in one tab
  is broadcast over PubSub and appears in every other tab, with no
  JavaScript written by hand.
  """

  use ShopLiveWeb, :live_view

  @topic "stock:widget"
  @initial_stock 10

  @impl true
  def mount(_params, _session, socket) do
    # mount/3 runs twice: once for the static HTML render, and again when
    # the browser's websocket connects. Subscribe only on the second.
    if connected?(socket), do: Phoenix.PubSub.subscribe(ShopLive.PubSub, @topic)

    {:ok, assign(socket, stock: @initial_stock, message: nil)}
  end

  @impl true
  def handle_event("reserve", %{quantity} => quantity, socket) do
    quantity = String.to_integer(quantity)

    if quantity <= socket.assigns.stock do
      new_stock = socket.assigns.stock - quantity
      broadcast(new_stock)
      {:noreply, assign(socket, stock: new_stock, message: "Reserved #{quantity}")}
    else
      {:noreply, assign(socket, message: "Only #{socket.assigns.stock} left")}
    end
  end

  def handle_event("restock", _params, socket) do
    new_stock = socket.assigns.stock + 5
    broadcast(new_stock)
    {:noreply, assign(socket, stock: new_stock, message: "Restocked 5")}
  end

  @impl true
  def handle_info({:stock_changed, stock}, socket) do
    {:noreply, assign(socket, stock: stock)}
  end

  @impl true
  def render(assigns) do
    ~H"""
    <div id="stock-panel">
      <h1>Widget stock</h1>
      <p id="level">In stock: {@stock}</p>
      <p :if={@message} id="message">{@message}</p>

      <button id="reserve-1" phx-click="reserve" phx-value-quantity="1">Reserve 1</button>
      <button id="reserve-5" phx-click="reserve" phx-value-quantity="5">Reserve 5</button>
      <button id="restock" phx-click="restock">Restock</button>
    </div>
    """
  end
end

```

```

    ""
  end

  defp broadcast(stock) do
    Phoenix.PubSub.broadcast_from(ShopLive.PubSub, self(), @topic, {:stock_changed, stock})
  end
end

```

```

# test/shop_live_web/live/stock_live_test.exs
defmodule ShopLiveWeb.StockLiveTest do
  use ShopLiveWeb.ConnCase

  import Phoenix.LiveViewTest

  test "the first render is plain HTML", %{conn: conn} do
    conn = get(conn, ~p"/stock")
    assert html_response(conn, 200) =~ "In stock: 10"
  end

  test "connects and shows the initial stock", %{conn: conn} do
    {:ok, view, html} = live(conn, ~p"/stock")

    assert html =~ "In stock: 10"
    assert has_element?(view, "#level", "In stock: 10")
  end

  test "reserving updates the page", %{conn: conn} do
    {:ok, view, _html} = live(conn, ~p"/stock")

    assert view |> element("#reserve-5") |> render_click() =~ "In stock: 5"
    assert has_element?(view, "#message", "Reserved 5")
  end

  test "over-reserving is refused and stock is unchanged", %{conn: conn} do
    {:ok, view, _html} = live(conn, ~p"/stock")

    view |> element("#reserve-5") |> render_click()
    view |> element("#reserve-5") |> render_click()
    html = view |> element("#reserve-5") |> render_click()

    assert html =~ "In stock: 0"
    assert html =~ "Only 0 left"
  end

  test "a change in one browser appears in another", %{conn: conn} do
    {:ok, alice, _} = live(conn, ~p"/stock")
    {:ok, bob, _} = live(build_conn(), ~p"/stock")

    alice |> element("#reserve-1") |> render_click()

    # Bob's LiveView receives the PubSub broadcast in handle_info/2.
    assert render(bob) =~ "In stock: 9"
  end
end

```

```

mix test

```

```

Running ExUnit with seed: 661497, max_cases: 24

```

```
.....  
Finished in 0.1 seconds (0.05s async, 0.08s sync)  
Result: 10 passed
```

Five are the generator's; five are above.

Debrief

Start the server and open `/stock` in two browser windows with `mix phx.server`. Click in one, watch the other update. Then point out that the test "a change in one browser appears in another" proved the same thing in milliseconds with no browser.

Spot the design flaw before revealing it. Stock lives in each tab's assigns. A fresh tab starts from 10 regardless of what others reserved, and two tabs clicking at the same instant can both succeed. The fix is to make `Catalog.reserve_stock/2` from Lab 23 the source of truth and broadcast what the database returned. That combination is the capstone of the follow-on Phoenix course.

Capstone — Ticketing Under a Stampede

Module 25 · 2 hours 20 minutes, including 20 minutes of paper design

The brief

Build `ticketing`. An event has a fixed number of seats. Five hundred buyers try to buy at once. Exactly the right number must succeed, a declined payment must not consume a seat, and a crashed event process must resume with the correct count.

Requirements by day

Day	Requirement
1–2	<code>Ticketing.Pricing.price/3</code> is pure: clauses, guards, <code>with</code> , doctests
3	<code>Ticketing.Order</code> struct, tagged errors, <code>purchase/4</code> and <code>purchase!/4</code> , a <code>PaymentGateway</code> behaviour with a fake chosen by configuration
4	One <code>EventServer</code> per event under a <code>DynamicSupervisor</code> , found through a <code>Registry</code>
5	Seat counts survive an <code>EventServer</code> crash because they are reloaded in <code>init/1</code>

The acceptance test

```
test "an event with 100 seats sells exactly 100 under a stampede", %{event: event} do
  {:ok, _} = Ticketing.create_event(event, seats: 100)

  results =
    1..500
    |> Task.async_stream(fn i -> Ticketing.purchase(event, "buyer-#{i}", 1) end,
      max_concurrency: 100
    )
    |> Enum.map(fn {:ok, result} -> result end)

  assert Enum.count(results, &match?({:ok, _}, &1)) == 100
  assert Enum.count(results, &match?({:error, :sold_out}, &1)) == 400
  assert Ticketing.remaining(event) == 0
end
```

Design review questions for the paper sketch

1. Where do seat counts live, such that an `EventServer` crash cannot lose them?
2. What serialises purchases for one event without serialising purchases for different events?
3. What is the top-level supervision strategy, and what restarts if the registry crashes?

Capstone Solution

```
mix new ticketing --sup
cd ticketing
```

The tree

```
# lib/ticketing/application.ex
defmodule Ticketing.Application do
  @moduledoc false

  use Application

  @impl true
  def start(_type, _args) do
    children = [
      # The seat store holds the durable copy of seat counts. It starts first
      # and lives outside every EventServer, so an EventServer crash cannot
      # lose seats. In production this would be Ecto, not an Agent.
      Ticketing.SeatStore,
      {Registry, keys: :unique, name: Ticketing.Registry},
      {DynamicSupervisor, name: Ticketing.EventSupervisor, strategy: :one_for_one}
    ]

    Supervisor.start_link(children, strategy: :rest_for_one, name: Ticketing.Supervisor)
  end
end
```

Days 1 and 2: pure pricing

```
# lib/ticketing/pricing.ex
defmodule Ticketing.Pricing do
  @moduledoc """
  Pure ticket pricing. No processes, no side effects: Days 1 and 2.
  Amounts are integer minor units.
  """

  @tier_prices %{standard: 5_000, vip: 15_000}
  @max_per_order 6

  defguard is_tier(tier) when tier in [:standard, :vip]
  defguard is_quantity(q) when is_integer(q) and q >= 1 and q <= @max_per_order

  @doc """
  Prices an order.

  ## Examples

      iex> Ticketing.Pricing.price(:standard, 2, nil)
      {:ok, 10_000}

      iex> Ticketing.Pricing.price(:vip, 1, "EARLY10")
      {:ok, 13_500}

      iex> Ticketing.Pricing.price(:vip, 7, nil)
      {:error, :invalid_quantity}
  """
end
```

```

iex> Ticketing.Pricing.price(:backstage, 1, nil)
{:error, :unknown_tier}

iex> Ticketing.Pricing.price(:standard, 1, "BOGUS")
{:error, :invalid_promo}

"""
def price(tier, quantity, promo) do
  with {:ok, unit} <- unit_price(tier),
       :ok <- check_quantity(quantity),
       {:ok, percent} <- discount(promo) do
    subtotal = unit * quantity
    {:ok, subtotal - div(subtotal * percent, 100)}
  end
end

defp unit_price(tier) when is_tier(tier), do: {:ok, Map.fetch!(@tier_prices, tier)}
defp unit_price(_tier), do: {:error, :unknown_tier}

defp check_quantity(quantity) when is_quantity(quantity), do: :ok
defp check_quantity(_quantity), do: {:error, :invalid_quantity}

defp discount(nil), do: {:ok, 0}
defp discount("EARLY10"), do: {:ok, 10}
defp discount("GROUP20"), do: {:ok, 20}
defp discount(_promo), do: {:error, :invalid_promo}
end

```

Day 3: structs, errors and a behaviour

```

# lib/ticketing/order.ex
defmodule Ticketing.Order do
  @moduledoc "A confirmed ticket order. Day 3: a struct with enforced keys."

  @enforce_keys [:event_id, :buyer, :quantity, :amount_cents]
  defstruct [:event_id, :buyer, :quantity, :amount_cents, tier: :standard, status: :confirmed]

  @type t :: %__MODULE__{
    event_id: String.t(),
    buyer: String.t(),
    quantity: pos_integer(),
    amount_cents: non_neg_integer(),
    tier: :standard | :vip,
    status: :confirmed
  }
end

defmodule Ticketing.SoldOutError do
  defexception [:event_id, :remaining]

  @impl true
  def message(%{event_id: id, remaining: remaining}),
    do: "event #{id} cannot fill the order, #{remaining} seats remain"
end

```

```

# lib/ticketing/payment_gateway.ex
defmodule Ticketing.PaymentGateway do
  @moduledoc "Contract for charging a buyer. Day 3: a behaviour chosen by configuration."

```

```

@callback charge(buyer :: String.t(), amount_cents :: non_neg_integer()) ::
  :ok | {:error, term()}

@doc "The configured implementation."
def impl, do: Application.get_env(:ticketing, :payment_gateway, Ticketing.PaymentGateway.Fake)
end

defmodule Ticketing.PaymentGateway.Fake do
  @moduledoc """
  Approves everyone except buyers whose id starts with `"declined"`.
  """

  @behaviour Ticketing.PaymentGateway

  @impl Ticketing.PaymentGateway
  def charge("declined" <> _rest, _amount), do: {:error, :card_declined}
  def charge(_buyer, _amount), do: :ok
end

```

Day 4: a process per event

```

# lib/ticketing/seat_store.ex
defmodule Ticketing.SeatStore do
  @moduledoc """
  The durable copy of remaining seats, keyed by event id.

  It is a separate supervised process from every EventServer, so a crashed
  EventServer can reload its count. A real system would use Ecto here; the
  course uses an Agent so the capstone needs no database.
  """

  use Agent

  def start_link(_opts), do: Agent.start_link(fn -> %{} end, name: __MODULE__)

  def fetch(event_id) do
    case Agent.get(__MODULE__, &Map.fetch(&1, event_id)) do
      {:ok, seats} -> {:ok, seats}
      :error -> {:error, :not_found}
    end
  end

  def put(event_id, seats), do: Agent.update(__MODULE__, &Map.put(&1, event_id, seats))

  def exists?(event_id), do: Agent.get(__MODULE__, &Map.has_key?(&1, event_id))
end

```

```

# lib/ticketing/event_server.ex
defmodule Ticketing.EventServer do
  @moduledoc """
  One process per event: Day 4.

  All purchases for an event are serialised through this process, one
  message at a time, which is what makes overselling impossible without
  any lock. Every successful purchase is written to the seat store before
  replying, so a restart reloads the right count.
  """

  use GenServer

```



```

alias Ticketing.{Order, PaymentGateway, Pricing, SeatStore}

def start_link(event_id) do
  GenServer.start_link(__MODULE__, event_id, name: via(event_id))
end

def via(event_id), do: {:via, Registry, {Ticketing.Registry, event_id}}

@impl GenServer
def init(event_id) do
  # Reload from the durable store: a restarted server resumes, not resets.
  {:ok, seats} = SeatStore.fetch(event_id)
  {:ok, %{event_id: event_id, remaining: seats}}
end

@impl GenServer
def handle_call(:remaining, _from, state), do: {:reply, state.remaining, state}

def handle_call({:purchase, buyer, quantity, tier, promo}, _from, state) do
  with {:ok, amount} <- Pricing.price(tier, quantity, promo),
       :ok <- check_seats(state.remaining, quantity),
       :ok <- PaymentGateway.impl().charge(buyer, amount) do
    remaining = state.remaining - quantity
    :ok = SeatStore.put(state.event_id, remaining)

    order = %Order{
      event_id: state.event_id,
      buyer: buyer,
      quantity: quantity,
      amount_cents: amount,
      tier: tier
    }

    {:reply, {:ok, order}, %{state | remaining: remaining}}
  else
    {:error, reason} -> {:reply, {:error, reason}, state}
  end
end

@impl GenServer
def handle_cast(:crash, _state), do: raise("deliberate crash")

defp check_seats(remaining, quantity) when remaining >= quantity, do: :ok
defp check_seats(_remaining, _quantity), do: {:error, :sold_out}
end

```

The public API

```

# lib/ticketing.ex
defmodule Ticketing do
  @moduledoc """
  Public API for the capstone ticketing service.
  """

  alias Ticketing.{EventServer, SeatStore}

  @doc "Creates an event with a number of seats and starts its server."
  def create_event(event_id, seats: seats) when is_integer(seats) and seats >= 0 do

```

```

    if SeatStore.exists?(event_id) do
      {:error, :already_exists}
    else
      SeatStore.put(event_id, seats)
      start_server(event_id)
    end
  end
end

@doc """
Buys tickets. Options: `:tier` (default `:standard`) and `:promo`.

Returns `{:ok, %Ticketing.Order{}}` or `{:error, reason}`.
"""
def purchase(event_id, buyer, quantity, opts \\ []) do
  tier = Keyword.get(opts, :tier, :standard)
  promo = Keyword.get(opts, :promo)

  with {:ok, _pid} <- ensure_server(event_id) do
    GenServer.call(EventServer.via(event_id), {:purchase, buyer, quantity, tier, promo})
  end
end

@doc "Like `purchase/4`, but raises when the order cannot be filled."
def purchase!(event_id, buyer, quantity, opts \\ []) do
  case purchase(event_id, buyer, quantity, opts) do
    {:ok, order} ->
      order

    {:error, :sold_out} ->
      raise Ticketing.SoldOutError, event_id: event_id, remaining: remaining(event_id)

    {:error, reason} ->
      raise ArgumentError, "purchase failed: #{inspect(reason)}"
  end
end

@doc "Seats left for an event."
def remaining(event_id) do
  with {:ok, _pid} <- ensure_server(event_id) do
    GenServer.call(EventServer.via(event_id), :remaining)
  end
end

@doc "Finds a running event server."
def whereis(event_id) do
  case Registry.lookup(Ticketing.Registry, event_id) do
    [{pid, _}] -> {:ok, pid}
    [] -> :error
  end
end

# An event that exists in the store but whose server is not running (for
# example after a node restart) is started on demand.
defp ensure_server(event_id) do
  if SeatStore.exists?(event_id), do: start_server(event_id), else: {:error, :not_found}
end

defp start_server(event_id) do
  case DynamicSupervisor.start_child(Ticketing.EventSupervisor, {EventServer, event_id}) do
    {:ok, pid} -> {:ok, pid}
    {:error, {:already_started, pid}} -> {:ok, pid}
  end
end

```

```

        {:error, reason} -> {:error, reason}
      end
    end
  end
end

```

The tests

```

# test/ticketing_test.exs
defmodule TicketingTest do
  use ExUnit.Case

  doctest Ticketing.Pricing

  alias Ticketing.{EventServer, Order}

  @moduletag :capture_log

  setup do
    {:ok, event: "gig-#{System.unique_integer([:positive])}}"}
  end

  describe "acceptance: the stampede" do
    test "an event with 100 seats sells exactly 100 under a stampede", %{event: event} do
      {:ok, _} = Ticketing.create_event(event, seats: 100)

      results =
        1..500
        |> Task.async_stream(fn i -> Ticketing.purchase(event, "buyer-#{i}", 1) end,
          max_concurrency: 100
        )
        |> Enum.map(fn {:ok, result} -> result end)

      assert Enum.count(results, &match?({:ok, _}, &1)) == 100
      assert Enum.count(results, &match?({:error, :sold_out}, &1)) == 400
      assert Ticketing.remaining(event) == 0
    end
  end

  describe "purchasing" do
    test "returns a priced order and decrements seats", %{event: event} do
      {:ok, _} = Ticketing.create_event(event, seats: 10)

      assert {:ok, %Order{quantity: 2, amount_cents: 27_000, tier: :vip}} =
        Ticketing.purchase(event, "priya", 2, tier: :vip, promo: "EARLY10")

      assert Ticketing.remaining(event) == 8
    end

    test "cannot buy more than remain", %{event: event} do
      {:ok, _} = Ticketing.create_event(event, seats: 3)
      assert {:error, :sold_out} = Ticketing.purchase(event, "priya", 4)
      assert Ticketing.remaining(event) == 3
    end

    test "a pricing error consumes no seats", %{event: event} do
      {:ok, _} = Ticketing.create_event(event, seats: 3)
      assert {:error, :invalid_promo} = Ticketing.purchase(event, "priya", 1, promo: "BOGUS")
      assert Ticketing.remaining(event) == 3
    end
  end
end

```

```

test "a declined payment consumes no seats", %{event: event} do
  {:ok, _} = Ticketing.create_event(event, seats: 3)
  assert {:error, :card_declined} = Ticketing.purchase(event, "declined-card", 1)
  assert Ticketing.remaining(event) == 3
end

test "an unknown event is an error, not a crash" do
  assert {:error, :not_found} = Ticketing.purchase("no-such-gig", "priya", 1)
end

test "an event cannot be created twice", %{event: event} do
  {:ok, _} = Ticketing.create_event(event, seats: 1)
  assert {:error, :already_exists} = Ticketing.create_event(event, seats: 1)
end

test "purchase! raises a structured exception when sold out", %{event: event} do
  {:ok, _} = Ticketing.create_event(event, seats: 1)
  Ticketing.purchase!(event, "first", 1)

  error = assert_raise Ticketing.SoldOutError, fn -> Ticketing.purchase!(event, "second",
    1) end
  assert error.remaining == 0
end

describe "fault tolerance" do
  test "a crashed event server resumes with the persisted seat count", %{event: event} do
    {:ok, old} = Ticketing.create_event(event, seats: 10)
    {:ok, _} = Ticketing.purchase(event, "priya", 4)

    ref = Process.monitor(old)
    GenServer.cast(EventServer.via(event), :crash)
    assert_receive {:DOWN, ^ref, :process, ^old, _}, 500

    new = wait_for_new_pid(event, old)
    assert new != old
    # Unlike Day 4's stock server, state was reloaded rather than reset.
    assert Ticketing.remaining(event) == 6
  end

  test "one event's crash leaves other events untouched", %{event: event} do
    other = event <> "-other"
    {:ok, victim} = Ticketing.create_event(event, seats: 5)
    {:ok, bystander} = Ticketing.create_event(other, seats: 5)

    ref = Process.monitor(victim)
    GenServer.cast(victim, :crash)
    assert_receive {:DOWN, ^ref, :process, _, _}, 500

    assert {:ok, ^bystander} = Ticketing.whereis(other)
  end
end

defp wait_for_new_pid(event, old, attempts \\ 50) do
  case Ticketing.whereis(event) do
    {:ok, pid} when pid != old -> pid
    _ when attempts > 0 -> Process.sleep(10) && wait_for_new_pid(event, old, attempts - 1)
  end
end

```

```
end
```

```
mix test
```

```
Running ExUnit with seed: 164833, max_cases: 24
.....
Finished in 0.1 seconds (0.00s async, 0.1s sync)
Result: 15 passed (5 doctests, 10 tests)
```

The suite, including the stampede test, passed on eight consecutive fixed seeds.

Answers to the design review questions

Where do seat counts live? In `Ticketing.SeatStore`, a separate supervised process. Each successful purchase writes the new count there **before** replying. When an `EventServer` crashes, the `DynamicSupervisor` restarts it, `init/1` reads the store, and the count is correct. The fault tolerance test proves it: 10 seats, 4 sold, crash, 6 remain.

Contrast this with Day 4's stock server, where a crash reset the count. The difference is one line in `init/1` and one line in the purchase path.

What serialises purchases? Each `EventServer` handles one message at a time, so the check `remaining >= quantity` and the decrement cannot interleave for one event. Different events have different processes, so they run in parallel. That is why the stampede sells exactly one hundred and not one hundred and three, with no lock anywhere.

Top-level strategy? `:rest_for_one`, with the store first, then the registry, then the event supervisor. If the registry crashes, every event server's name is gone, so the event supervisor restarts after it. Because the counts live in the store, which started first and is untouched, every event resumes with the correct seats.

Stretch goals

- Replace `SeatStore` with an Ecto repository and `Catalog`-style conditional update from Lab 23, so the counts survive a whole node restart
- Add a `LiveView` at `/events/:id` that shows remaining seats updating as the stampede runs
- Add `POST /api/events/:id/purchase` using the Lab 22 fallback-controller pattern, returning 409 when sold out

End of Day 5 review

1. In Lab 21, what exactly did `Macro.escape/1` store, and why did that break `Regex.match?/2`?
2. In Lab 22, trace a 409 response from the controller to the JSON body. Which modules does it pass through?
3. In Lab 23, why would a naive read-check-write reservation still pass the test suite, and why is it still wrong?
4. In Lab 24, name every process involved when Alice's click updates Bob's page.

5. In the capstone, why is the stampede count exactly one hundred, and what single design decision makes a crash resume rather than reset?